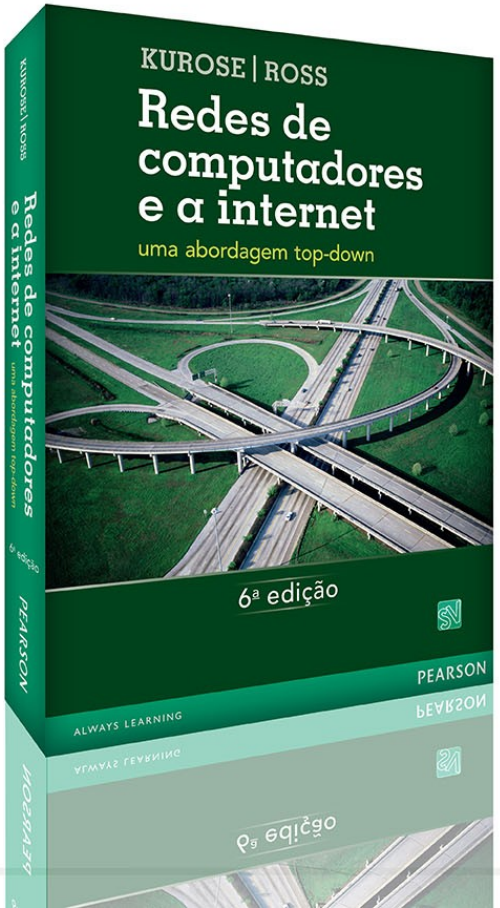
Capítulo 4

A camada de REDE



Redes de computadores I

Prof.: Leandro Soares de Sousa

E-mail: leandro.uff.puro@gmail.com

Site: http://www.ic.uff.br/~lsousa

Não deixem a matéria acumular!!!

Datas das avaliações, exercícios propostos, transparências,... no site!

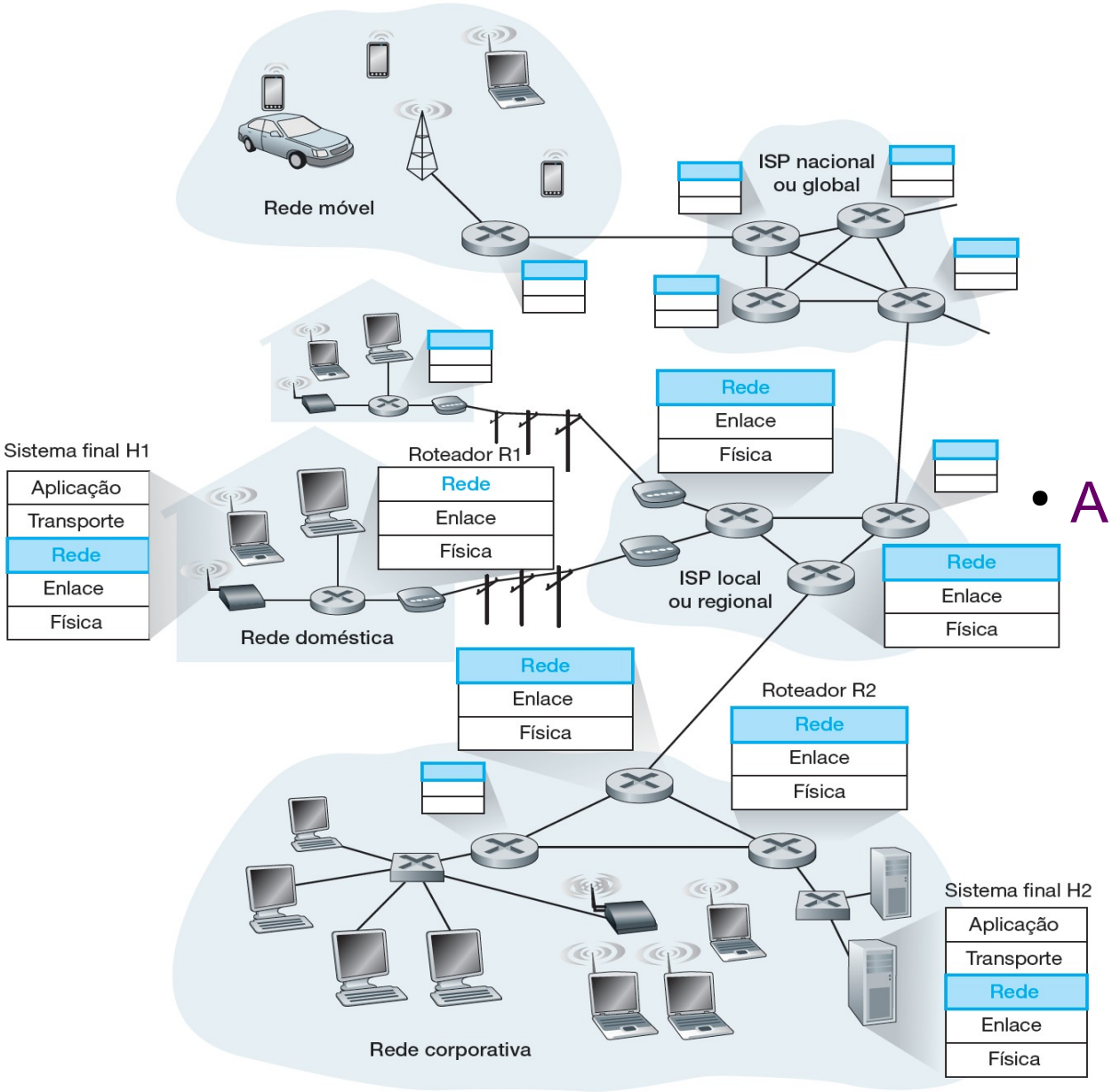
Sumário

4.1 – Introdução
4.2 – Redes de circuitos virtuais e de datagramas
4.3 – O que há dentro de um roteador?
4.4 – O Protocolo da Internet (IP): repasse e endereçamento na Internet
4.5 – Algoritmos de roteamento
4.6 – Roteamento na Internet
4.7 – Roteamento por difusão e para um grupo

Sumário

4.1 – Introdução
4.2 – Redes de circuitos virtuais e de datagramas
4.3 – O que há dentro de um roteador?
4.4 – O Protocolo da Internet (IP): repasse e endereçamento na Internet
4.5 – Algoritmos de roteamento
4.6 – Roteamento na Internet
4.7 – Roteamento por difusão e para um grupo

Introdução

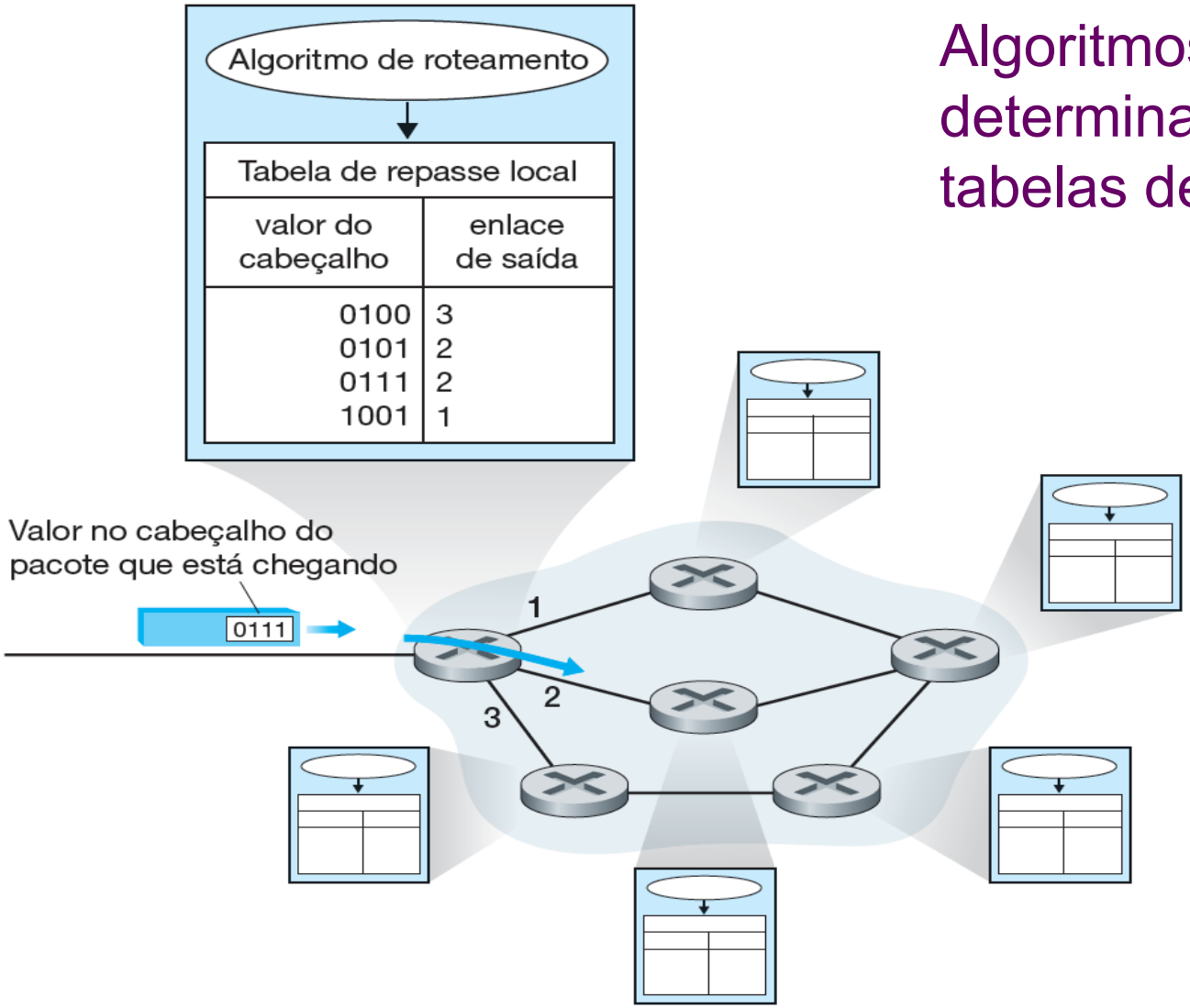


- A camada de rede

Repasse e roteamento

- O papel da camada de rede é transportar pacotes de um hospedeiro remetente a um hospedeiro destinatário.
- **Repasse**. Quando um pacote chega ao enlace de entrada de um roteador, este deve conduzi-lo até o enlace de saída apropriado.
- **Roteamento**. A camada de rede deve determinar a rota ou o caminho tomado pelos pacotes ao fluírem de um remetente a um destinatário.

Repasse e roteamento



Algoritmos de roteamento determinam valores em tabelas de repasse:

Modelos de serviço de rede

- O **modelo de serviço de rede** define as características do transporte de dados fim a fim entre uma borda da rede e a outra.

Alguns serviços específicos que poderiam ser oferecidos são:

- Entrega garantida.
- Entrega garantida com atraso limitado.
- Entrega de pacotes na ordem.
- Largura de banda mínima garantida.
- Jitter máximo garantido.
- Serviços de segurança.

Modelos de serviço de rede

- Modelos de serviço das redes Internet, ATM CBR e ATM ABR

Arquitetura da rede	Modelo de serviço	Garantia de largura de banda	Garantia contra perda	Ordenação	Temporização	Indicação de congestionamento
Internet	Melhor esforço	Nenhuma	Nenhuma	Qualquer ordem possível	Não mantida	Nenhuma
ATM	CBR	Taxa constante garantida	Sim	Na ordem	Mantida	Não haverá congestionamento
ATM	ABR	Mínima garantida	Nenhuma	Na ordem	Não mantida	Indicação de congestionamento

Sumário

4.1 – Introdução
4.2 – Redes de circuitos virtuais e de datagramas
4.3 – O que há dentro de um roteador?
4.4 – O Protocolo da Internet (IP): repasse e endereçamento na Internet
4.5 – Algoritmos de roteamento
4.6 – Roteamento na Internet
4.7 – Roteamento por difusão e para um grupo

Redes de circuitos virtuais

- Um circuito virtual (CV) consiste em:

1. um caminho (isto é, uma série de enlaces e roteadores) entre hospedeiros de origem e de destino,

2. números de CVs, um número para cada enlace ao longo do caminho e

3. registros na tabela de repasse em cada roteador ao longo do caminho.

Redes de circuitos virtuais

- Uma rede de circuitos virtuais simples:

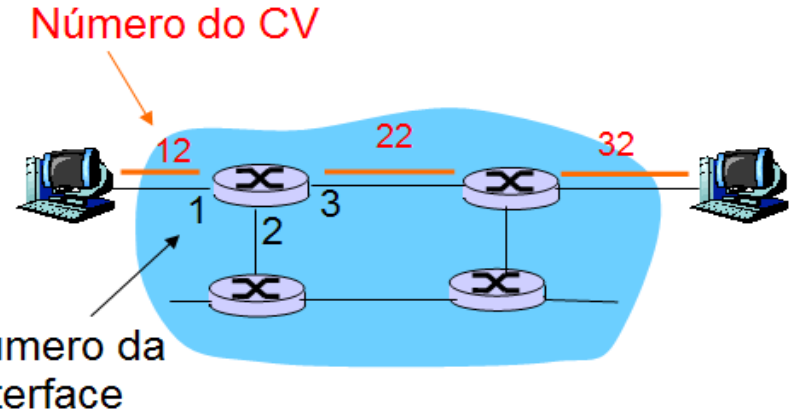


Tabela de encaminhamento no roteador noroeste:

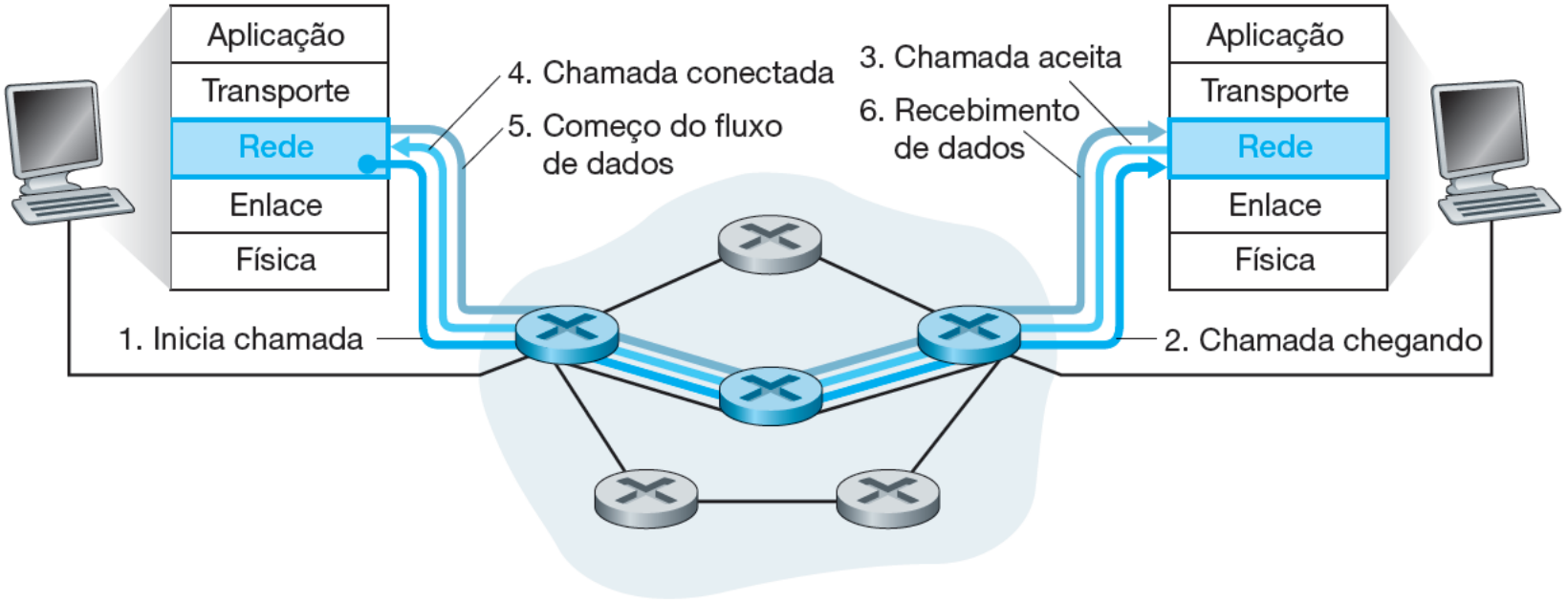
Interface de entrada	# CV de entrada	Interface de saída	# CV de saída
1	12	3	22
2	63	1	18
3	7	2	17
1	97	3	87
…	…	…	…

Roteadores mantêm informação sobre o estado da conexão!

Redes de circuitos virtuais

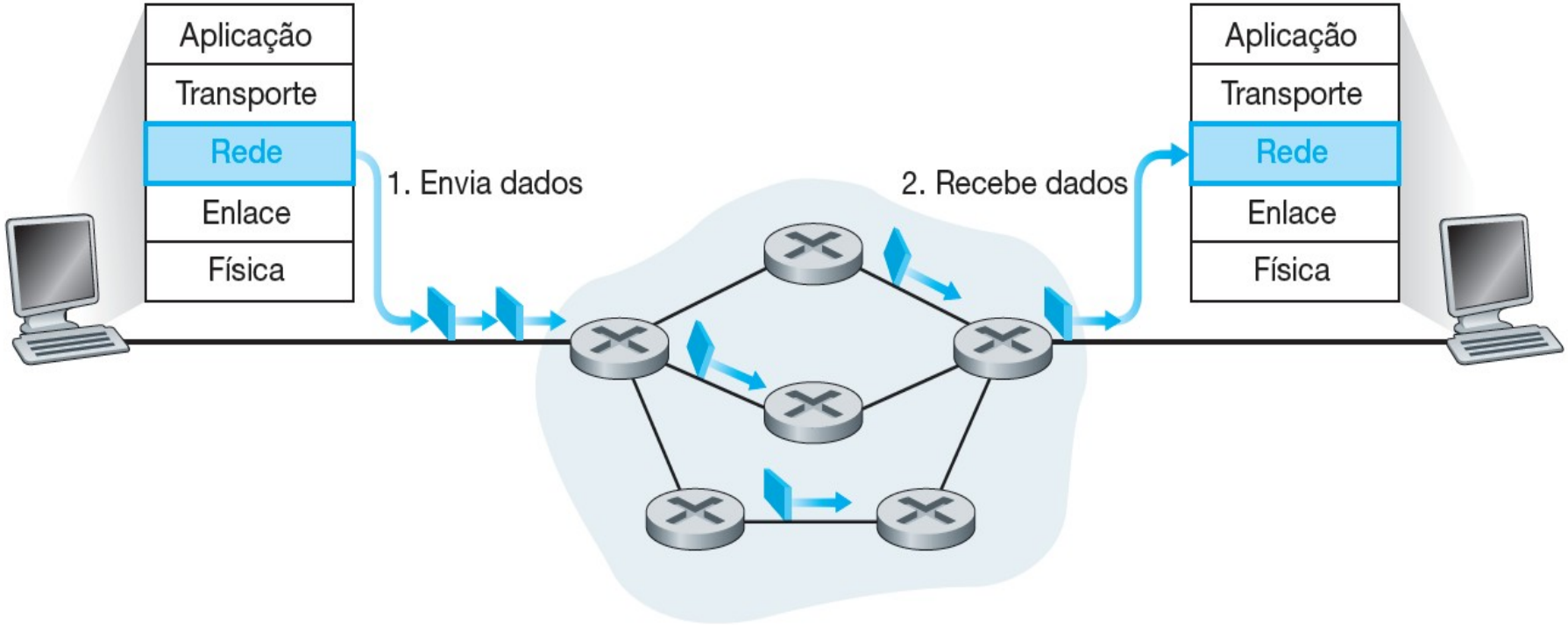
- Há três fases que podem ser identificadas em um circuito virtual:

1. Estabelecimento de CV.
2. Transferência de dados.
3. Encerramento do CV.



Redes de datagramas

- Em uma rede de datagramas, toda vez que um sistema final quer enviar um pacote, ele marca o pacote com o endereço do sistema final de destino e então o envia para dentro da rede.



Redes de datagramas

- Ao ser transmitido da origem ao destino, um pacote passa por uma série de roteadores.
- Cada um desses roteadores usa o endereço de destino do pacote para repassá-lo.
- Então, o roteador transmite o pacote para aquela interface de enlace de saída.
- A tabela de repasse de um roteador em uma rede de CVs é modificada sempre que é estabelecida uma nova conexão através do roteador ou sempre que uma conexão existente é desativada.

Redes de datagramas

Faixa de Endereços de Destino	Interface de Saída
11001000 00010111 00010000 00000000 a 11001000 00010111 00010111 11111111	0
11001000 00010111 00011000 00000000 a 11001000 00010111 00011000 11111111	1
11001000 00010111 00011001 00000000 a 11001000 00010111 00011111 11111111	2
caso contrário	3

4 bilhões de entradas possíveis

Redes de datagramas

- Casamento com o prefixo mais longo:

Casamento com o prefixo	Interface de Saída
11001000 00010111 00010	0
11001000 00010111 00011000	1
11001000 00010111 00011	2
caso contrário	3

Exemplos

ED: 11001000 00010111 00010110 10100001 Qual interface?

ED: 11001000 00010111 00011000 10101010 Qual interface?

Redes de datagramas

- Rede de datagramas ou CVs: por quê?

- **Internet:**
- troca de dados entre computadores
  - serviço “elástico”, sem reqs. temporais estritos
- sistemas terminais “inteligentes” (computadores)
  - podem se adaptar, exercer controle, recuperar de erros
  - núcleo da rede simples, complexidade na “borda”
- muitos tipos de enlaces
  - características diferentes
  - serviço uniforme difícil

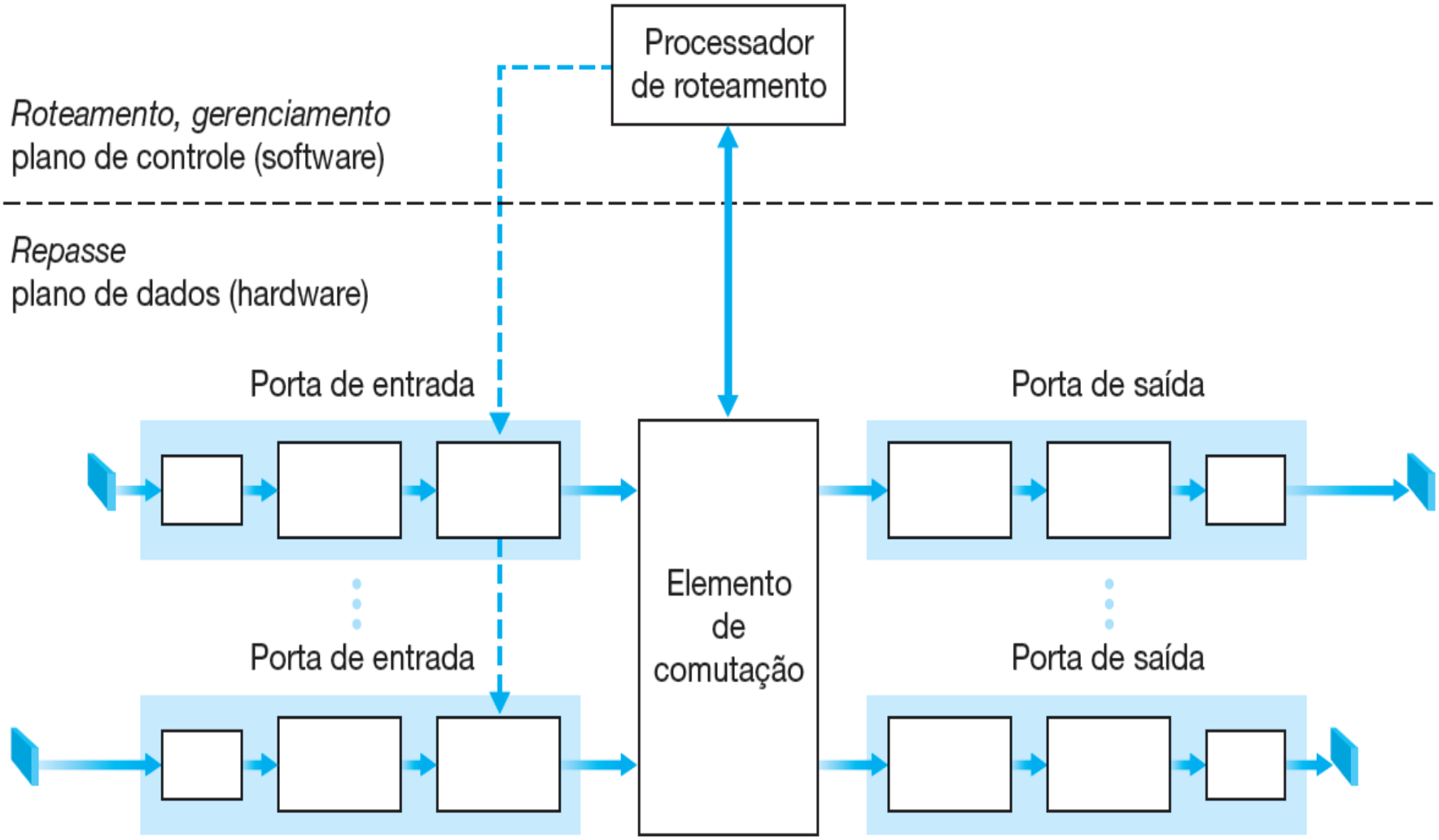
- **ATM:**
- evoluiu da telefonia
- conversação humana:
  - temporização estrita, requisitos de confiabilidade
  - requer serviço garantido
- sistemas terminais “burros”
  - telefones
  - complexidade dentro da rede

Sumário

4.1 – Introdução
4.2 – Redes de circuitos virtuais e de datagramas
4.3 – O que há dentro de um roteador?
4.4 – O Protocolo da Internet (IP): repasse e endereçamento na Internet
4.5 – Algoritmos de roteamento
4.6 – Roteamento na Internet
4.7 – Roteamento por difusão e para um grupo

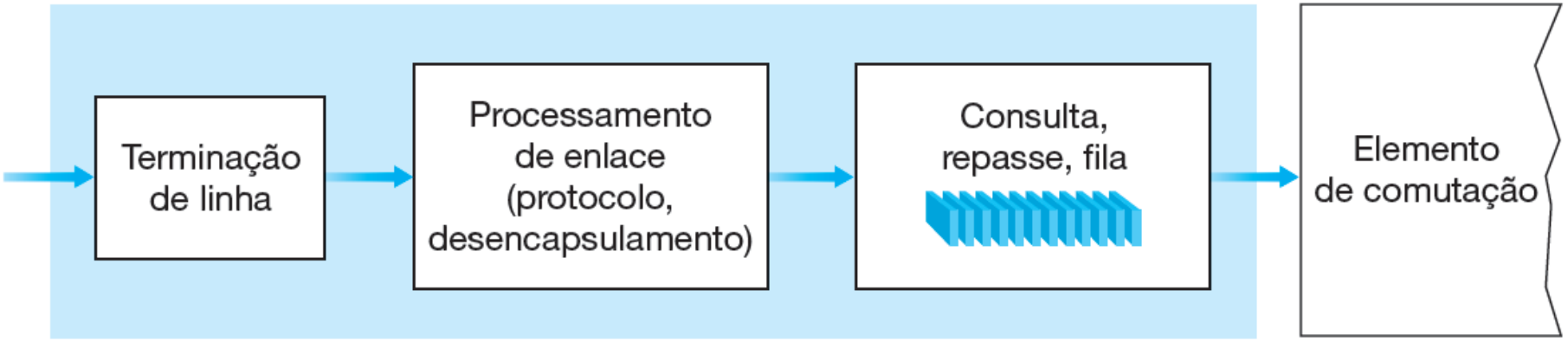
O que há dentro de um roteador?

- Arquitetura de roteador



Processamento de entrada

- Processamento na porta de entrada



Camada Física

Camada de Enlace, ex.: Ethernet, 802.11...

Comutação descentralizada:

- dado o destino do datagrama, procura porta de saída usando tab. de rotas na memória da porta de entrada
- **meta**: completar processamento da porta de entrada na 'velocidade da linha'
- **filas**: se datagramas chegam mais rápido que taxa de reenvio para matriz de comutação

Elemento de comutação

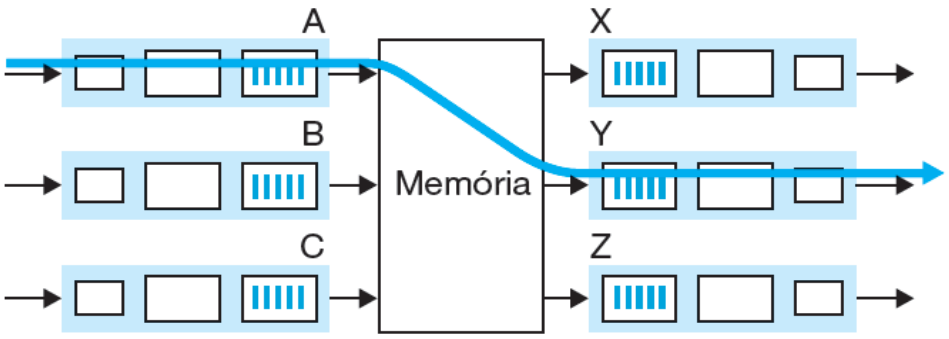
É por meio do elemento de comutação que os pacotes são comutados de uma porta de entrada para uma porta de saída.

A comutação pode ser realizada de inúmeras maneiras:

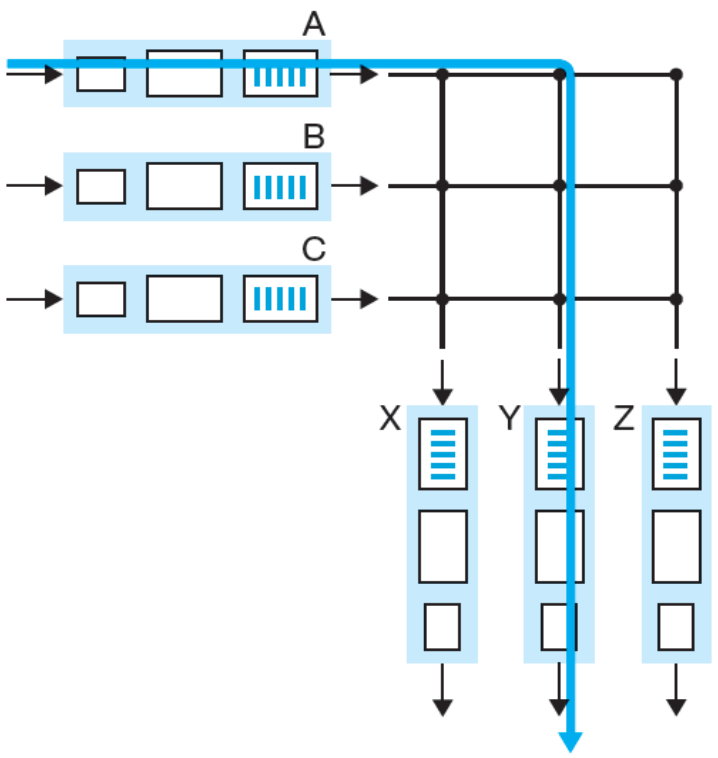
- Comutação por memória.
- Comutação por um barramento.
- Comutação por uma rede de interconexão.

Elemento de comutação

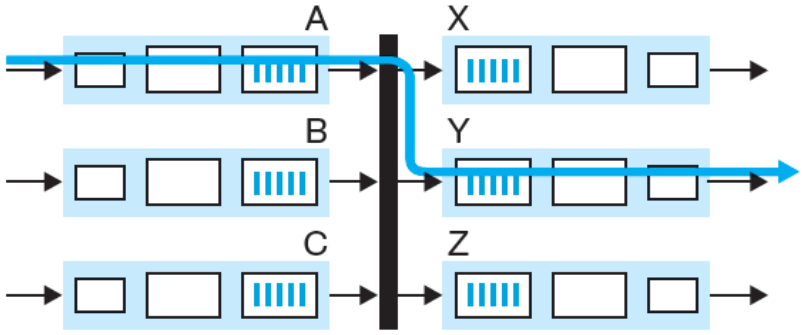
Memória



Rede de interconexão



Barramento



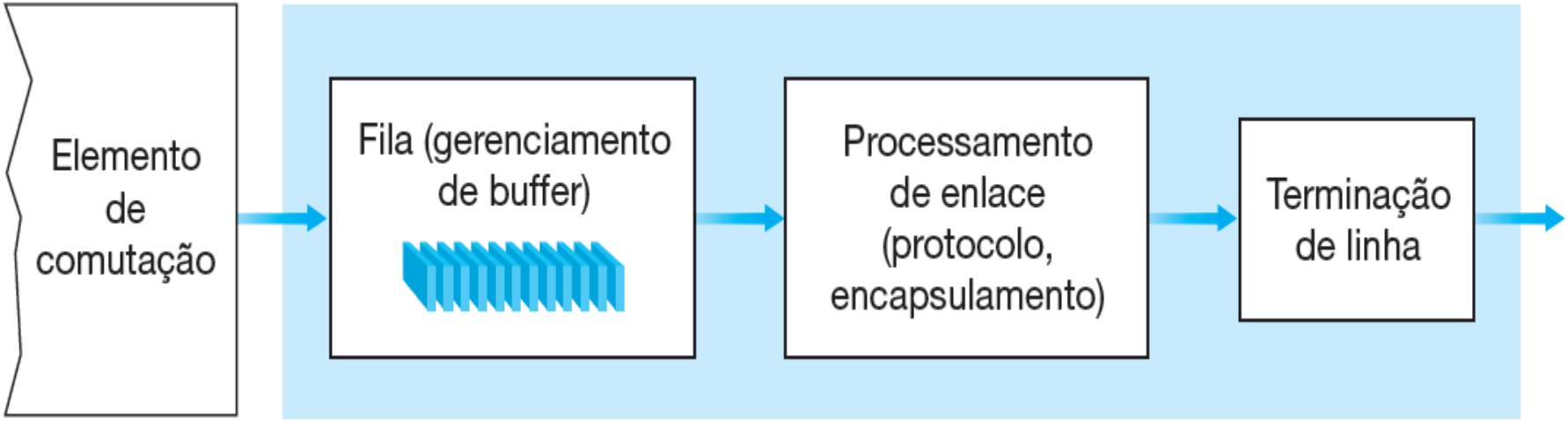
Legenda:



Rápido?
Lento?
Banda?
Interferência?

Processamento de saída

- Processamento de porta de saída



- **Buffers** necessários quando datagramas chegam da matriz de comutação mais rapidamente que a taxa de transmissão
- **Disciplina de escalonamento** escolhe um dos datagramas enfileirados para transmissão

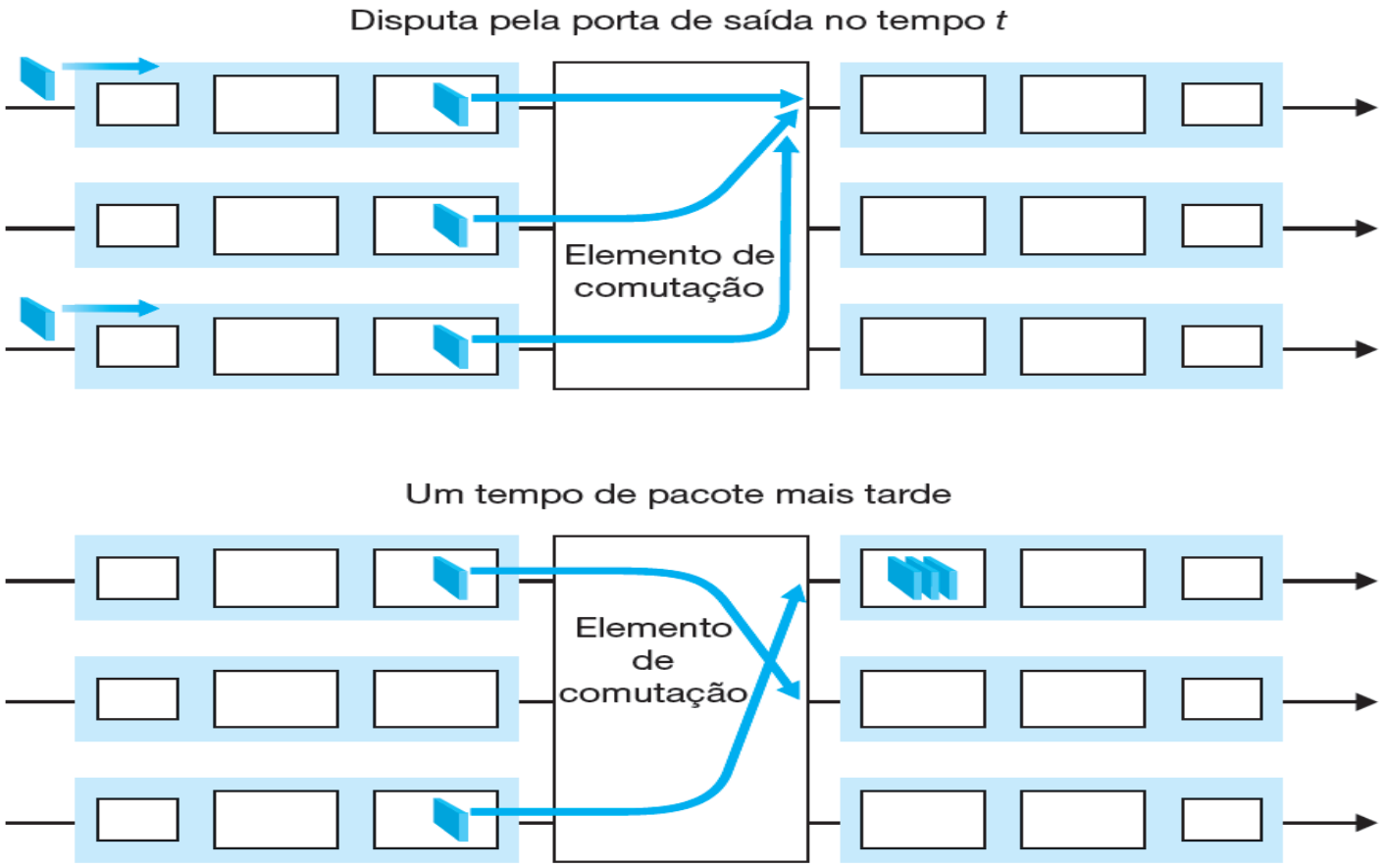
Onde ocorre formação de fila?

Filas de pacotes podem se formar tanto nas portas de entrada como nas de saída.

O local e a extensão da formação de fila dependerão:

- da carga de tráfego,
- da velocidade relativa do elemento de comutação e
- da taxa da linha.

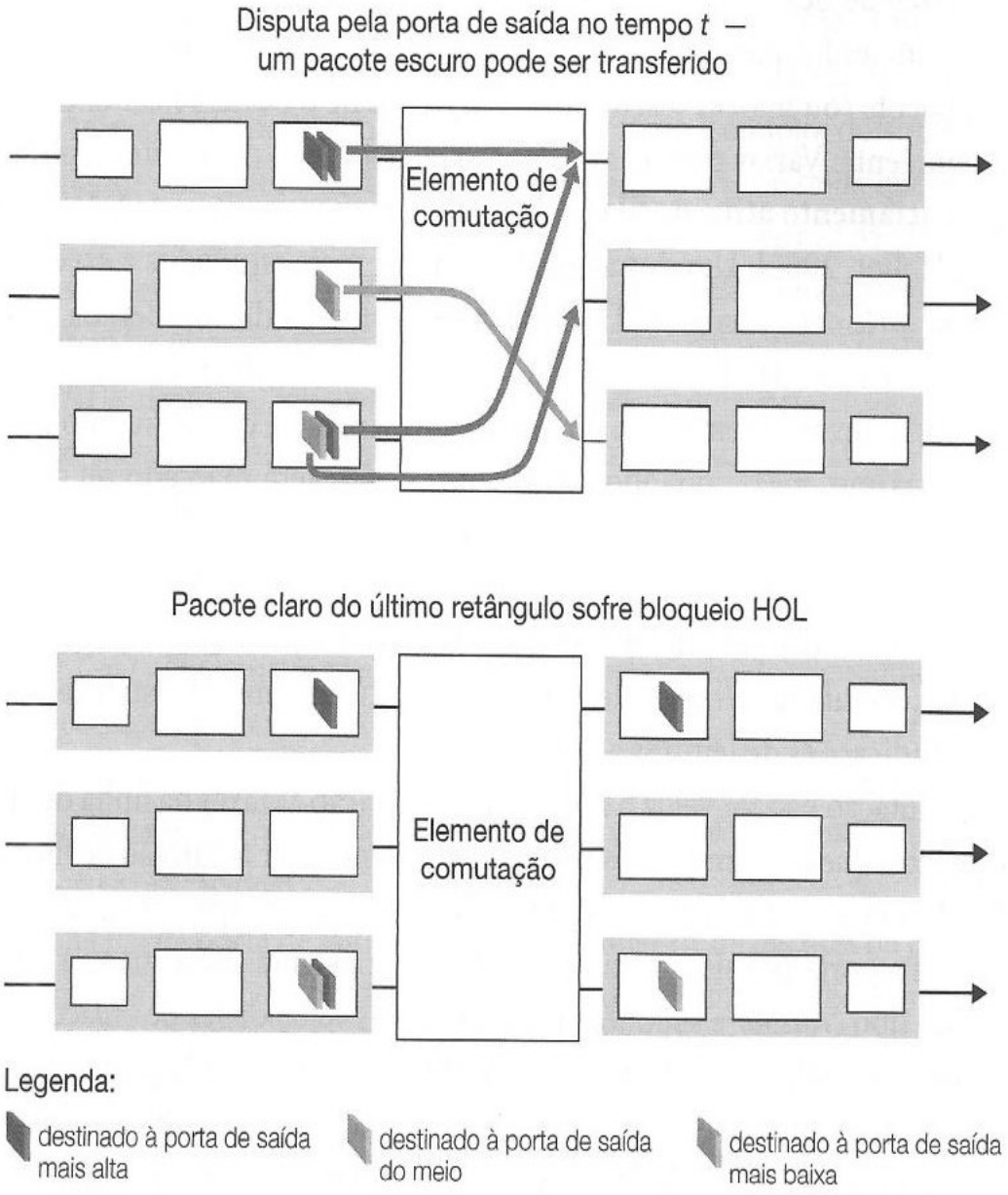
Onde ocorre formação de fila?



- usa buffers quando taxa de chegada através do comutador excede taxa de transmissão de saída
- enfileiramento (retardo), e perdas devidas ao transbordo do buffer da porta de saída!

Onde ocorre formação de fila?

- Se matriz de comutação for mais lenta do que a soma das portas de entrada juntas -> pode haver filas nas portas de entrada
- **Bloqueio cabeça-de-linha**: datagrama na cabeça da fila impede outros na mesma fila de avançarem
- Retardo de enfileiramento e perdas devido ao transbordo do buffer de entrada!

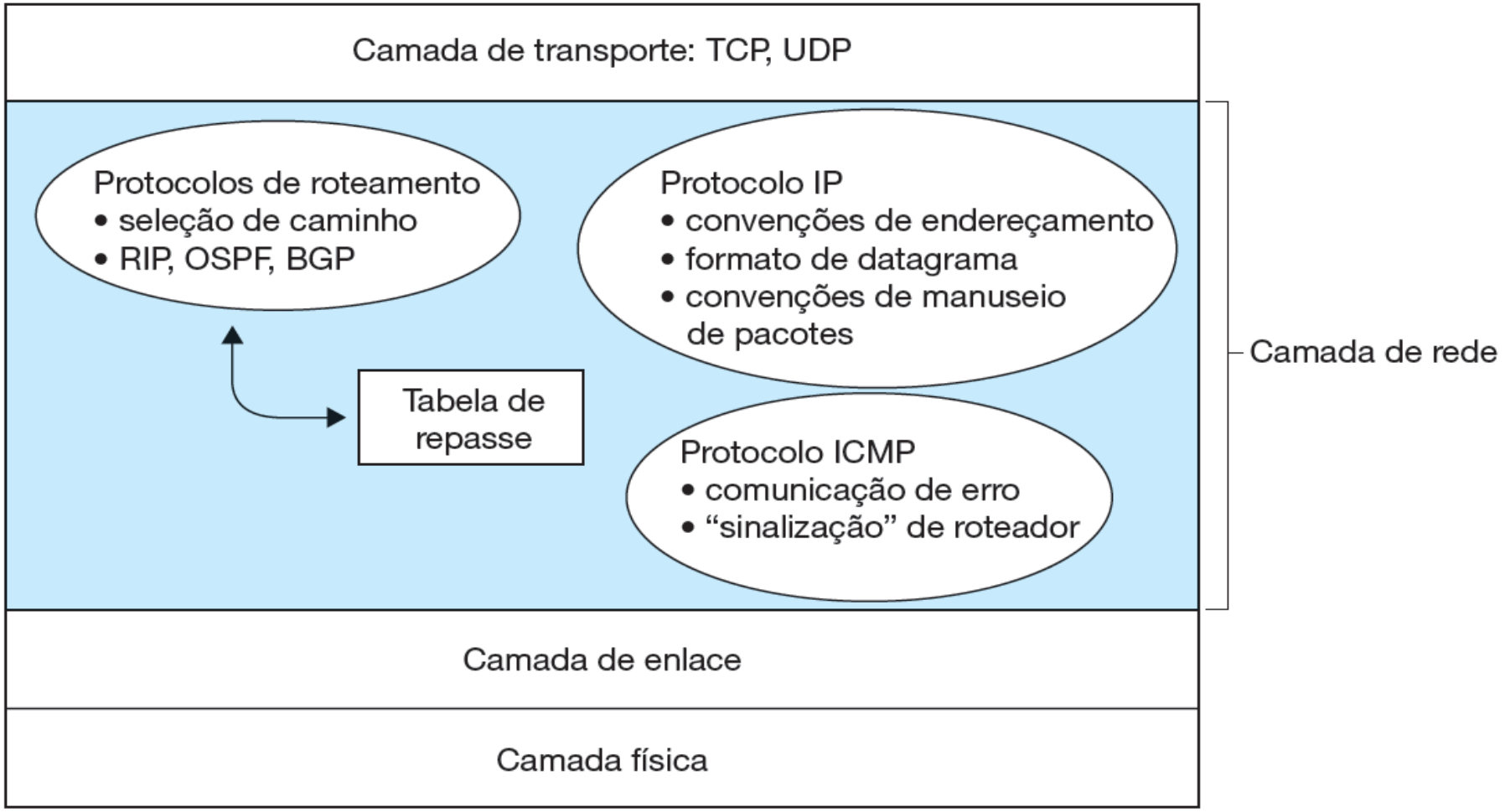


Sumário

4.1 – Introdução
4.2 – Redes de circuitos virtuais e de datagramas
4.3 – O que há dentro de um roteador?
4.4 – O Protocolo da Internet (IP): repasse e endereçamento na Internet
4.5 – Algoritmos de roteamento
4.6 – Roteamento na Internet
4.7 – Roteamento por difusão e para um grupo

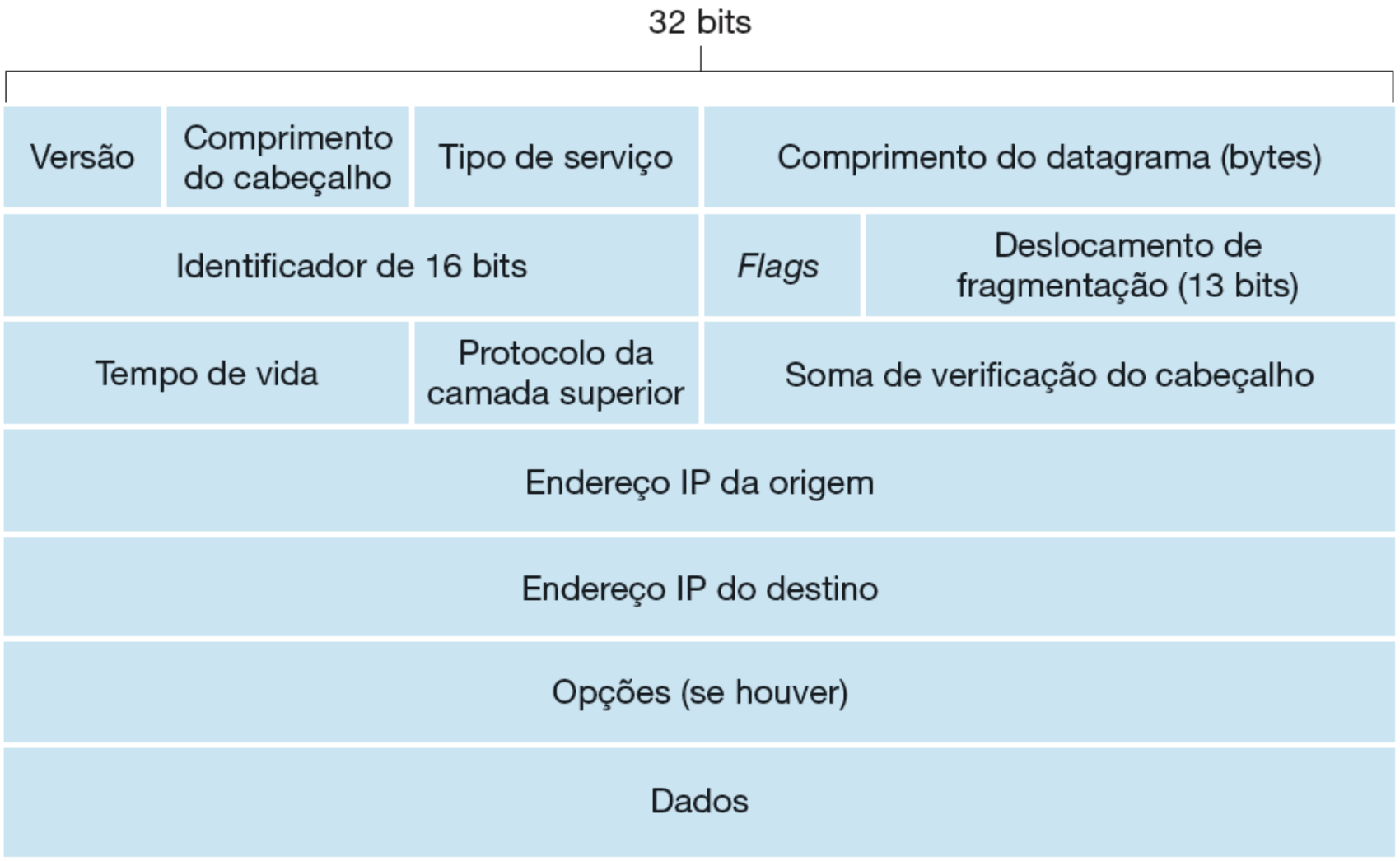
O Protocolo da Internet (IP): repasse e endereçamento na Internet

- O interior da camada de rede da Internet



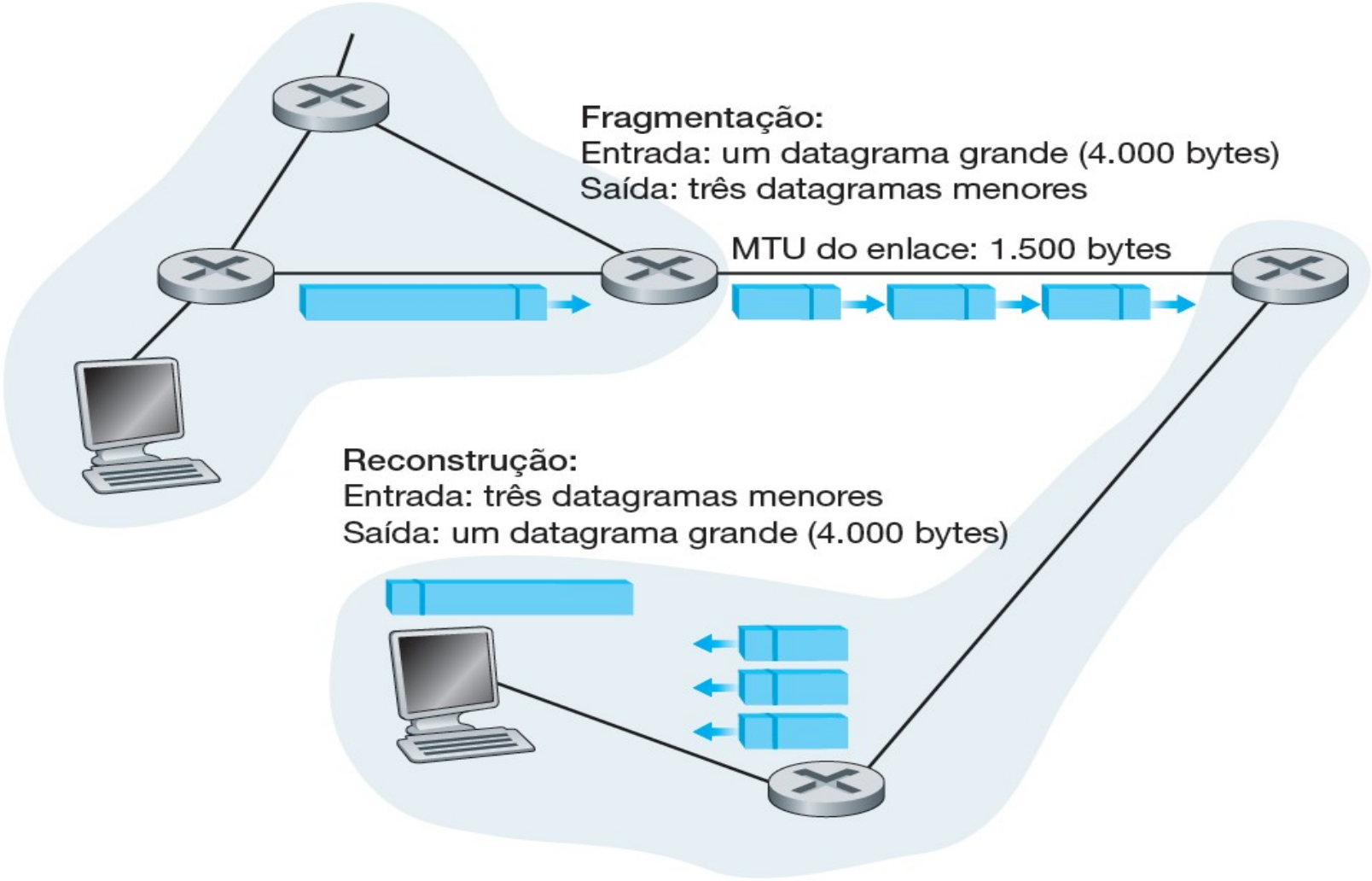
Formato de datagrama

- Formato do datagrama IPv4



Fragmentação do datagrama IP

- Fragmentação e reconstrução IP



Fragmentação do datagrama IP

- Fragmentos IP

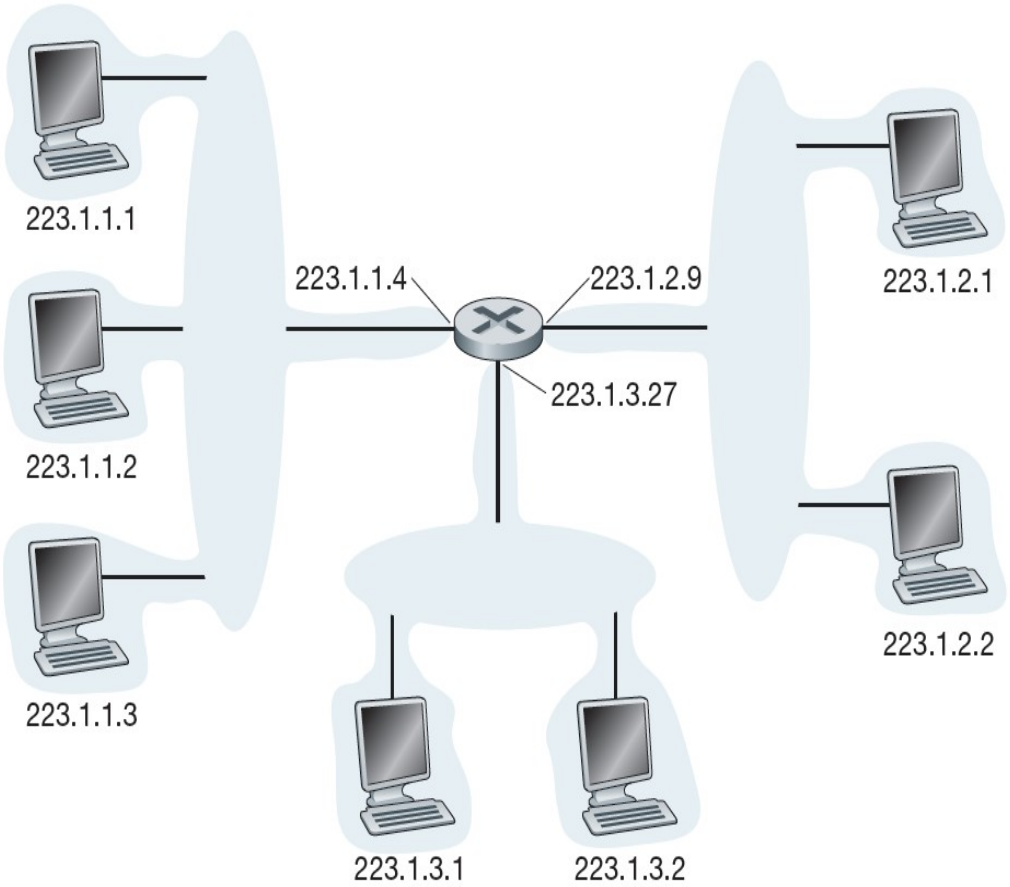
Fragmento	Bytes	ID	Deslocamento	*Flag*
1º fragmento	1.480 bytes no campo de dados do datagrama IP	identificação = 777	0 (o que significa que os dados devem ser inseridos a partir do byte 0)	1 (o que significa que há mais)
2º fragmento	1.480 bytes de dados	identificação = 777	185 (o que significa que os dados devem ser inseridos a partir do byte 1.480. Note que 185 x 8 = 1.480)	1 (o que significa que há mais)
3º fragmento	1.020 bytes de dados (= 3.980 –1.480 –1.480)	identificação = 777	370 (o que significa que os dados devem ser inseridos a partir do byte 2.960. Note que 370 x 8 = 2.960)	0 (o que significa que esse é o último fragmento)

Endereçamento IPv4

- **Um endereço IP está tecnicamente associado com uma interface**, ou seja, o IP não é da “máquina”, mas sim da interface de rede.
- Cada endereço IP tem comprimento de 32 bits (equivalente a 4 bytes).
- Portanto, há um total de 2^{32} endereços IP possíveis.
- Há cerca de 4 bilhões de endereços IP possíveis.
- Esses endereços são escritos em **notação decimal separada por pontos**.

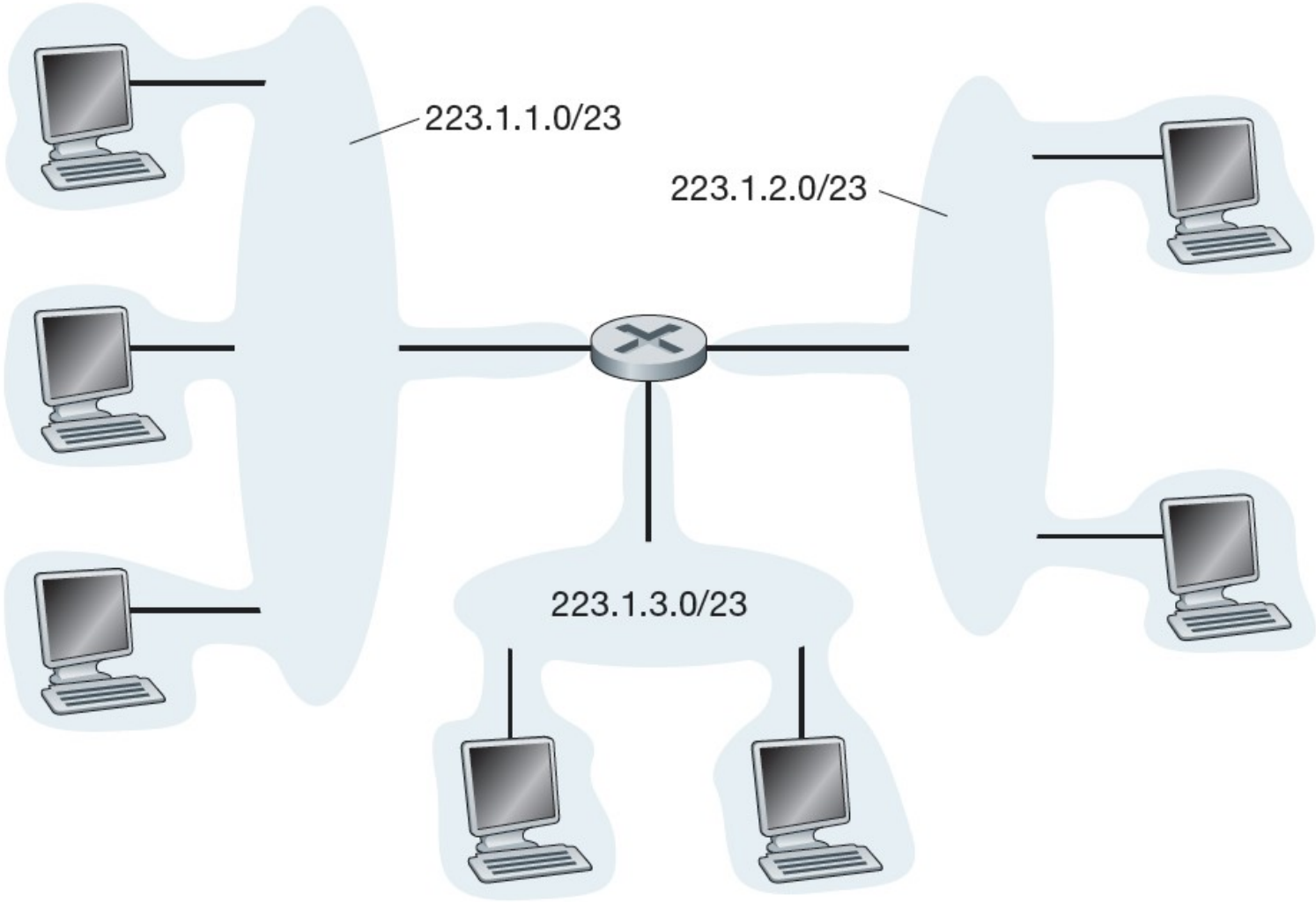
Endereçamento IPv4

- Endereços de interfaces e sub-redes:
  - **endereço IP**
    - parte de rede (bits de mais alta ordem)
    - parte de estação (bits de mais baixa ordem)
  - **O que é uma subrede IP? (da perspectiva do endereço IP)**
    - interfaces de dispositivos com a mesma parte de rede nos seus endereços IP
    - podem alcançar um ao outro sem passar por um roteador



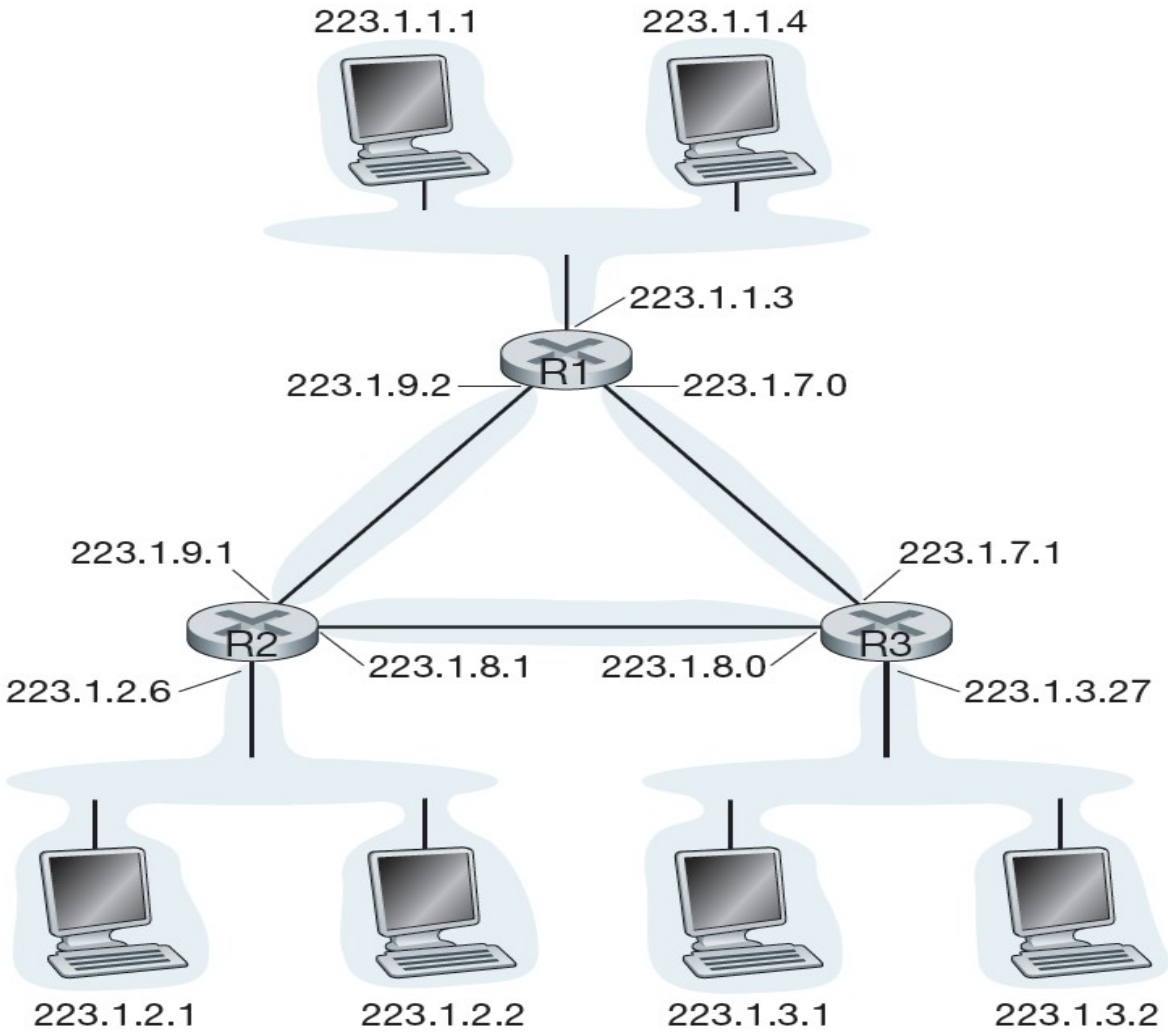
Endereçamento IPv4

- Endereços de sub-redes



Endereçamento IPv4

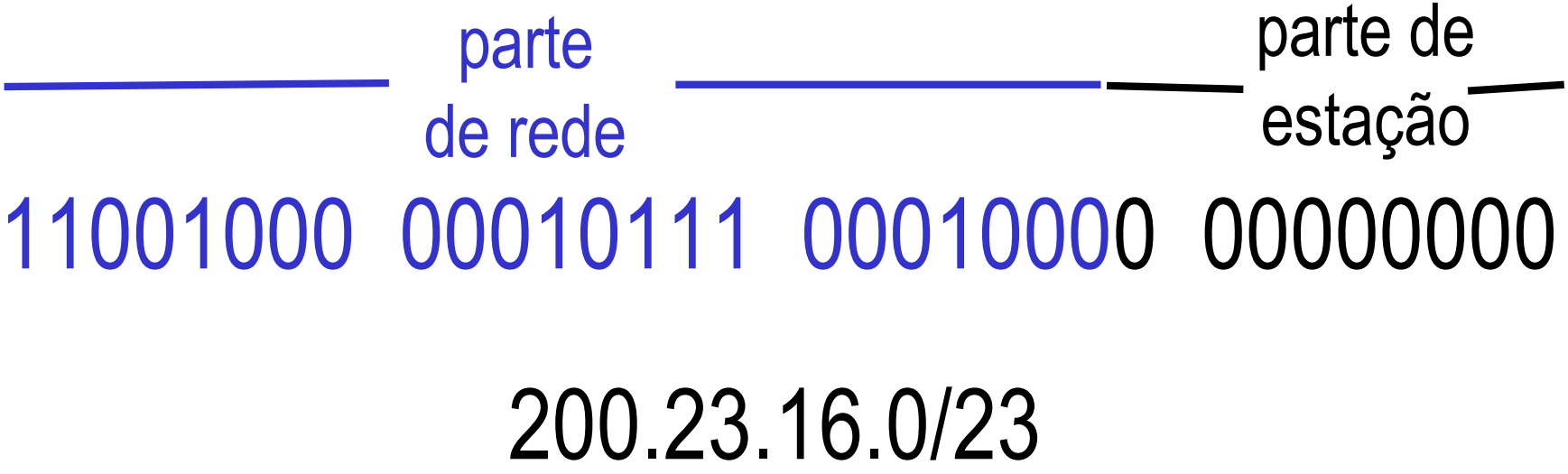
- Três roteadores interconectando seis sub-redes



Endereçamento IPv4

- Endereçamento IP: CIDR

- **CIDR**: ***C****lassless* ***I****nter****D****omain* ***R****outing*
  - parte de rede do endereço de comprimento arbitrário
  - **formato de endereço**: **a.b.c.d/x**, onde x é no. de bits na parte de rede do endereço



Obtenção de um bloco de endereços

- Para obter um bloco de endereços IP para utilizar dentro da sub-rede de uma organização, um administrador de rede poderia:

1. contatar seu ISP, que forneceria endereços a partir de um bloco maior de endereços que já estão alocados ao ISP.

2. O ISP, por sua vez, dividiria seu bloco de endereços em oito blocos de endereços contíguos, do mesmo tamanho, e daria um deles a cada uma de um conjunto de oito organizações suportadas por ele (veja figura a seguir):

Obtenção de um bloco de endereços

Bloco do ISP	200.23.16.0/20	11001000	00010111	00010000	00000000
Organização 0	200.23.16.0/23	11001000	00010111	00010000	00000000
Organização 1	200.23.18.0/23	11001000	00010111	00010010	00000000
Organização 2	200.23.20.0/23	11001000	00010111	00010100	00000000
...	...		...		
Organização 7	200.23.30.0/23	11001000	00010111	00011110	00000000

Obtenção de um bloco de endereços

- **P**: Como um provedor IP consegue um bloco de endereços?
- **R**: ICANN: *Internet Corporation for Assigned Names and Numbers*
  - aloca endereços
  - gerencia DNS
  - aloca nomes de domínio, resolve disputas
- No Brasil, estas funções foram delegadas ao NIC.br pelo Comitê Gestor Internet BR – www.cg.org.br)

Obtenção de um endereço de hospedeiro: o Protocolo de Configuração Dinâmica de Hospedeiros (DHCP)

- O DHCP permite que um hospedeiro obtenha (seja alocado a) um endereço IP de maneira automática (**UDP → 67**).

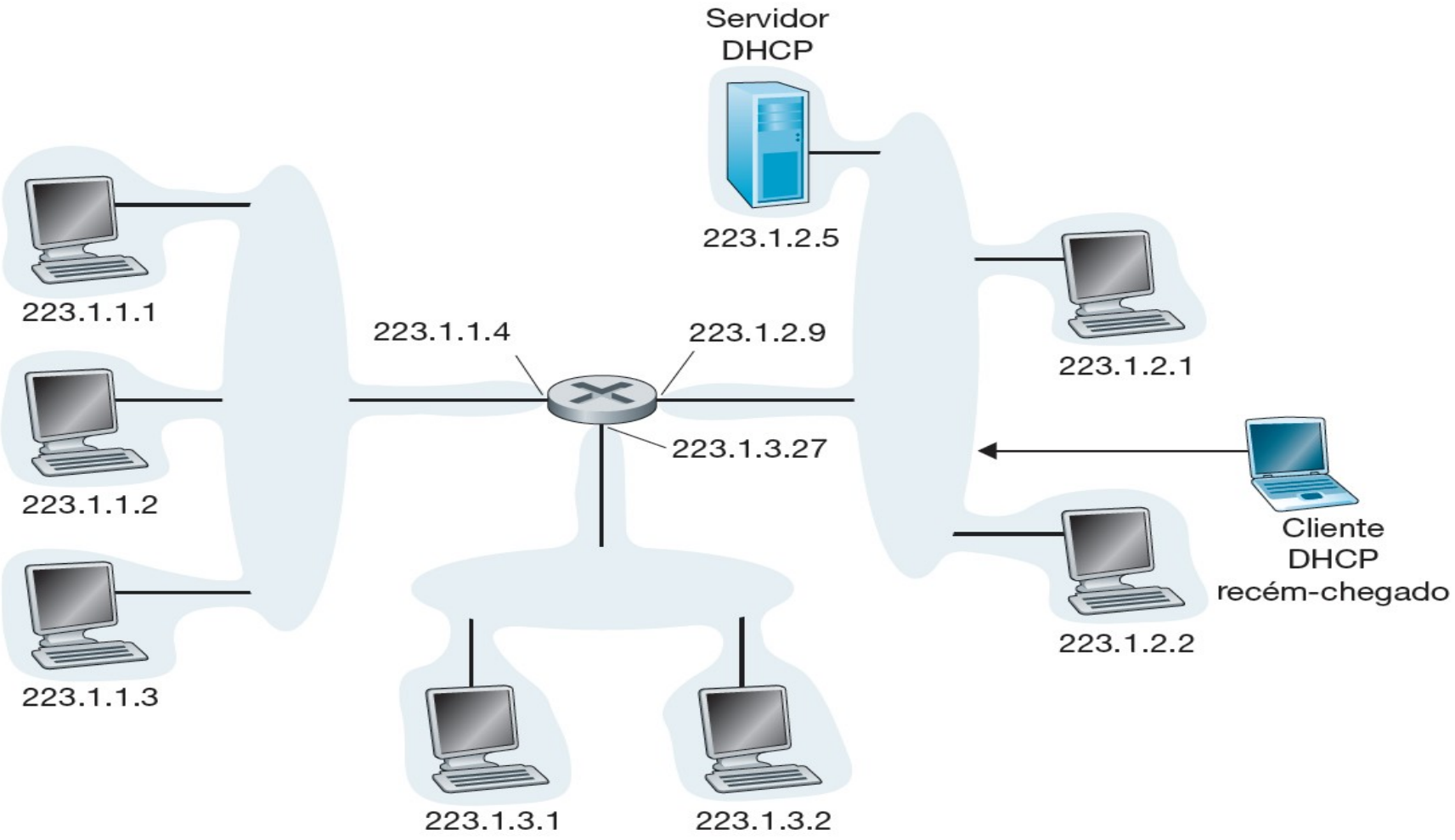
- O DHCP é em geral denominado um **protocolo *plug and play***.

- O protocolo DHCP é um processo de quatro etapas:

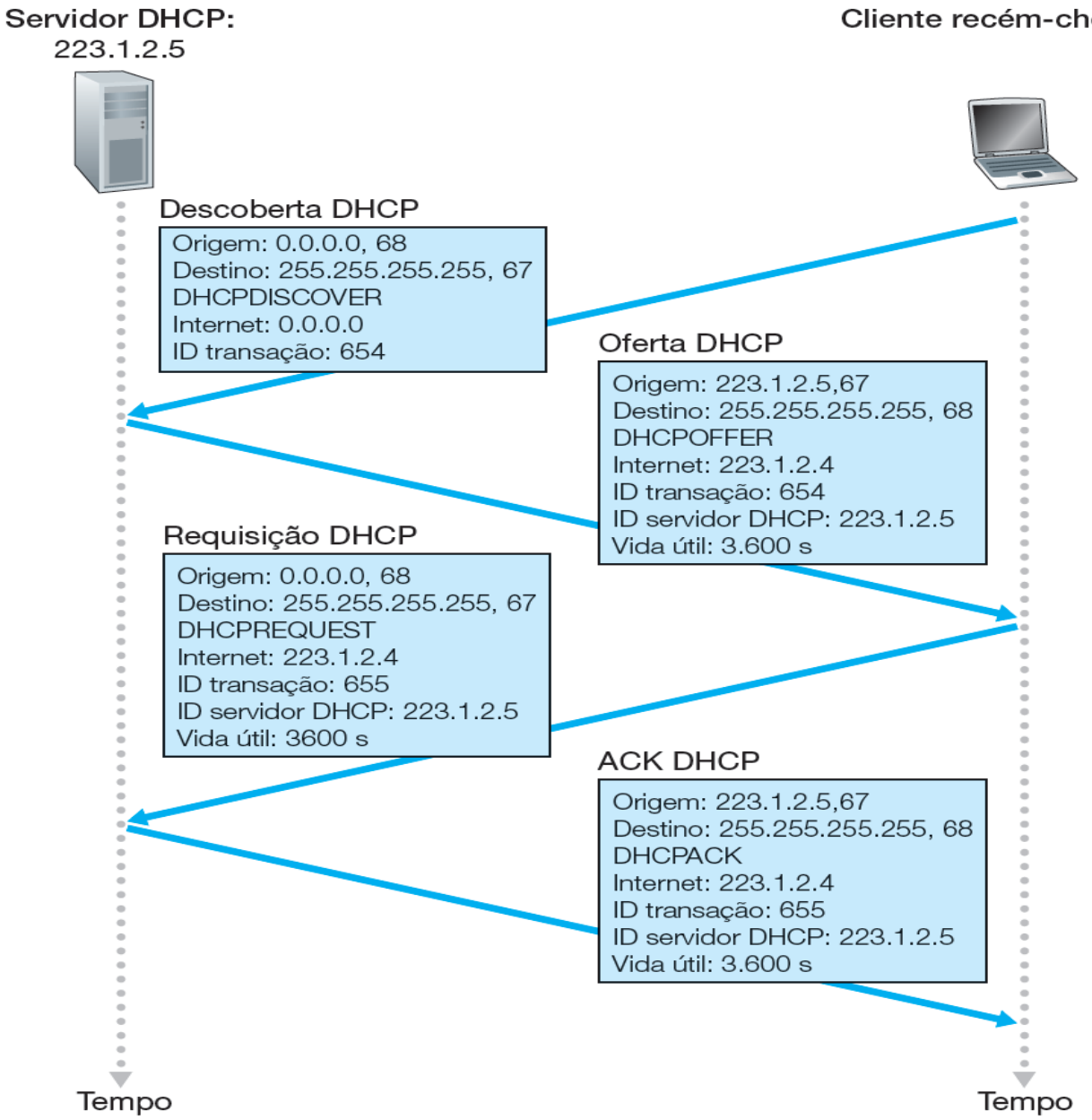
1. Descoberta do servidor DHCP.
2. Oferta(s) dos servidores DHCP.
3. Solicitação DHCP.
4. DHCP ACK.

Obtenção de um endereço de hospedeiro: o Protocolo de Configuração Dinâmica de Hospedeiros (DHCP)

- Cenário cliente-servidor DHCP



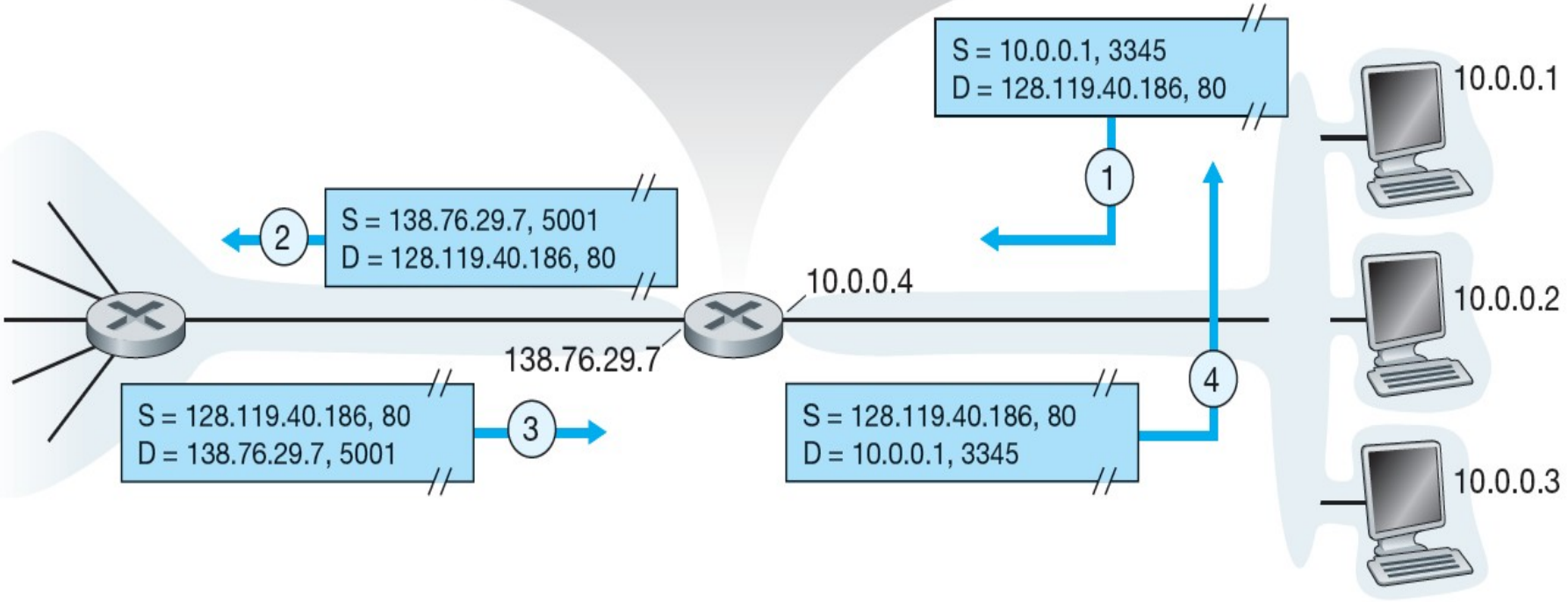
Obtenção de um endereço de hospedeiro: DHCP



Tradução de endereços na rede (NAT)

- Tradução de endereços de rede (S = Origem, D = Destino)

Tabela de tradução NAT	
Lado da WAN	Lado da LAN
138.76.29.7, 5001	10.0.0.1, 3345
. . .	. . .



Protocolo de Mensagens de Controle da Internet (ICMP)

- O ICMP é usado por hospedeiros e roteadores para comunicar informações de camada de rede entre si.
- A utilização mais comum do ICMP é para comunicação de erros.
- Mensagens ICMP têm um campo de tipo e um campo de código.
- O conhecido programa **ping** envia uma mensagem ICMP do tipo 8 código 0 para o hospedeiro especificado (**Como funciona?**).
- Alguns tipos de mensagens ICMP selecionadas são mostrados a seguir.

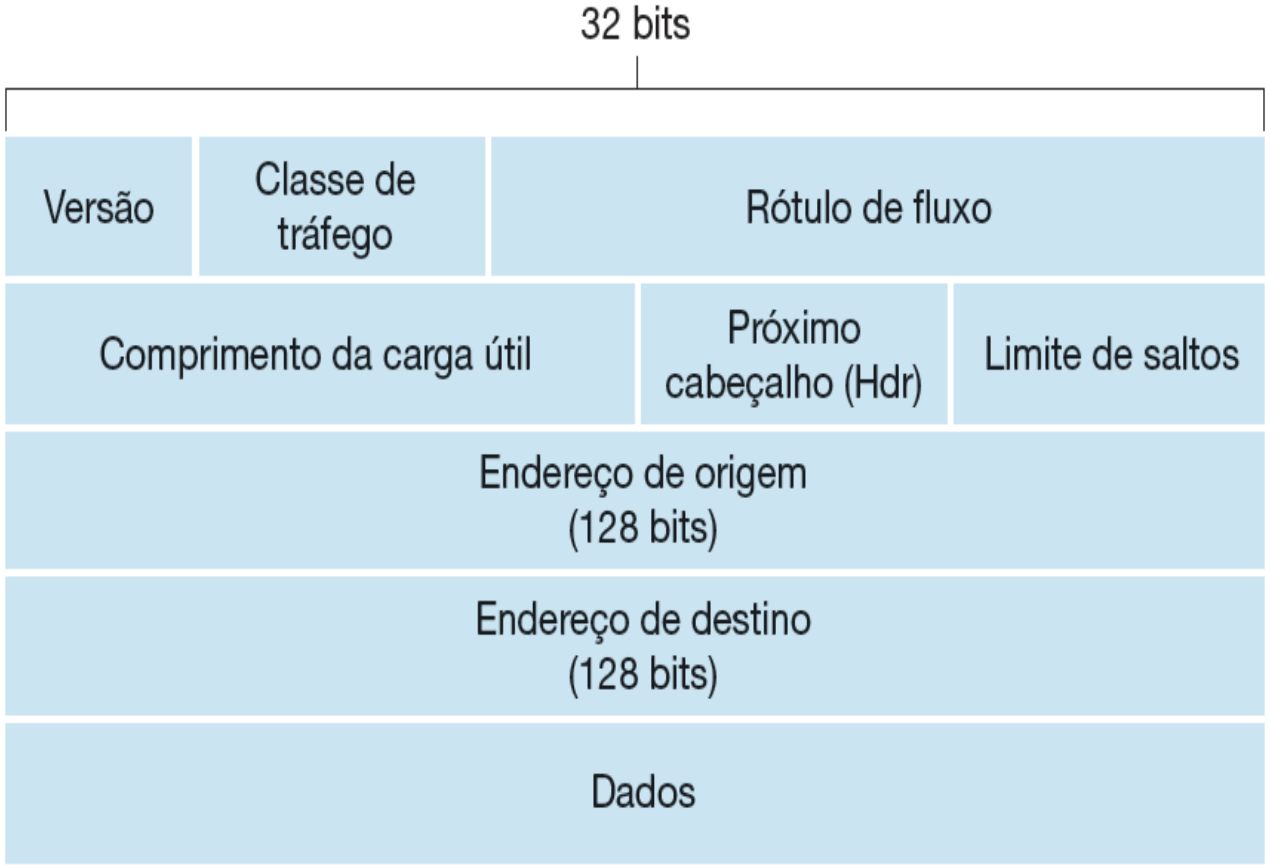
Protocolo de Mensagens de Controle da Internet (ICMP)

- Tipos de mensagens ICMP

Tipo ICMP	Código	Descrição
0	0	resposta de eco (para *ping*)
3	0	rede de destino inalcançável
3	1	hospedeiro de destino inalcançável
3	2	protocolo de destino inalcançável
3	3	porta de destino inalcançável
3	6	rede de destino desconhecida
3	7	hospedeiro de destino desconhecido
4	0	repressão da origem (controle de congestionamento)
8	0	solicitação de eco
9	0	anúncio do roteador
10	0	descoberta do roteador
11	0	TTL expirado
12	0	cabeçalho IP inválido

IPv6

- Para atender a essa necessidade de maior espaço para endereços IP, foi desenvolvido um novo protocolo IP, o IPv6.
- Formato do datagrama IPv6



IPv6

- **Motivação inicial**: espaço de endereços de 32-bits completamente alocado.

- **Motivação adicional**:

  - Formato do cabeçalho facilita acelerar processamento/reencaminhamento
  - mudanças no cabeçalho para facilitar QoS
  - novo endereço "*anycast*": rota para o "melhor" de vários servidores replicados

- **Formato do datagrama IPv6**:

  - cabeçalho de tamanho fixo de 40 bytes
  - não admite fragmentação

IPv6

- **Checksum**: removido completamente para reduzir tempo de processamento a cada roteador

- **Opções**: permitidas, porém fora do cabeçalho, indicadas pelo campo “Próximo Cabeçalho”

- **ICMPv6**: versão nova de ICMP

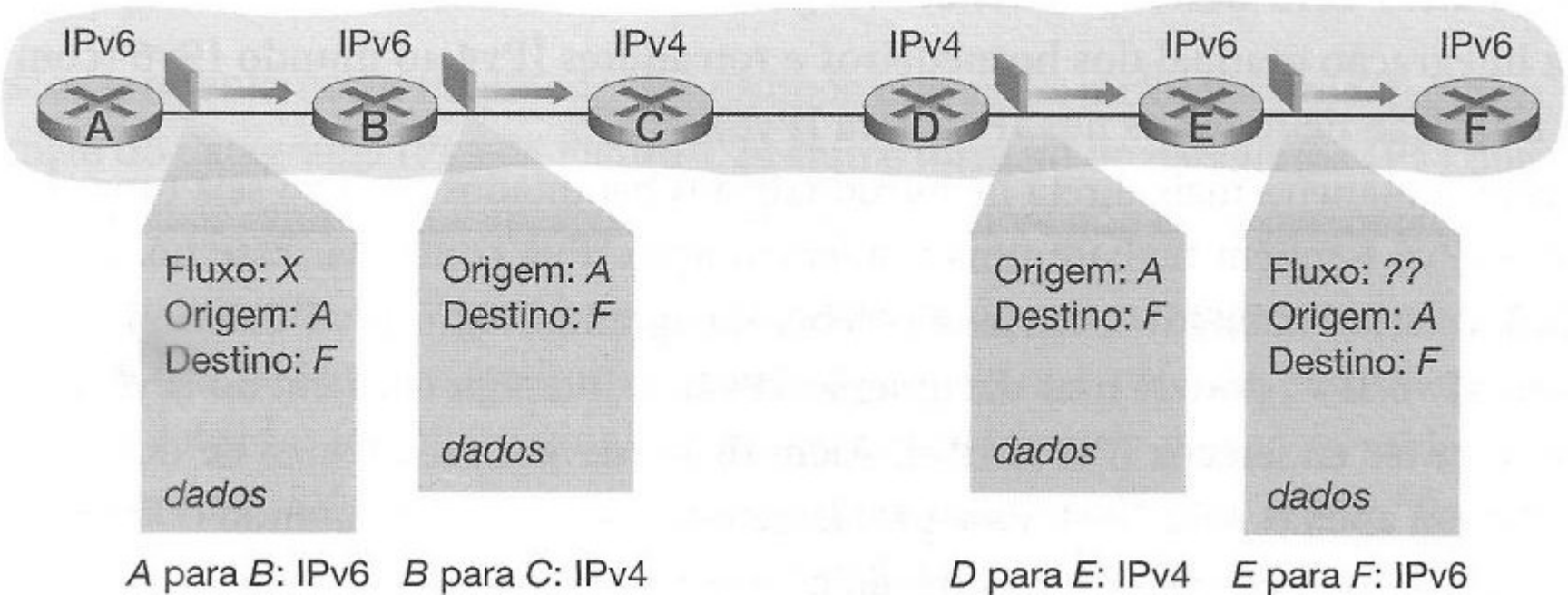
  - tipos adicionais de mensagens, p.ex. “Pacote Muito Grande”
  - Funções de gerenciamento de grupo multiponto

Transição do IPv4 para o IPv6

- Nem todos os roteadores podem ser atualizados simultaneamente
- “Dias de mudança geral” inviáveis
- Como a rede pode funcionar com uma mistura de roteadores IPv4 e IPv6?
- **Tunelamento**: datagramas IPv6 carregados em datagramas IPv4 entre roteadores IPv4

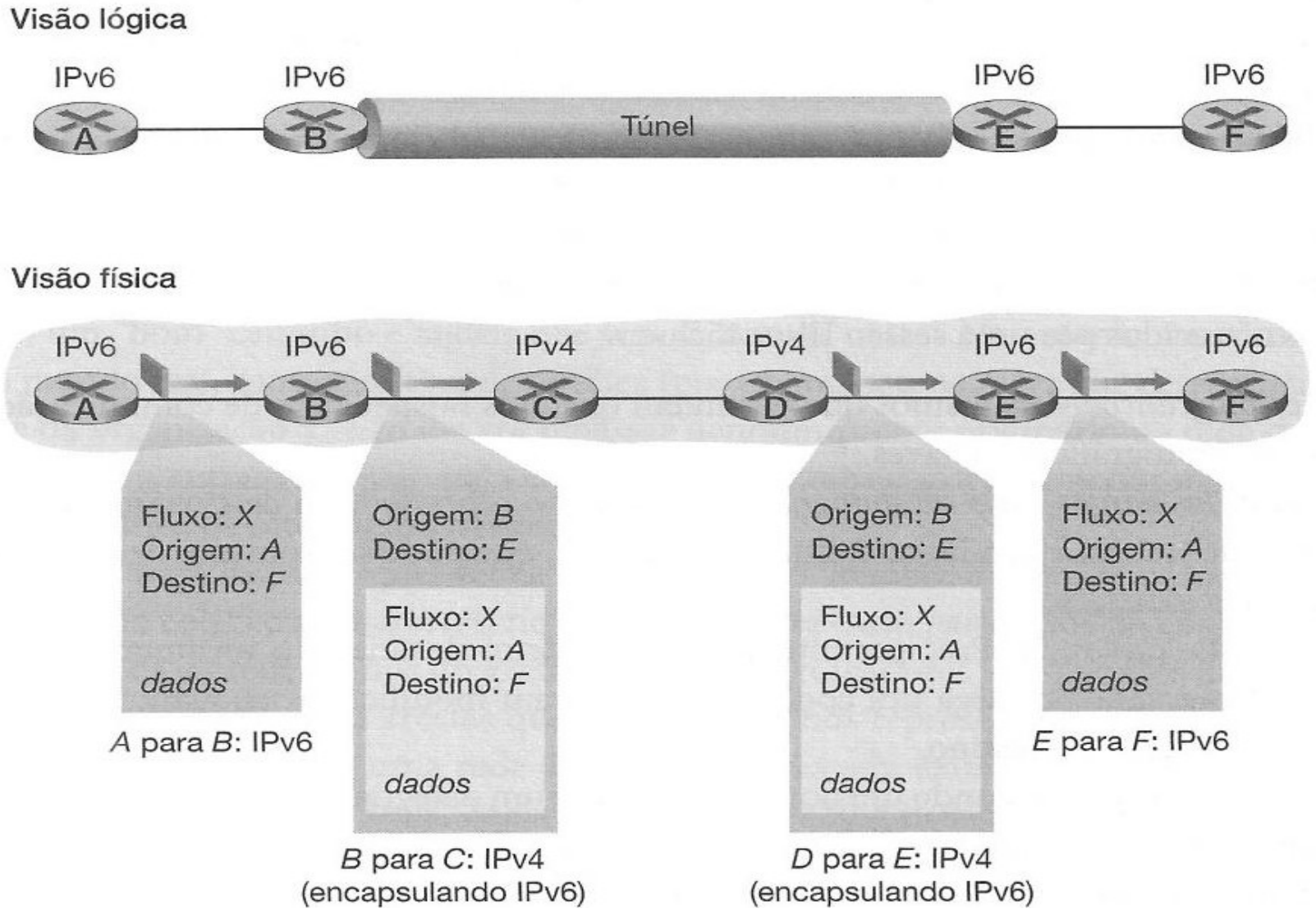
Transição do IPv4 para o IPv6

- Abordagem de pilha dupla:
  - Roteadores habilitados para funcionar IPv6/IPv4



Transição do IPv4 para o IPv6

- Abordagem de tunelamento:



Uma breve investida em segurança IP

- O IPsec foi desenvolvido para ser compatível com o IPv4 e o IPv6.
- Em particular, para obter os benefícios do IPv6, não precisamos substituir as pilhas dos protocolos em todos os roteadores e hospedeiros na Internet.

Os serviços oferecidos por uma sessão IPsec incluem:

- Acordo criptográfico.

Uma breve investida em segurança IP

- Codificação das cargas úteis do datagrama IP.
- Integridade dos dados.
- Autenticação de origem.

Quando dois hospedeiros estabelecem uma sessão IPsec, todos os segmentos TCP e UDP enviados entre eles serão codificados e autenticados.

O IPsec oferece uma cobertura geral, protegendo toda a comunicação entre os dois hospedeiros para todas as aplicações de rede.

Sumário

4.1 – Introdução
4.2 – Redes de circuitos virtuais e de datagramas
4.3 – O que há dentro de um roteador?
4.4 – O Protocolo da Internet (IP): repasse e endereçamento na Internet
4.5 – Algoritmos de roteamento
4.6 – Roteamento na Internet
4.7 – Roteamento por difusão e para um grupo

Algoritmos de roteamento

- Em geral um hospedeiro está ligado diretamente a um roteador, o **roteador *default*** para esse hospedeiro.
- Denominamos **roteador de origem** o roteador *default* do hospedeiro de origem e **roteador de destino** o roteador *default* do hospedeiro de destino.
- O problema de rotear um pacote do hospedeiro de origem até o hospedeiro de destino se reduz, claramente, ao problema de direcionar o pacote do roteador de origem ao roteador de destino.

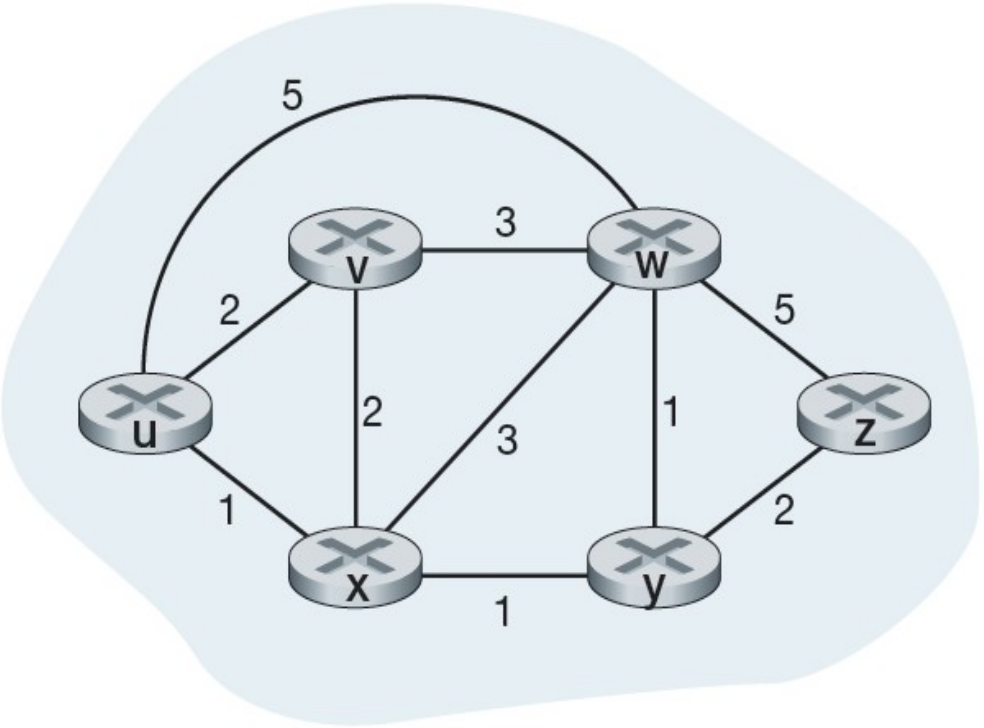
Algoritmos de roteamento

- Um grafo é usado para formular problemas de roteamento.

- Grafo: G = (N,E)
- N = conj. de roteadores = { u, v, w, x, y, z }
- E = conj. de enlaces = { (u,v), (u,x), (v,x), (v,w), (x,w), (x,y), (w,y), (w,z), (y,z) }

Comentário: a abstração com grafos é útil em outros contextos da rede

Exemplo: **P2P**, onde **N** é o conj. dos pares e **E** é o conj. das conexões TCP



Algoritmos de roteamento

- Custo do caminho (x1, x2, x3,…, xp) = c(x1,x2) + c(x2,x3) + … + c(xp-1,xp) .

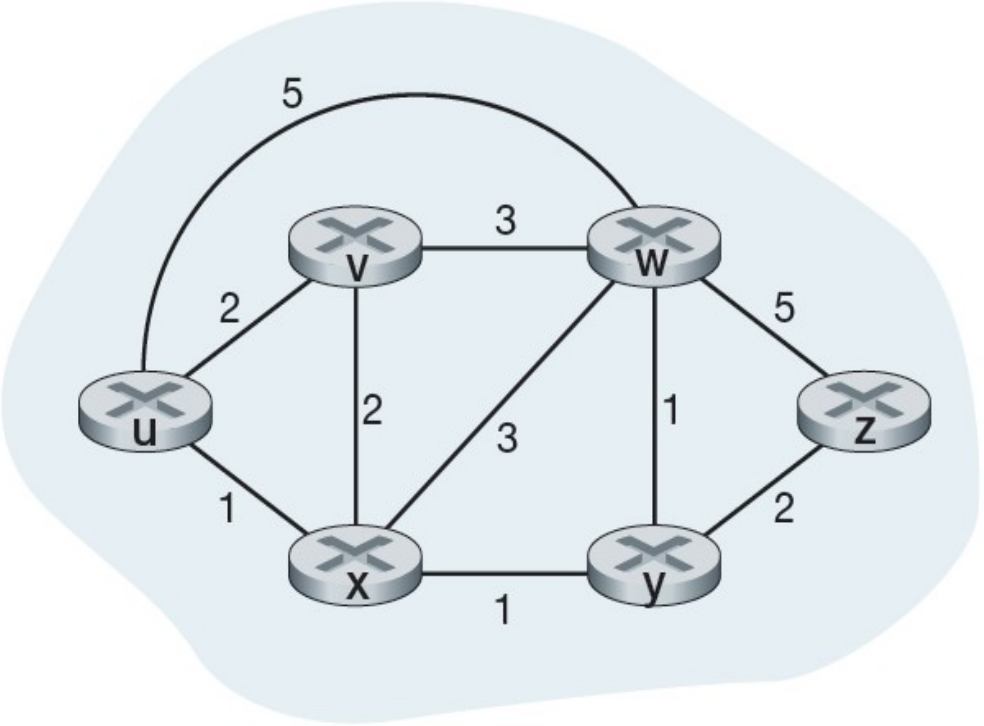
Q: Qual o caminho de menor custo entre u e z?

Algoritmo de roteamento: algoritmo que encontra o caminho de menor custo

c(x,x') = custo do enlace (x,x')

- p.e., c(w,z) = 5

custo poderia também ser 1, ou inversamente relacionado à banda, ou inversamente relacionado ao congestionamento



Algoritmos de roteamento

- Um **algoritmo de roteamento global** calcula o caminho de menor custo entre uma origem e um destino usando conhecimento completo e global sobre a rede.
- Em um **algoritmo de roteamento descentralizado**, o cálculo do caminho de menor custo é realizado de modo iterativo e distribuído.
- Em **algoritmos de roteamento estático**s, as rotas mudam muito devagar ao longo do tempo, muitas vezes como resultado de intervenção humana
- **Algoritmos de roteamento dinâmicos** mudam os caminhos de roteamento à medida que mudam as cargas de tráfego ou a topologia da rede.
- Em um **algoritmo sensível à carga**, custos de enlace variam dinamicamente para refletir o nível corrente de congestionamento no enlace subjacente.

Algoritmos de roteamento

Algoritmo de Dijkstra:

- topologia da rede, custos dos enlaces conhecidos por **todos** os nós
  - realizado através de “difusão do estado dos enlaces”
  - todos os nós têm mesma info.
- calcula caminhos de menor custo de um nó (“origem”) para todos os demais
  - gera **tabela de rotas** para aquele nó
- **iterativo**: depois de **k** iterações, sabemos menor custo p/ **k** destinos

- **Notação:**

- **c(i,j):** custo do enlace do nó **i** ao nó **j**. custo é infinito se não forem vizinhos diretos
- **D(V):** valor corrente do custo do caminho da origem ao destino **V**
- **p(V):** nó antecessor no caminho da origem ao nó **V**, imediatamente antes de **V**
- **N’:** conjunto de nós cujo caminho de menor custo já foi determinado

O algoritmo de roteamento de estado de enlace (LS)

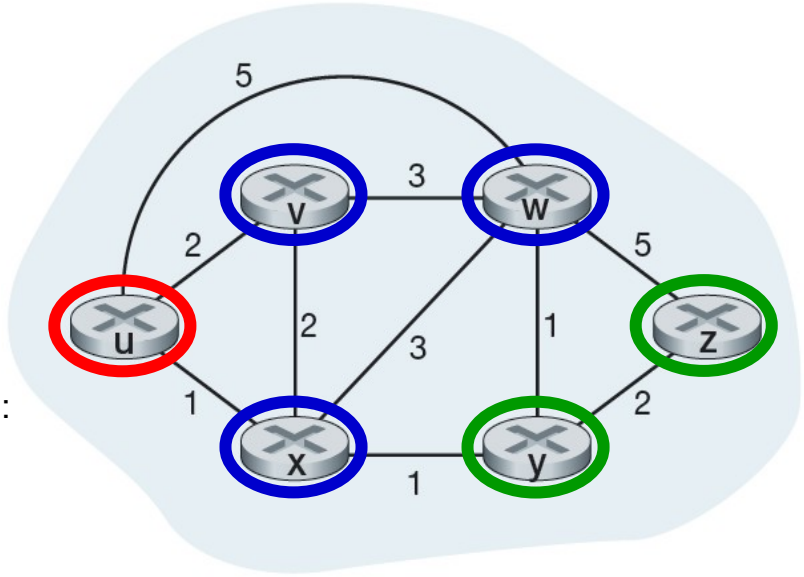
- Algoritmo de estado de enlace para o nó de origem *u*

```
1   Inicialização
2       N' = {u}
3       para todos os nós v
4           se v for um vizinho de u
5               então D(v) = c(u,v)
6           senão D(v) = ∞
7
8   Loop
9       encontre w não em N' tal que D(w) é um mínimo
10      adicione w a N'
11      atualize D(v) para cada vizinho v de w e não em N':
12              D(v) = min( D(v), D(w) + c(w,v) )
13      /* o novo custo para v é o velho custo para v ou
14          o custo do menor caminho conhecido para w mais o custo de w para v */
15  até N'= N
```

O algoritmo de roteamento de estado de enlace (LS)

Passo	N'	D(v), p(v)	D(w), p(w)	D(x), p(x)	D(y), p(y)	D(z), p(z)
0	u	2, u	5, u	**1, u**	α	α

```
1  Inicialização:
2    N' = {u}
3    para todos os nós v
4      se v for adjacente ao nó u
5        então D(v) = c(u,v)
6        senão D(v) = ∞
7
8   Repete
9     determina w não contido em N' tal que D(w) é o mínimo
10    adiciona w ao conjunto N'
11    atualiza D(v) para todo v adjacente ao nó w e ainda não em N':
12        D(v) = min( D(v), D(w) + c(w,v) )
13    /* novo custo ao nó v ou é o custo velho a v ou o custo do
14     menor caminho ao nó w, mais o custo de w a v */
15  até que todos nós estejam em N'
```

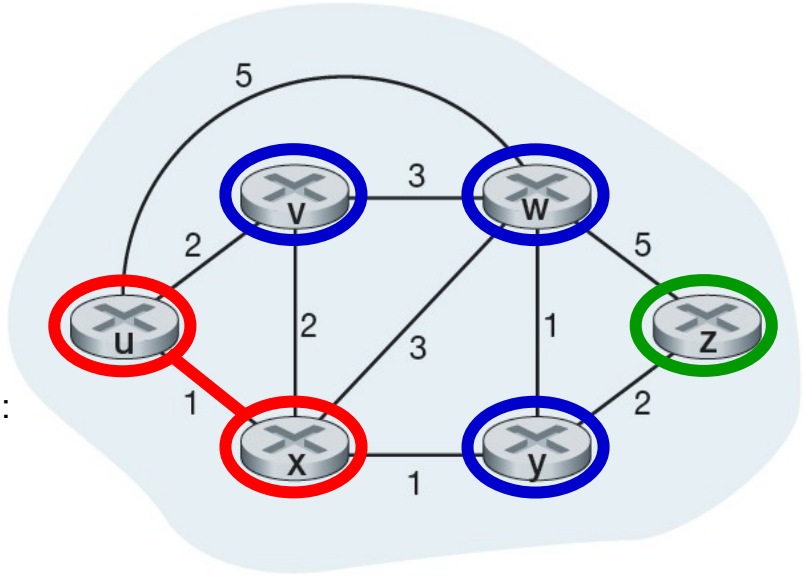


O algoritmo de roteamento de estado de enlace (LS)

Passo	N'	D(v), p(v)	D(w), p(w)	D(x), p(x)	D(y), p(y)	D(z), p(z)
0	u	2, u	5, u	**1, u**	α	α
1	ux	2, u	4, x		**2, x**	α

`D(v) = min(D(v), D(x) + c(x,v)) = min(2,1+2)`
`D(w) = min(D(w), D(x) + c(x,w)) = min(5,1+3)`
`D(y) = min(D(y), D(x) + c(x,y)) = min(α,1+1)`

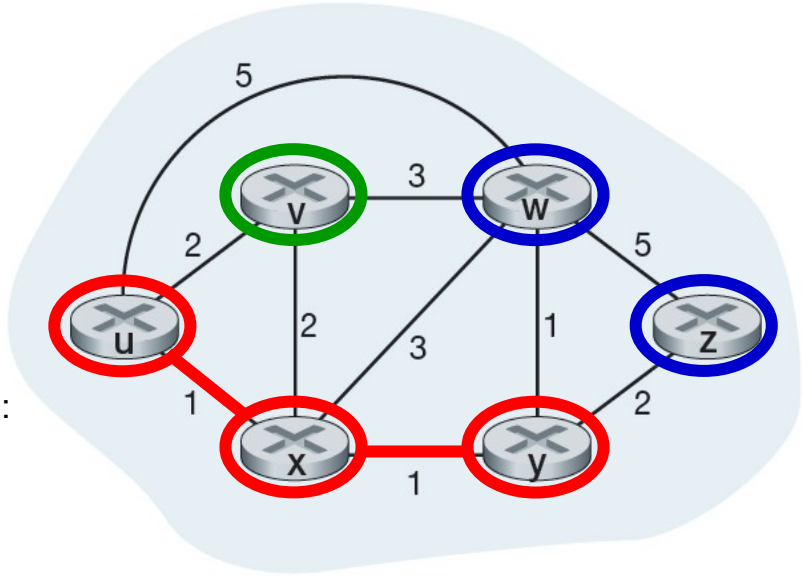
```
1  Inicialização:
2    N' = {u}
3    para todos os nós v
4      se v for adjacente ao nó u
5        então D(v) = c(u,v)
6        senão D(v) = ∞
7
8  Repete
9    determina w não contido em N' tal que D(w) é o mínimo
10   adiciona w ao conjunto N'
11   atualiza D(v) para todo v adjacente ao nó w e ainda não em N':
12       D(v) = min( D(v), D(w) + c(w,v) )
13   /* novo custo ao nó v ou é o custo velho a v ou o custo do
14    menor caminho ao nó w, mais o custo de w a v */
15 até que todos nós estejam em N'
```



O algoritmo de roteamento de estado de enlace (LS)

Passo	N'	D(v), p(v)	D(w), p(w)	D(x), p(x)	D(y), p(y)	D(z), p(z)
0	u	2, u	5, u	**1, u**	α	α
1	ux	2, u	4, x		**2, x**	α
2	uxy	**2, u**	3, y			4, y
		`D(w) = min(D(w), D(y) + c(y,w)) = min(4,2+1)`				
		`D(z) = min(D(z), D(y) + c(y,z)) = min(α,2+2)`				

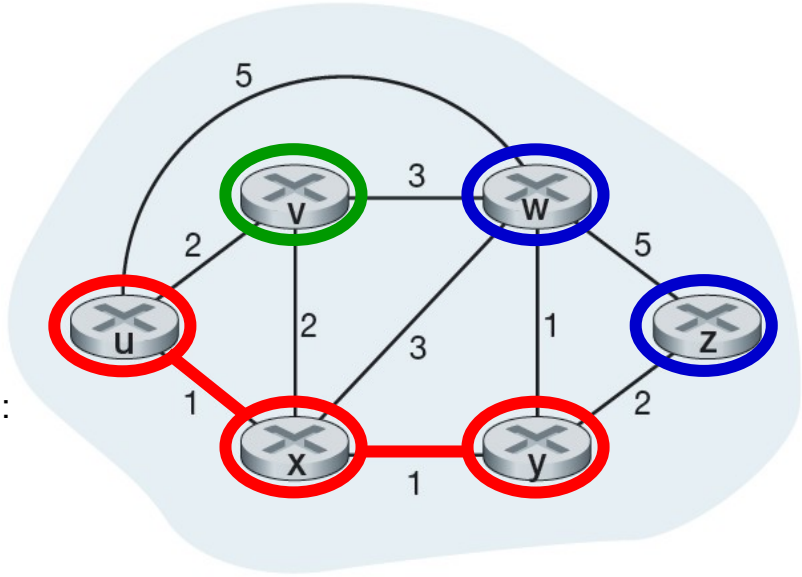
```
1  Inicialização:
2    N' = {u}
3    para todos os nós v
4      se v for adjacente ao nó u
5        então D(v) = c(u,v)
6        senão D(v) = ∞
7
8  Repete
9    determina w não contido em N' tal que D(w) é o mínimo
10   adiciona w ao conjunto N'
11   atualiza D(v) para todo v adjacente ao nó w e ainda não em N':
12       D(v) = min( D(v), D(w) + c(w,v) )
13   /* novo custo ao nó v ou é o custo velho a v ou o custo do
14    menor caminho ao nó w, mais o custo de w a v */
15 até que todos nós estejam em N'
```



O algoritmo de roteamento de estado de enlace (LS)

Passo	N'	D(v), p(v)	D(w), p(w)	D(x), p(x)	D(y), p(y)	D(z), p(z)
0	u	2, u	5, u	**1, u**	α	α
1	ux	2, u	4, x		**2, x**	α
2	uxy	**2, u**	3, y			4, y
		`D(w) = min(D(w), D(y) + c(y,w)) = min(4,2+1)`				
		`D(z) = min(D(z), D(y) + c(y,z)) = min(α,2+2)`				

```
1  Inicialização:
2    N' = {u}
3    para todos os nós v
4      se v for adjacente ao nó u
5        então D(v) = c(u,v)
6        senão D(v) = ∞
7
8  Repete
9    determina w não contido em N' tal que D(w) é o mínimo
10   adiciona w ao conjunto N'
11   atualiza D(v) para todo v adjacente ao nó w e ainda não em N':
12       D(v) = min( D(v), D(w) + c(w,v) )
13   /* novo custo ao nó v ou é o custo velho a v ou o custo do
14    menor caminho ao nó w, mais o custo de w a v */
15 até que todos nós estejam em N'
```

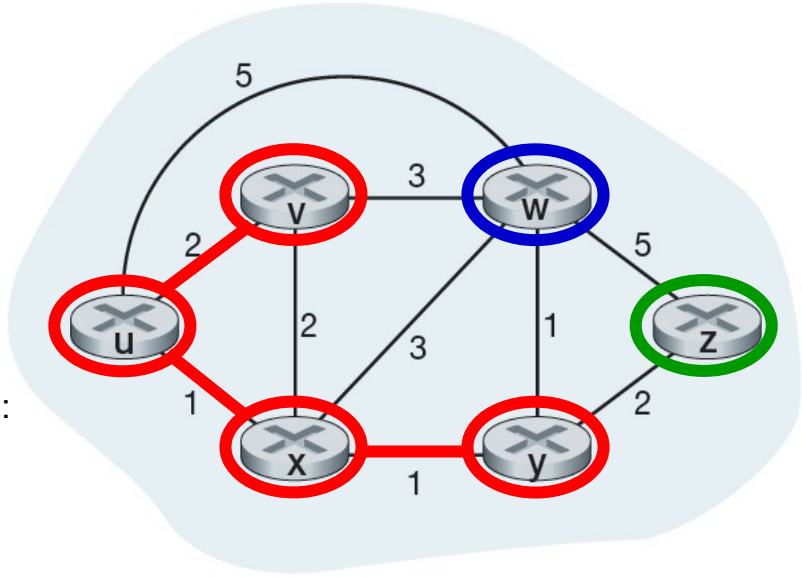


O algoritmo de roteamento de estado de enlace (LS)

Passo	N'	D(v), p(v)	D(w), p(w)	D(x), p(x)	D(y), p(y)	D(z), p(z)
0	u	2, u	5, u	**1, u**	α	α
1	ux	2, u	4, x		**2, x**	α
2	uxy	**2, u**	3, y			4, y
3	uxyv		**3, y**			4, y

`D(w) = min(D(w), D(v) + c(v,w)) = min(3,2+3)`

```
1  Inicialização:
2    N' = {u}
3    para todos os nós v
4      se v for adjacente ao nó u
5        então D(v) = c(u,v)
6        senão D(v) = ∞
7
8   Repete
9     determina w não contido em N' tal que D(w) é o mínimo
10    adiciona w ao conjunto N'
11    atualiza D(v) para todo v adjacente ao nó w e ainda não em N':
12       D(v) = min( D(v), D(w) + c(w,v) )
13    /* novo custo ao nó v ou é o custo velho a v ou o custo do
14     menor caminho ao nó w, mais o custo de w a v */
15  até que todos nós estejam em N'
```

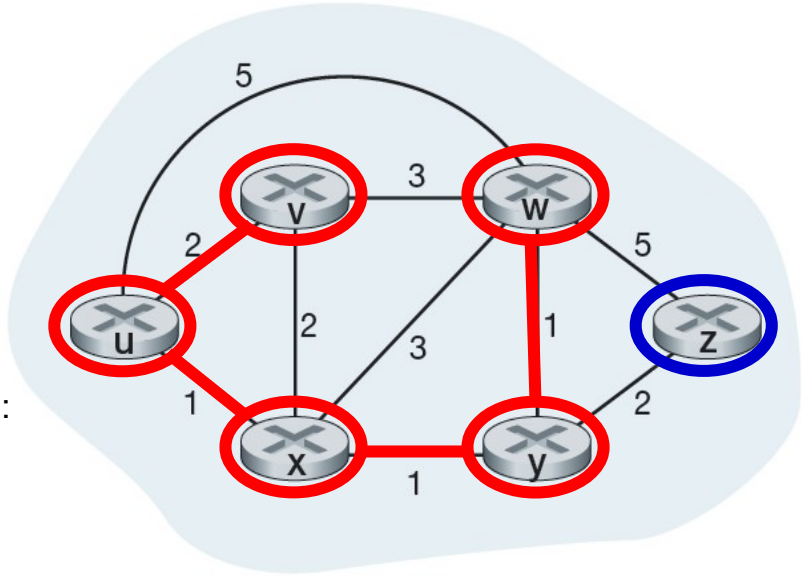


O algoritmo de roteamento de estado de enlace (LS)

Passo	N'	D(v), p(v)	D(w), p(w)	D(x), p(x)	D(y), p(y)	D(z), p(z)
0	u	2, u	5, u	**1, u**	α	α
1	ux	2, u	4, x		**2, x**	α
2	uxy	**2, u**	3, y			4, y
3	uxyv		**3, y**			4, y
4	uvyvw					**4, y**

`D(z) = min(D(z), D(w) + c(w,z)) = min(4,3+5)`

```
1  Inicialização:
2    N' = {u}
3    para todos os nós v
4      se v for adjacente ao nó u
5        então D(v) = c(u,v)
6        senão D(v) = ∞
7
8  Repete
9     determina w não contido em N' tal que D(w) é o mínimo
10    adiciona w ao conjunto N'
11    atualiza D(v) para todo v adjacente ao nó w e ainda não em N':
12       D(v) = min( D(v), D(w) + c(w,v) )
13    /* novo custo ao nó v ou é o custo velho a v ou o custo do
14     menor caminho ao nó w, mais o custo de w a v */
15 até que todos nós estejam em N'
```

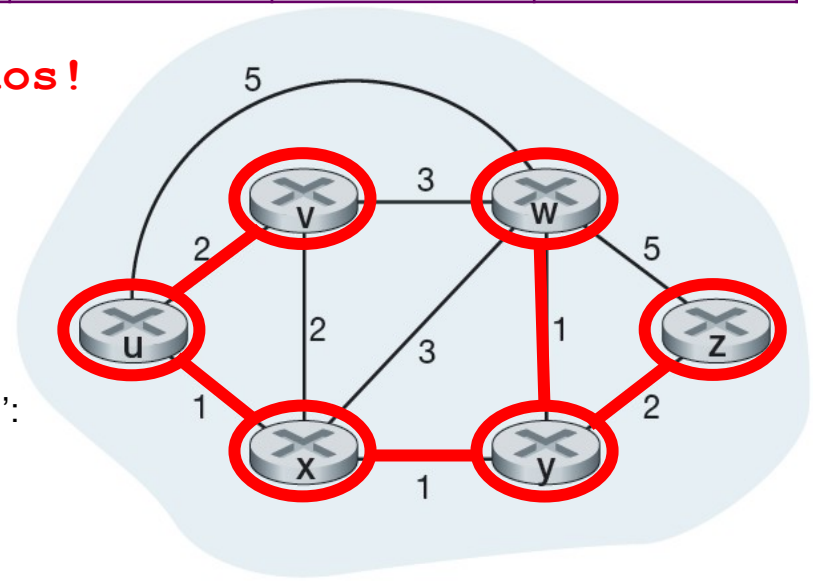


O algoritmo de roteamento de estado de enlace (LS)

Passo	N'	D(v), p(v)	D(w), p(w)	D(x), p(x)	D(y), p(y)	D(z), p(z)
0	u	2, u	5, u	**1, u**	α	α
1	ux	2, u	4, x		**2, x**	α
2	uxy	**2, u**	3, y			4, y
3	uxyv		**3, y**			4, y
4	uvyvw					**4, y**
5	uvyvwz					

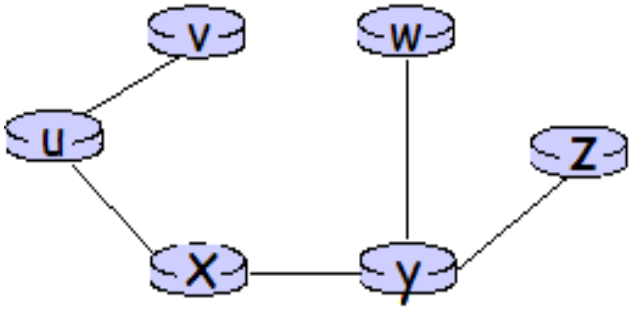
```
1  Inicialização:
2    N' = {u}
3    para todos os nós v
4      se v for adjacente ao nó u
5        então D(v) = c(u,v)
6        senão D(v) = ∞
7
8  Repete
9    determina w não contido em N' tal que D(w) é o mínimo
10   adiciona w ao conjunto N'
11   atualiza D(v) para todo v adjacente ao nó w e ainda não em N':
12       D(v) = min( D(v), D(w) + c(w,v) )
13   /* novo custo ao nó v ou é o custo velho a v ou o custo do
14    menor caminho ao nó w, mais o custo de w a v */
15 até que todos nós estejam em N'
```

Não tem mais vizinhos!



O algoritmo de roteamento de estado de enlace (LS)

- Algoritmo de estado de enlace para o nó de origem *u*:

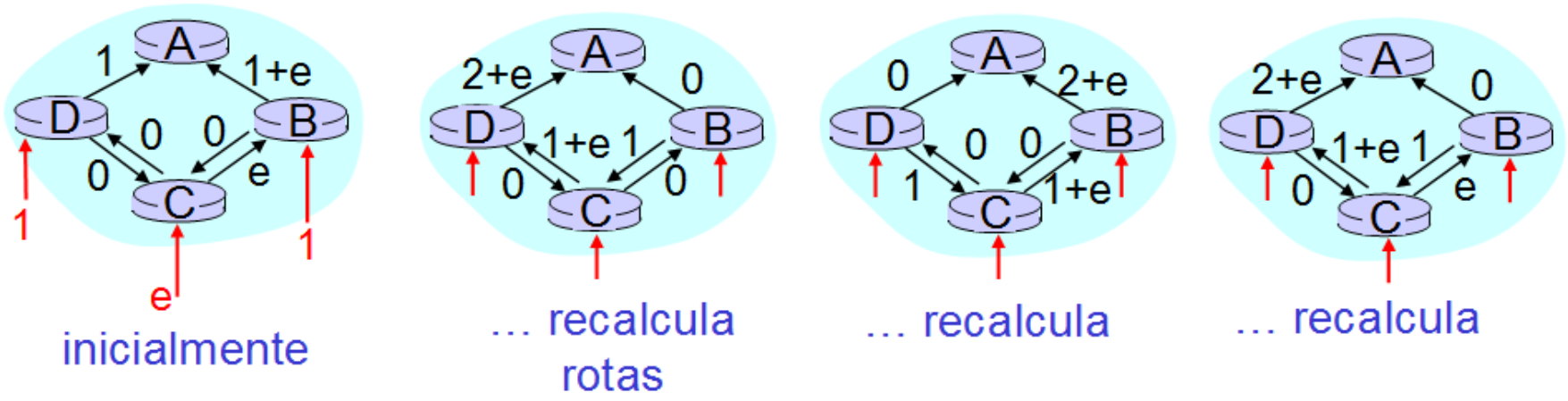


- Tabela de encaminhamento resultante em *u*:

Destino	Enlace
v	(u,v)
x	(u,x)
y	(u,x)
w	(u,x)
z	(u,x)

O algoritmo de roteamento de estado de enlace (LS)

- Algoritmo de Dijkstra, discussão:
- Complexidade algorítmica: **n** nós
  - a cada iteração: precisa checar todos nós, w, não em N'
  - n*(n+1)/2 comparações => O(n2)
  - implementações mais eficientes possíveis: O(nlogn)
- Oscilações possíveis:
  - p.ex., custo do enlace = carga do tráfego carregado



O algoritmo de roteamento de vetor de distâncias (DV)

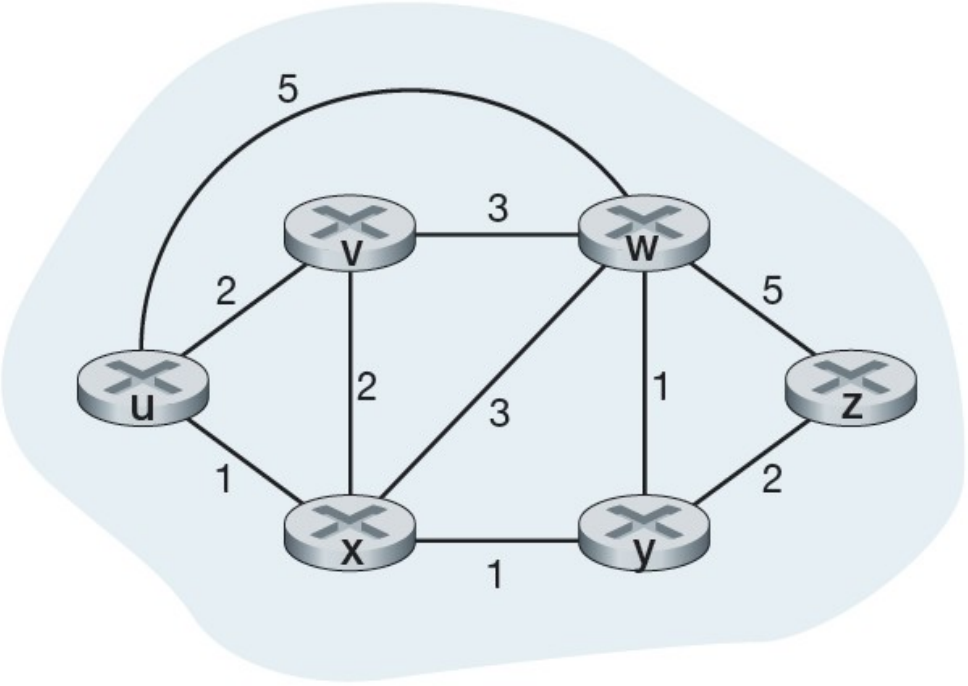
- **Equação de Bellman-Ford (programação dinâmica)**
- Define
- $\mathbf{d_x(y)}$ = custo do caminho de menor custo entre **x** e **y**
- Então
- $\mathbf{d_x(y) = \min \{c(x,v) + d_v(y) \}}$
- onde **min** é tomado entre todos os vizinhos **v** de **x**

O algoritmo de roteamento de vetor de distâncias (DV)

Claramente, $d_v(z) = 5$, $d_x(z) = 3$, $d_w(z) = 3$

A equação B-F diz:

$$d_u(z) = \min \{ c(u,v) + d_v(z), \; c(u,x) + d_x(z), \; c(u,w) + d_w(z) \}$$
$$= \min \{2 + 5, \; 1 + 3, \; 5 + 3\} = 4$$



O nó que leva ao custo mínimo é o próximo passo ao longo do caminho mais curto➜ tab. de encaminhamento

O algoritmo de roteamento de vetor de distâncias (DV)

- Algoritmo de vetor de distâncias (DV)

Para cada nó, x:

```
1   Inicialização:
2       para todos os destinos y em N:
3           Dx(y) = c(x,y) /* se y não é um vizinho então c(x,y) = ∞ */
4   para cada vizinho w
5       Dw(y) = ? para todos os destinos y em N
6   para cada vizinho w
7       envia vetor de distâncias Dx  = [Dx(y): y em N] para w
8
9   loop
10      espere (até que ocorra uma mudança no custo do enlace ao vizinho
11              w ou até a recepção de um vetor de distâncias do vizinho w)
12
13      para cada y em N:
14          Dx(y) = minv{c(x,v) + Dv(y)}
15
16      se Dx(y) mudou para algum destino y
17          envia vetor de distâncias Dx = [Dx(y): y em N] para todos os vizinhos
18
19  para sempre
```

O algoritmo de roteamento de vetor de distâncias (DV)

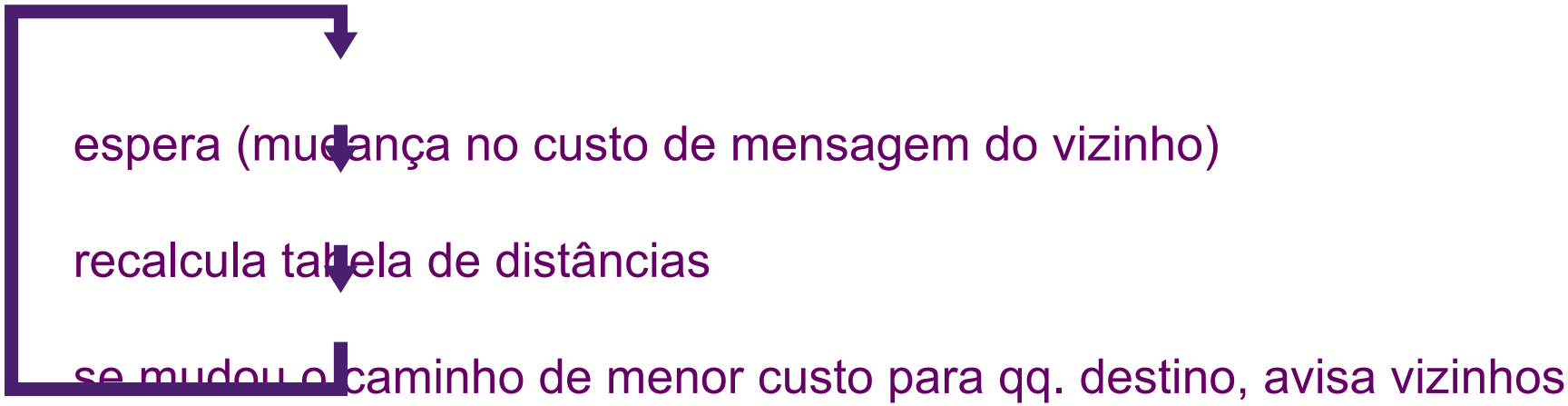
- **Ideia básica**:
  - Cada nó envia periodicamente o seu próprio vetor de distâncias estimado para os vizinhos
  - Quando um nó **x** recebe um novo VD estimado de um vizinho, ele atualiza o seu VD usando a eq. B-F:

    $$D_x(y) \leftarrow \min_v\{c(x,v) + D_v(y)\} \quad \text{p/ cada nó } y \in N$$

  - Sob condições mínimas, naturais, a estimativa $D_x(y)$ converge para o menor custo real $d_x(y)$

O algoritmo de roteamento de vetor de distâncias (DV)

- **Iterativo, assíncrono: cada iteração local causada por:**
  - mudança do custo do enlace local
  - mensagem do vizinho: mudança de caminho de menor custo para algum destino
- **Distribuído:**
  - cada nó avisa a seus vizinhos apenas quando muda seu caminho de menor custo para qualquer destino
    - os vizinhos então avisam a seus vizinhos, se for necessário
- **Cada nó**:



O algoritmo de roteamento DV

tabela nó x

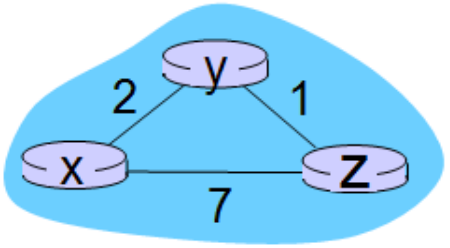
origem \ custo para	x	y	z
x	0	2	7
y	∞	∞	∞
z	∞	∞	∞

tabela nó y

origem \ custo para	x	y	z
x	∞	∞	∞
y	2	0	1
z	∞	∞	∞

tabela nó z

origem \ custo para	x	y	z
x	∞	∞	∞
y	∞	∞	∞
z	7	1	0



tempo

O algoritmo de roteamento DV

$D_x(y) = \min\{c(x,y) + D_y(y), c(x,z) + D_z(y)\} = \min\{2+0, 7+1\} = 2$

$D_x(z) = \min\{c(x,y) + D_y(z), c(x,z) + D_z(z)\} = \min\{2+1, 7+0\} = 3$

tabela nó x

custo para

origem	x	y	z
x	0	2	7
y	∞	∞	∞
z	∞	∞	∞

custo para

origem	x	y	z
x	0	2	3
y	2	0	1
z	7	1	0

tabela nó y

custo para

origem	x	y	z
x	∞	∞	∞
y	2	0	1
z	∞	∞	∞

custo para

origem	x	y	z
x	0	2	7
y	2	0	1
z	7	1	0

$D_z(x) = \min\{c(z,y) + D_y(x), c(z,x) + D_x(x)\} = \min\{1+2, 7+0\} = 3$

tabela nó z

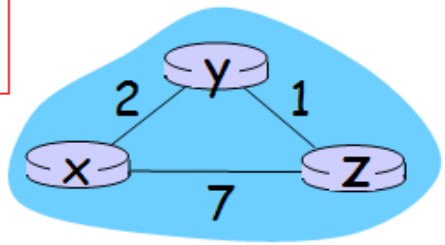
custo para

origem	x	y	z
x	∞	∞	∞
y	∞	∞	∞
z	7	1	0

custo para

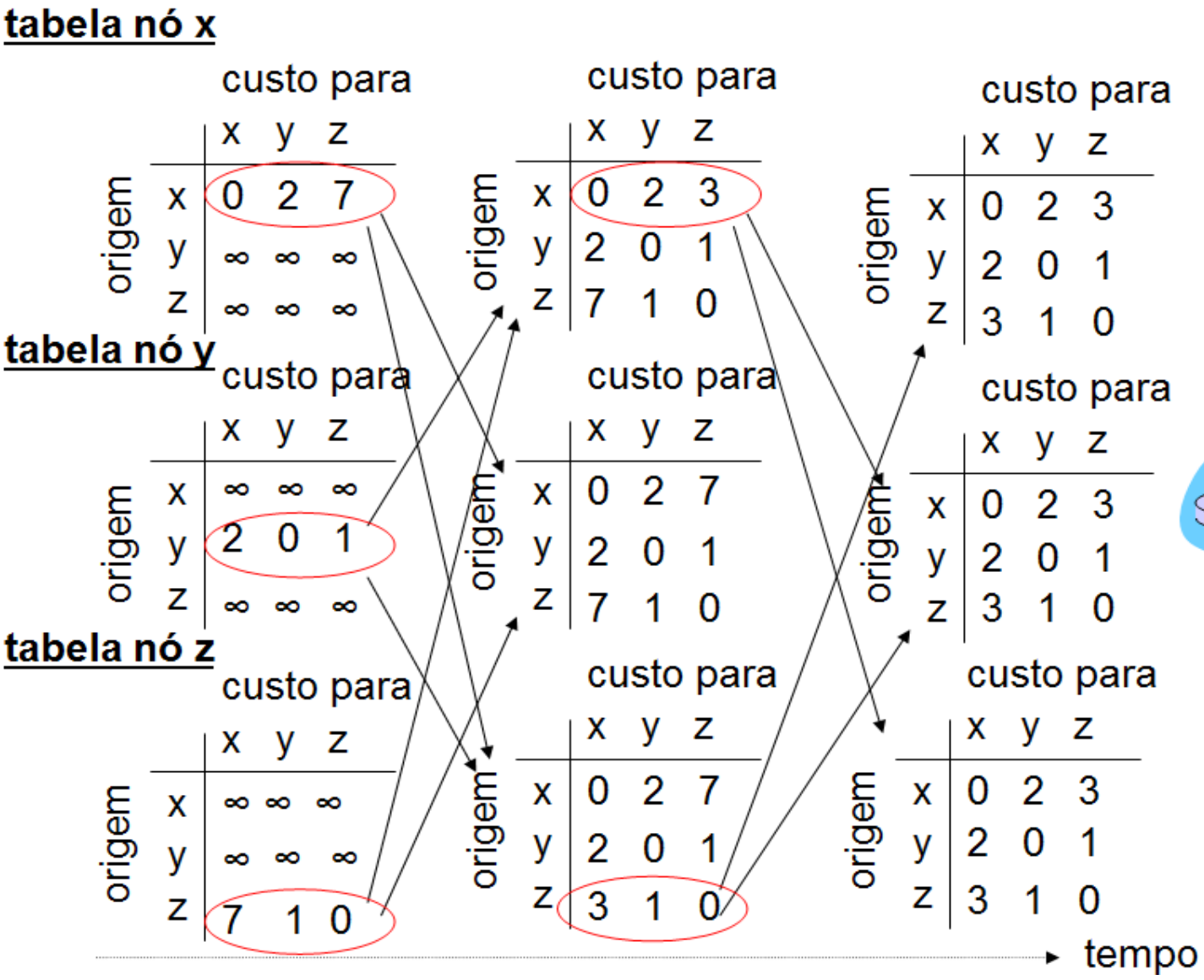
origem	x	y	z
x	0	2	7
y	2	0	1
z	3	1	0

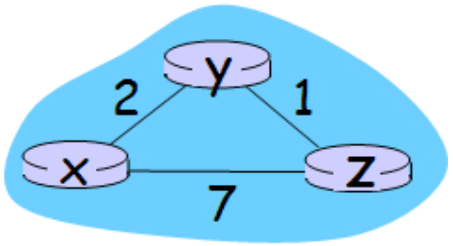
$D_z(y) = \min\{c(z,y) + D_y(y), c(z,x) + D_x(y)\} = \min\{1+0, 7+1\} = 1$



tempo

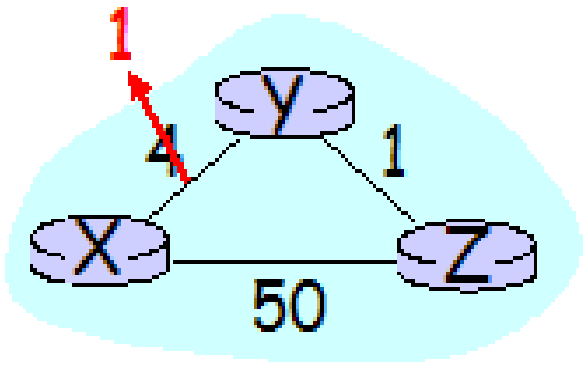
O algoritmo de roteamento DV





O algoritmo de roteamento de vetor de distâncias (DV)

- Mudança no custo dos enlaces:
  - nó detecta mudança no custo do enlace local
  - atualiza tabela de distâncias
  - se mudou o VD, avisa aos vizinhos



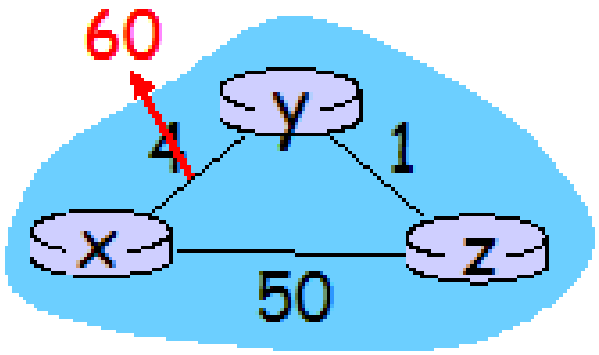
- No tempo t0, y detecta a mudança no custo do enlace, atualiza o seu VD e informa os vizinhos.

- No tempo t1, z recebe a atualização de y e atualiza a sua tabela. Computa o novo menor custo p/ x e envia o seu VD p/ os vizinhos.

- No tempo t2, y recebe a atualização de z e atualiza a sua tabela. Os custos mínimos de y não mudam e portanto y não envia nenhuma mensagem para z.

- **“boas notícias chegam logo”**

O algoritmo de roteamento de vetor de distâncias (DV)



- **Mudança no custo dos enlaces:**
  - boas notícias chegam logo
  - más notícias demoram para chegar - problema da "contagem ao infinito"!
  - 44 iterações antes do algoritmo estabilizar: veja texto
- **Reverso envenenado**:
  - Se z roteia via y p/ chegar a x:
  - z informa p/ y que sua distância p/ x é infinita (p/ que y não roteie p/ x via z)
  - será que isto resolve completamente o problema da contagem ao infinito?

O algoritmo de roteamento de vetor de distâncias (DV)

- **Comparação dos algoritmos EE e VD:**
- **Complexidade de mensagens**:
  - **EE**: com n nós, E enlaces, $O(n^E)$ mensagens enviadas
  - **VD**: trocar mensagens apenas entre vizinhos
    - varia o tempo de convergência
- **Rapidez de Convergência**:
  - **EE**: algoritmo $O(n^2)$ requer $O(n^E)$ mensagens
    - podem ocorrer oscilações
  - **VD**: varia tempo para convergir
    - podem ocorrer rotas cíclicas
    - problema de contagem ao infinito
- **Robustez**: o que acontece se houver falha do roteador?
  - **EE**:
    - nó pode anunciar valores incorretos de custo de enlace
    - cada nó calcula sua própria tabela
  - **VD**:
    - um nó VD pode anunciar um custo de caminho incorreto
    - a tabela de cada nó é usada pelos outros nós
    - um erro propaga pela rede

Roteamento hierárquico

- Neste estudo de roteamento fizemos uma **idealização**:
  - todos os roteadores idênticos
  - rede “não hierarquizada” (“flat”)
  - … **não é verdade, na prática!**

- **Escala**: com bem mais de 200 milhões de destinos:
  - impossível guardar todos destinos na tabela de rotas!
  - troca de tabelas de rotas afogaria os enlaces!

- **Autonomia administrativa**:
  - Internet = rede de redes
  - cada administrador de rede pode querer controlar roteamento em sua própria rede

Roteamento hierárquico

- Agregar roteadores em regiões, **“sistemas autônomos”** (**SAs**)

- roteadores no mesmo SA usam o mesmo protocolo de roteamento

  - protocolo de roteamento “intra-SA”
  - roteadores em SAs diferentes podem usar diferentes protocolos de roteamento intra-SA

- Roteador de borda

  - Enlace direto para roteador em outro SA

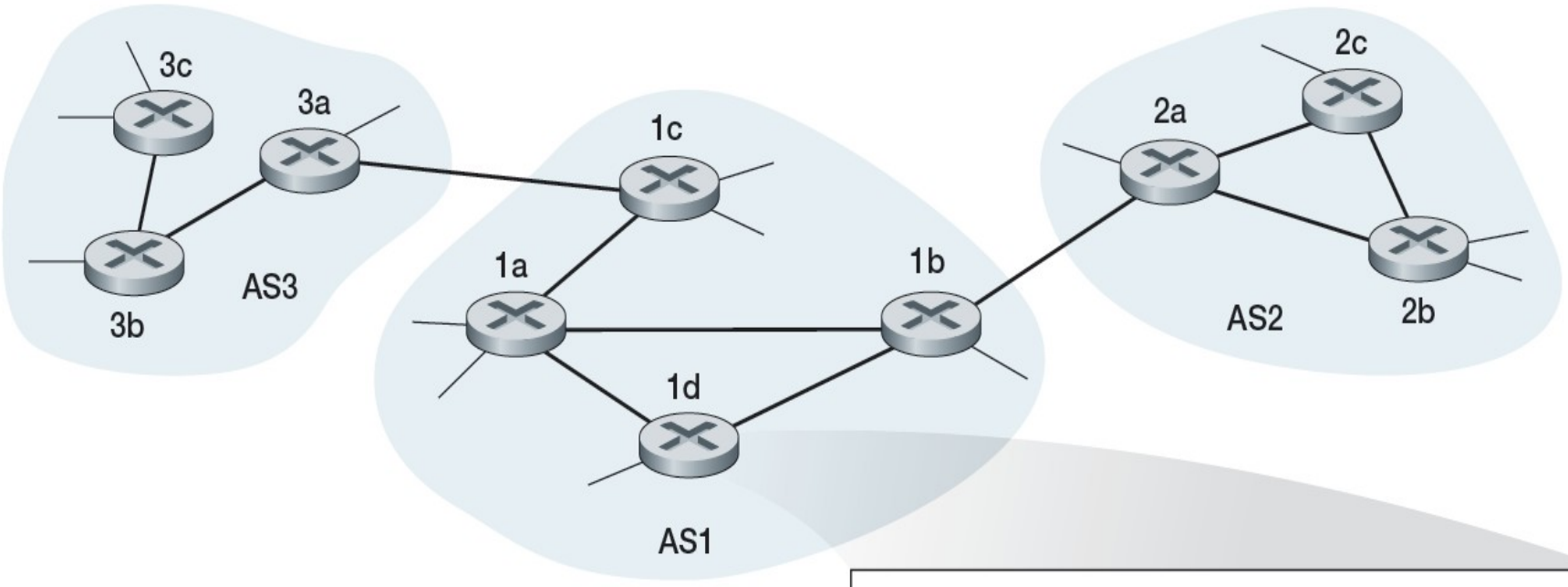
Roteamento hierárquico

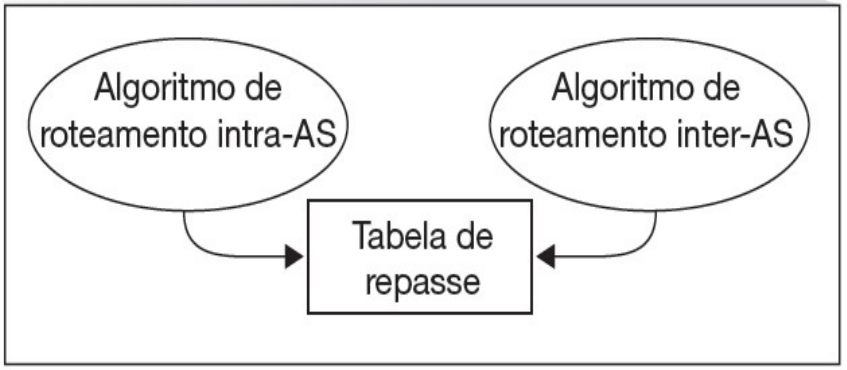
- Todos os roteadores dentro do mesmo **SA** rodam o mesmo algoritmo de roteamento e dispõem das informações sobre cada um dos outros.

- O algoritmo de roteamento que roda dentro de um SA é denominado um **protocolo de roteamento intrassistema autônomo**.

Roteamento hierárquico

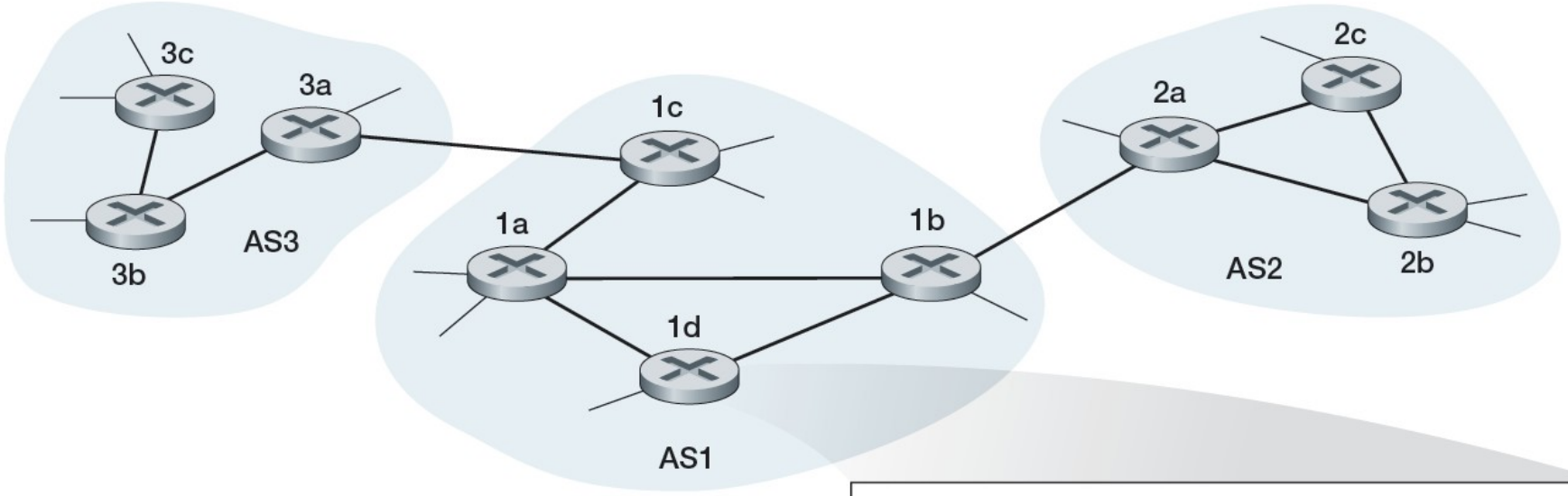
- Um exemplo simples com três SAs: SA1, SA2 E SA3:

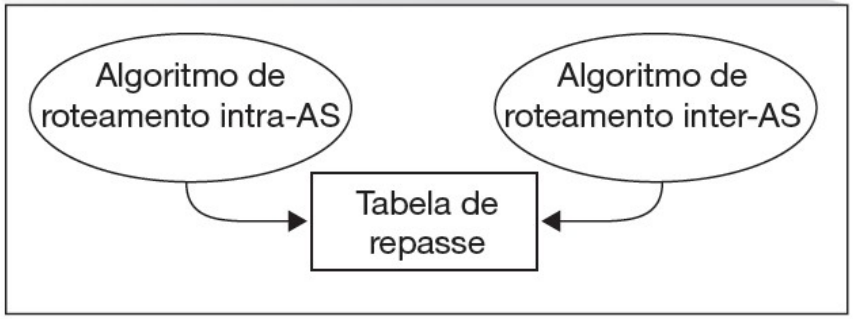




Roteamento hierárquico

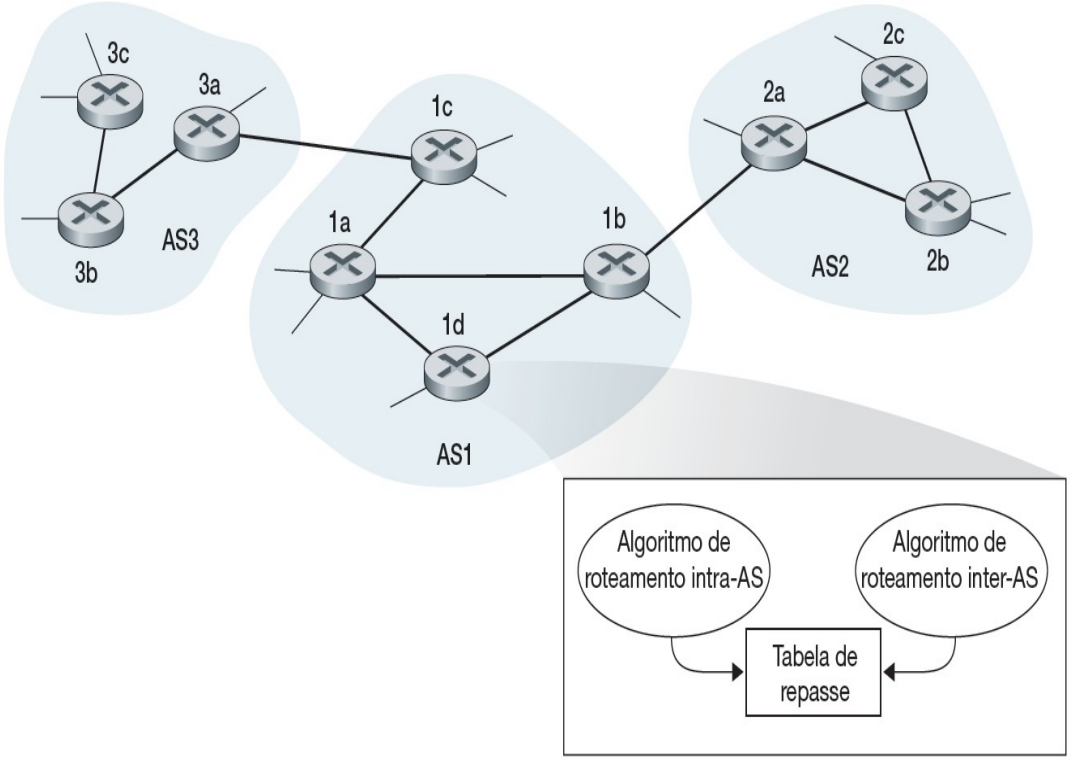
- Um exemplo simples com três SAs: SA1, SA2 E SA3:
  - Tab. de encaminhamento é configurada pelos algoritmos intra-SA e inter-SA
    - Intra-SA define entradas p/ dest. internos
    - Inter-SA e Intra-SA define entradas p/ dest. externos





Roteamento hierárquico

- Um exemplo simples com três SAs: SA1, SA2 E SA3:
  - Suponha que um roteador em SA1 recebe um datagrama cujo destino está fora de SA1
  - Roteador deveria encaminhar o pacote p/ um dos roteadores de borda, mas qual?



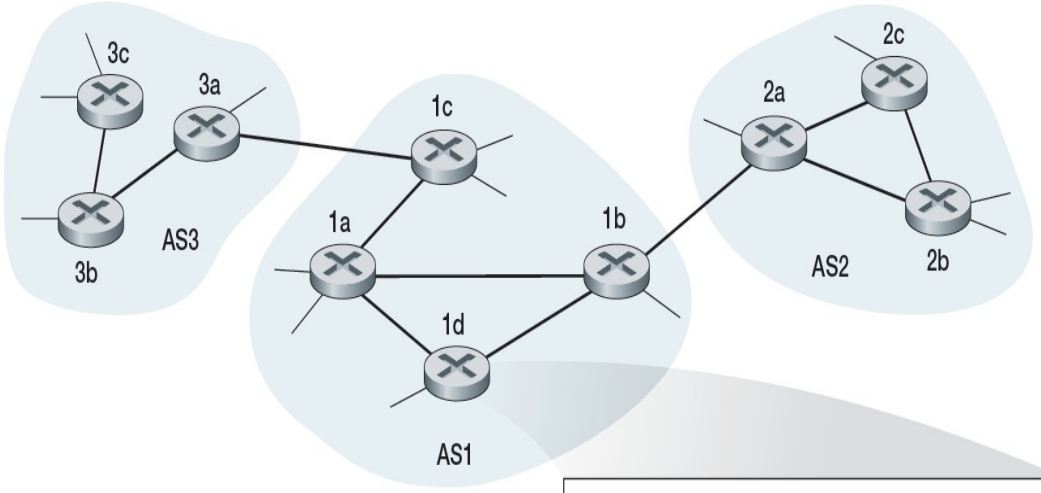
SA1 precisa:

1. aprender quais destinos são alcançáveis via SA2 e quais são alcançáveis via SA3
2. propagar estas info. de alcançabilidade para todos os roteadores em SA1

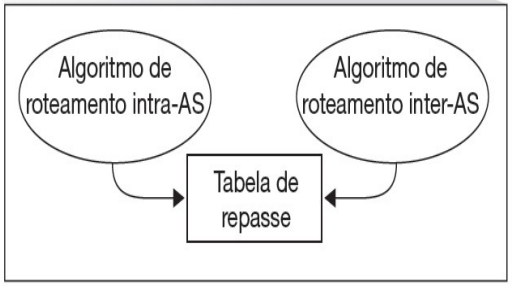
Tarefas do rot. inter-SA!

Exemplo: definindo a tabela de encaminhamento no roteador 1d

- Suponha que **SA1** aprende através do protocolo **inter-SA** que a sub-rede **x** é alcançável via **SA3** (rot. de borda **1c**) mas não via **SA2**.
- Protocolo **Inter-SA** propaga info. de alcançabilidade para todos os roteadores internos.

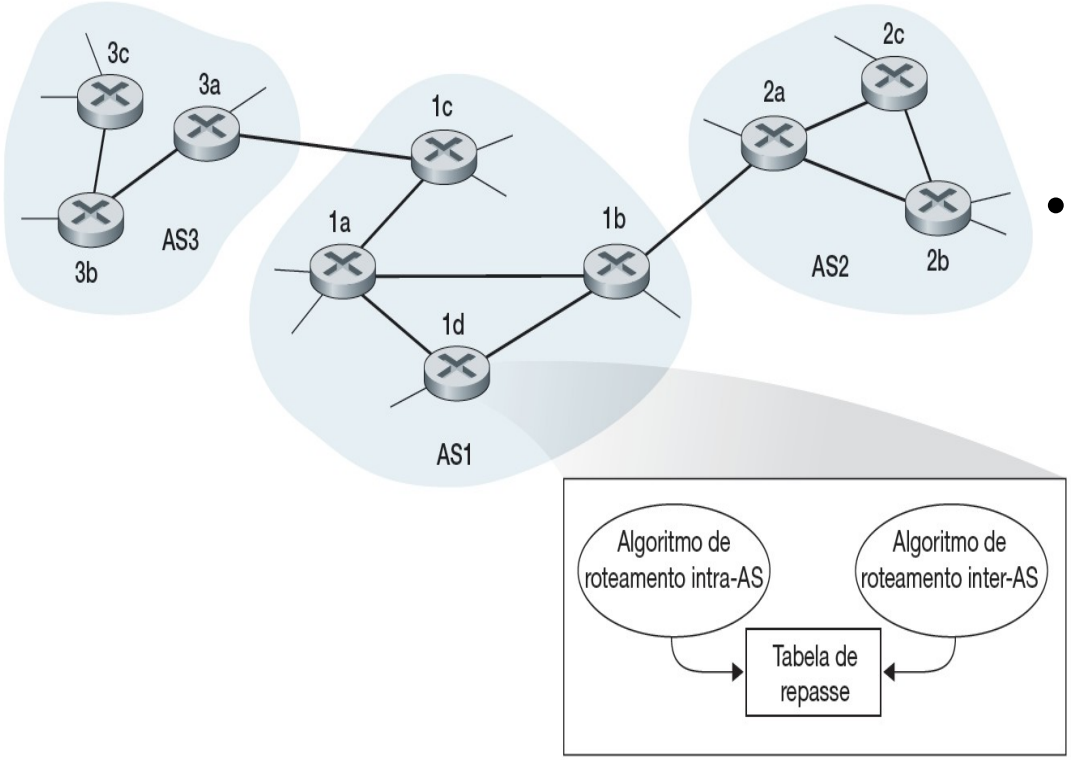


- Roteador **1d** determina através de info. de roteamento **intra-SA** que sua interface **I** está no caminho mínimo para **1c**.
- Coloca par **(x,I)** na tab. de encaminhamento.



Exemplo: escolhendo entre múltiplos SAs

- Suponha agora que **SA1** aprende através do protocolo **inter-SA** que a sub-rede **x** é alcançável via **SA3** e via **SA2**.
- Para configurar a tabela de encaminhamento, o roteador **1d** deve determinar para qual roteador de borda ele deve enviar pacotes com destino **x** .



- Isto também é tarefa do protocolo de roteamento **inter-SA**!
- **Roteamento batata quente (hot potato):** envia pacote para o roteador de borda mais próximo.

Sumário

4.1 – Introdução
4.2 – Redes de circuitos virtuais e de datagramas
4.3 – O que há dentro de um roteador?
4.4 – O Protocolo da Internet (IP): repasse e endereçamento na Internet
4.5 – Algoritmos de roteamento
4.6 – Roteamento na Internet
4.7 – Roteamento por difusão e para um grupo

Roteamento intra-SA na Internet

- Um protocolo de roteamento intra-SA é usado para determinar como é rodado o roteamento dentro de um sistema autônomo (SA).

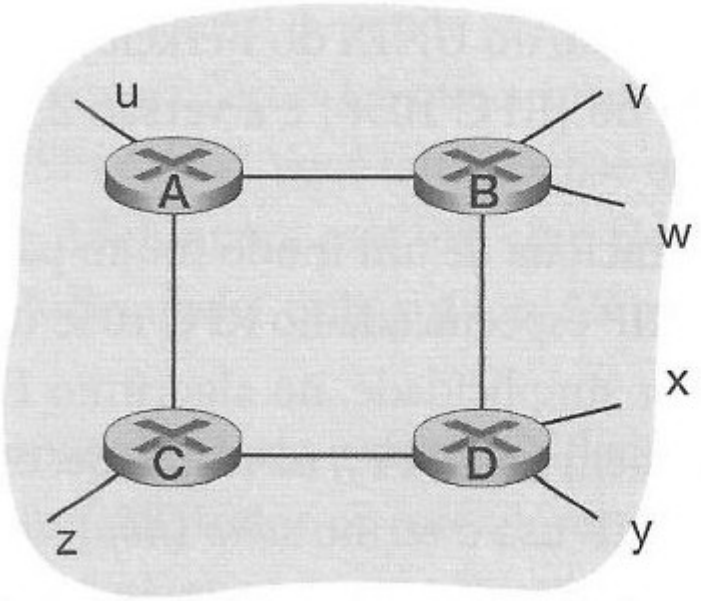
- Historicamente, dois protocolos de roteamento têm sido usados para roteamento dentro de um sistema autônomo na Internet:

1. o protocolo de informações de roteamento, **RIP** (*Routing Information Protocol*) e

2. o **OSPF** (*Open Shortest Path First*).

Roteamento intra-SA na Internet

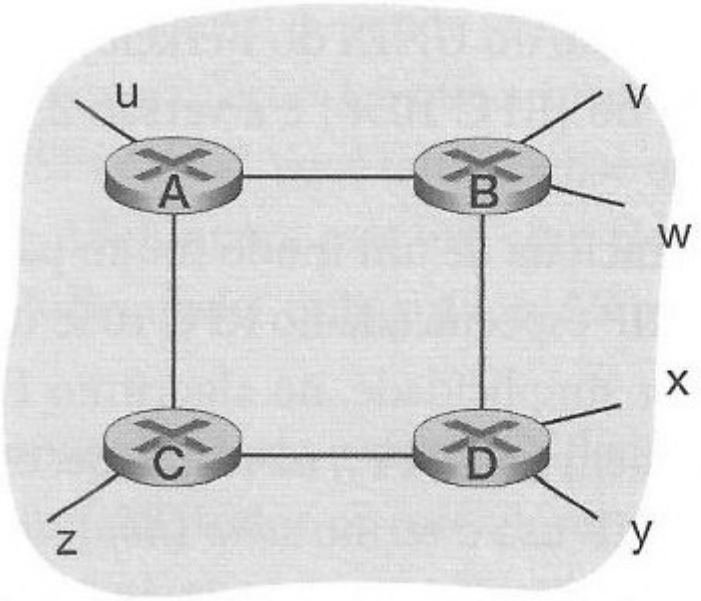
- **RIP (*Routing Information Protocol*)**
- **Anúncios RIP**
  - **Vetores de distâncias**: trocados a cada 30 seg via Mensagem de Resposta (também chamada de anúncio)
  - **Cada anúncio**: rotas para até 25 redes destino dentro do SA



Destino	Saltos
u	1
v	2
w	2
x	3
y	3
z	2

Roteamento intra-SA na Internet

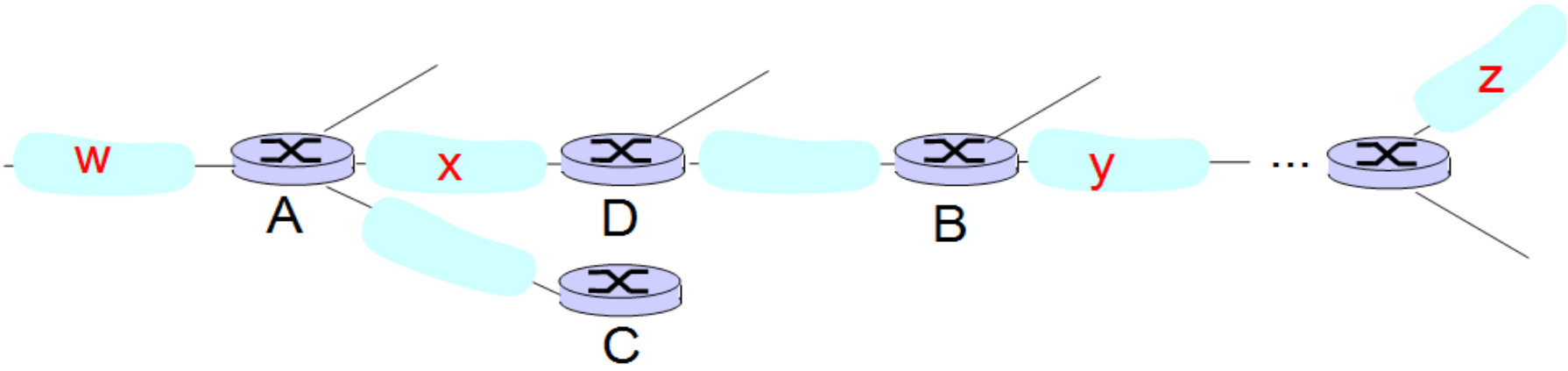
- **RIP (*Routing Information Protocol*)**
  - Algoritmo vetor de distâncias
  - Incluído na distribuição do BSD-UNIX em 1982
  - Métrica de distância: # de enlaces (máx = 15 enlaces)



Destino	Saltos
u	1
v	2
w	2
x	3
y	3
z	2

Roteamento intra-SA na Internet

- **Exemplo RIP (*Routing Information Protocol*)**



Rede Destino	Próximo Roteador	No. de enlaces ao destino
w	A	2
y	B	2
z	B	7
x	--	1
....	....	....

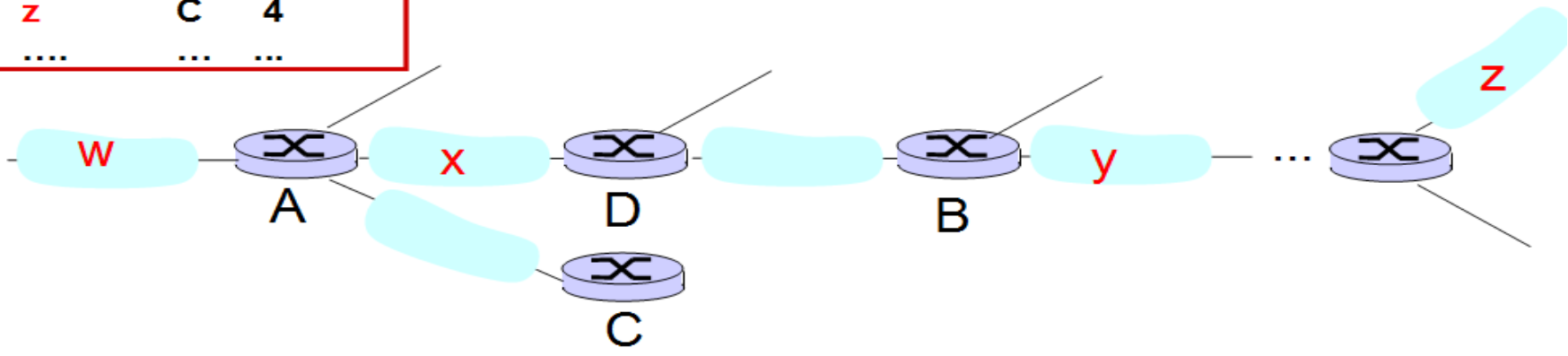
Tabela de rotas em D

Roteamento intra-SA na Internet

- **Exemplo RIP (*Routing Information Protocol*)**

Dest	Prox	Saltos
w	-	1
x	-	1
z	C	4
....	...	...

Anúncios de A para D



Rede Destino	Próximo Roteador	No. de enlaces ao destino
w	A	2
y	B	2
z	~~B~~ A	~~7~~ 5
x	--	1
....	....	....

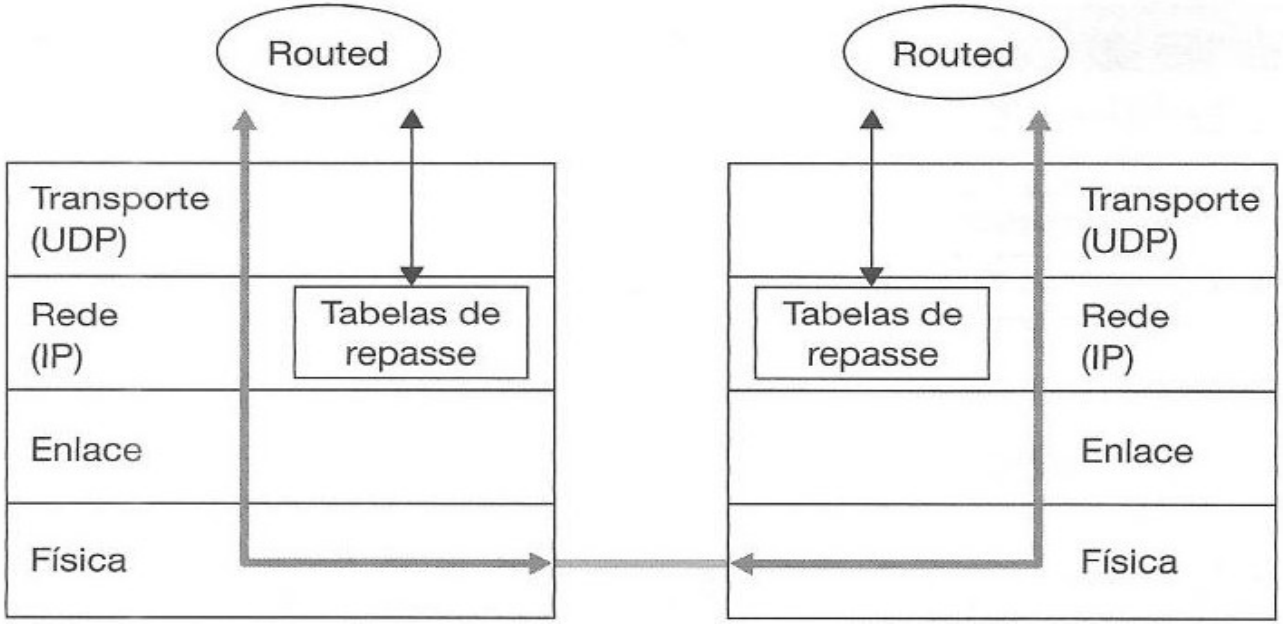
Tabela de rotas em D

Roteamento intra-SA na Internet

- **RIP: Falha e Recuperação de Enlaces**
  - Se não for recebido anúncio novo durante 180 seg --> vizinho/enlace declarados mortos
  - rotas via vizinho invalidadas
  - novos anúncios enviados aos vizinhos
  - na sua vez, os vizinhos publicam novos anúncios (se foram alteradas as suas tabelas)
  - informação sobre falha do enlace rapidamente propaga para a rede inteira
  - reverso envenenado usado para impedir rotas cíclicas (ping-pong) (distância infinita = 16 enlaces)

Roteamento intra-SA na Internet

- **RIP: Processamento de tabelas**
  - Tabelas de roteamento RIP gerenciadas por processo de nível de aplicação chamado *route-d* (*routing daemon*)
  - anúncios enviados em pacotes UDP, repetidos periodicamente



Roteamento intra-SA na Internet

- **OSPF (*Open Shortest Path First*)**
  - "***open***" (aberto): publicamente disponível
  - Usa algoritmo do Estado de Enlaces
  - disseminação de pacotes EE
  - mapa da topologia a cada nó
  - cálculo de rotas usando o algoritmo de Dijkstra
  - Anúncio de OSPF inclui uma entrada por roteador vizinho
  - Anúncios disseminados para SA inteiro (via inundação)
  - Carregados em mensagens OSPF diretamente sobre IP (ao invés de TCP ou UDP)

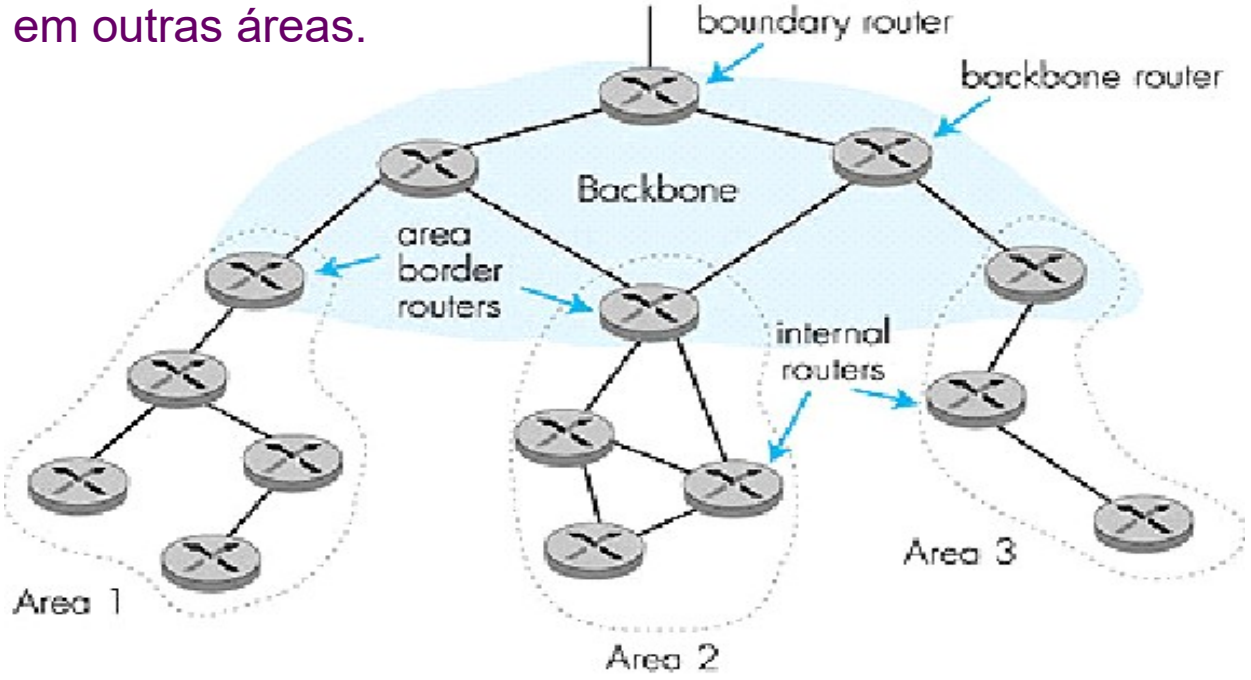
Roteamento intra-SA na Internet

- **OSPF: características "avançadas" (não existentes no RIP)**
  - **Segurança**: todas mensagens OSPF autenticadas (para impedir intrusão maliciosa)
  - Caminhos Múltiplos de custos iguais permitidos (o RIP permite e usa apenas uma rota)
  - Para cada enlace, múltiplas métricas de custo para TOS diferentes (p.ex, custo de enlace de satélite colocado como "baixo" para melhor esforço; "alto" para tempo real)
  - Suporte integrado para ponto a ponto e multiponto:
    - OSPF multiponto (MOSPF) usa mesma base de dados de topologia usado por OSPF
  - OSPF hierárquico em domínios grandes.

Roteamento intra-SA na Internet

- **OSPF: Hierárquico**

- **Hierarquia de dois níveis**: área local, ***backbone***.
- Anúncios de EE disseminados apenas na mesma área
- Cada nó possui topologia detalhada da área; apenas sabe a direção (caminho mais curto) para redes em outras áreas.



- **Roteador de fronteira de área**: "sumariza" distâncias às redes na sua própria área, anuncia a outros roteadores de fronteira de área.
- **Roteadores do *backbone***: realizam roteamento OSPF limitado ao *backbone*.
- **Roteadores de fronteira**: ligam a outros SAs.

Roteamento inter-SA: BGP

- O BGP oferece a cada SA meios de:

1. Obter de SAs vizinhos informações de alcançabilidade de sub-redes.

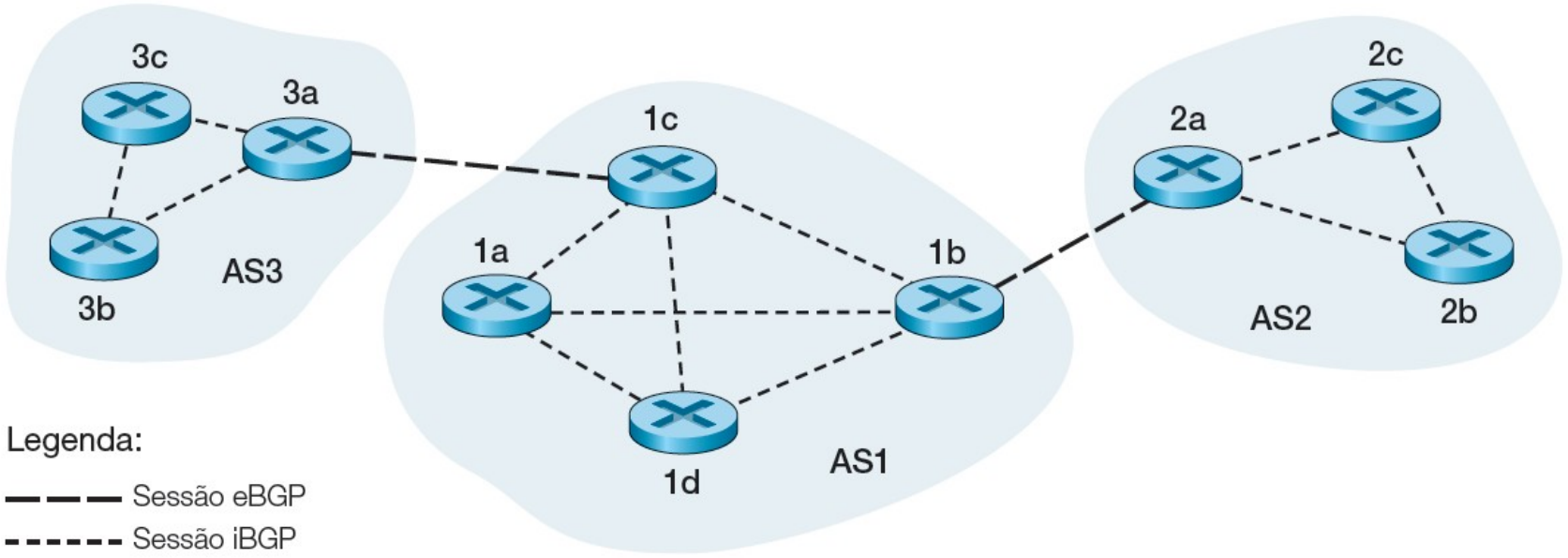
2. Propagar a informação de alcançabilidade a todos os roteadores internos ao SA.

3. Determinar rotas “boas” para sub-redes com base na informação de alcançabilidade e na política do SA.

- Permite que uma sub-rede anuncie a sua existência para o resto da Internet: ***“Estou aqui!”***

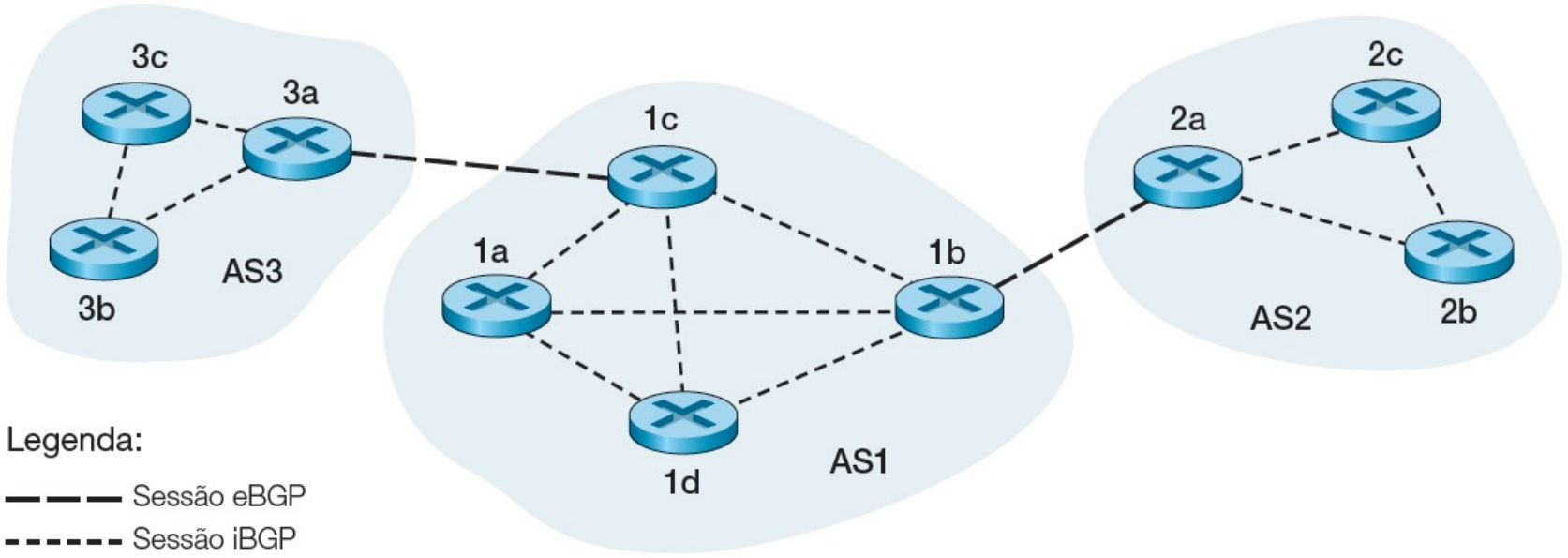
Roteamento inter-SA: BGP

- No BGP, pares de roteadores trocam informações de roteamento por conexões **TCP semipermanentes usando a porta 179**.
- Note que sessões BGP não correspondem a enlaces físicos.
- Quando um **SA2** anuncia um prefixo para **SA1**, **SA2** está prometendo que vai enviar àquele prefixo quaisquer datagramas destinados ao mesmo.
  - **SA2** pode agregar prefixos nos seus anúncios
- Sessões **eBGP** e **iBGP**



Roteamento inter-SA: BGP

- **Distribuindo informação de alcançabilidade:**
  - Com a sessão eBGP **3a-para-1c**, **SA3** envia informação de alcançabilidade de prefixos para **SA1**.
  - **1c** pode usar iBGP para distribuir esta nova informação de alcance de prefixo para todos os roteadores em **SA1**.
  - **1b** pode então re-anunciar a nova informação de alcance para **SA2** através da sessão eBGP **1b-para-2a**.
  - Quando um roteador aprende sobre um novo prefixo, ele cria uma entrada para o prefixo na sua tabela de encaminhamento.



Roteamento inter-SA: BGP

- O BGP permite que cada SA conheça quais destinos podem ser alcançados por meio de seus SAs vizinhos.
- No BGP, um sistema autônomo é identificado por seu **número de sistema autônomo** (NSA) globalmente exclusivo [RFC 1930].
- Quando um roteador anuncia um prefixo para uma sessão BGP, inclui vários atributos BGP juntamente com o prefixo.
- O BGP usa eBGP e iBGP para distribuir rotas a todos os roteadores dentro de SAs.

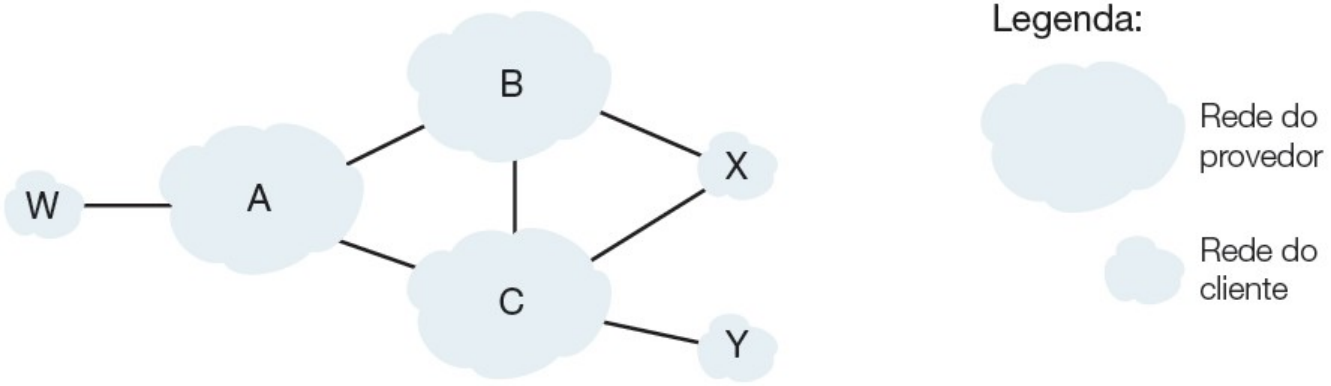
Roteamento inter-SA: BGP

- Mensagens BGP trocadas usando TCP.
- Mensagens BGP:
  - **OPEN**: abre conexão TCP ao roteador par e autentica remetente
  - **UPDATE**: anuncia caminho novo (ou retira velho)
  - **KEEPALIVE** mantém conexão viva na ausência de UPDATES; também reconhece pedido OPEN
  - **NOTIFICATION**: reporta erros na mensagem anterior; também usada para fechar conexão

Roteamento inter-AS: BGP

- Um cenário BGP simples

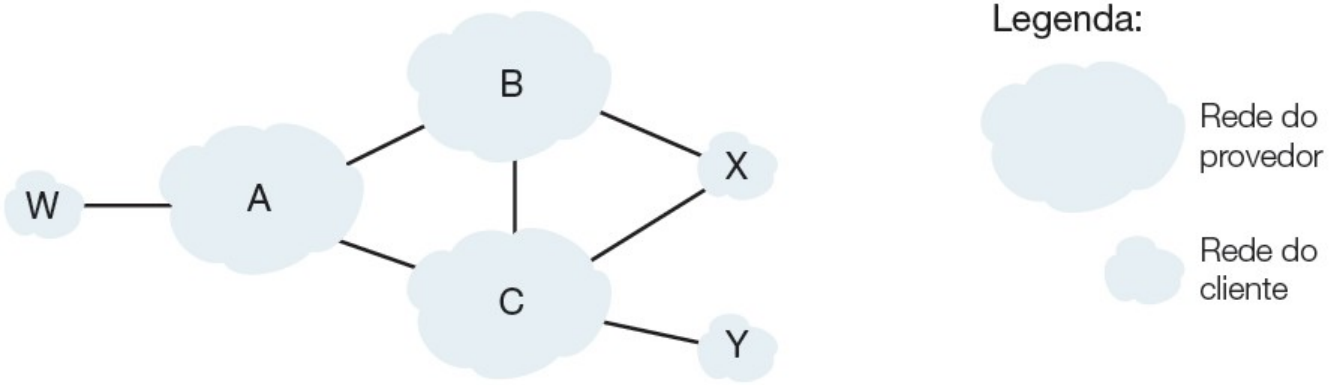
- **A,B,C** são redes de **provedores**
- **X,W,Y** são **clientes** (das redes de provedores)
- **X com duas interfaces**: conectadas a **duas redes**
  - **X** não quer rotear de **B para C**
  - ... então **X não vai anunciar para B a rota para C**



Roteamento inter-AS: BGP

- Um cenário BGP simples (**outro**)

- **A** anuncia para **B** o caminho **AW**
- **B** anuncia para **X** o caminho **BAW**
- **Deveria B anunciar para C o caminho BAW?**
  - **Nem pensar! B** não obtém "rendimento" pelo roteamento **CBAW**, já que nem **W** ou **C** são clientes de **B**
  - **B** quer forçar **C** a rotear para **W** via **A**
  - **B** quer rotear apenas para/dos seus clientes!

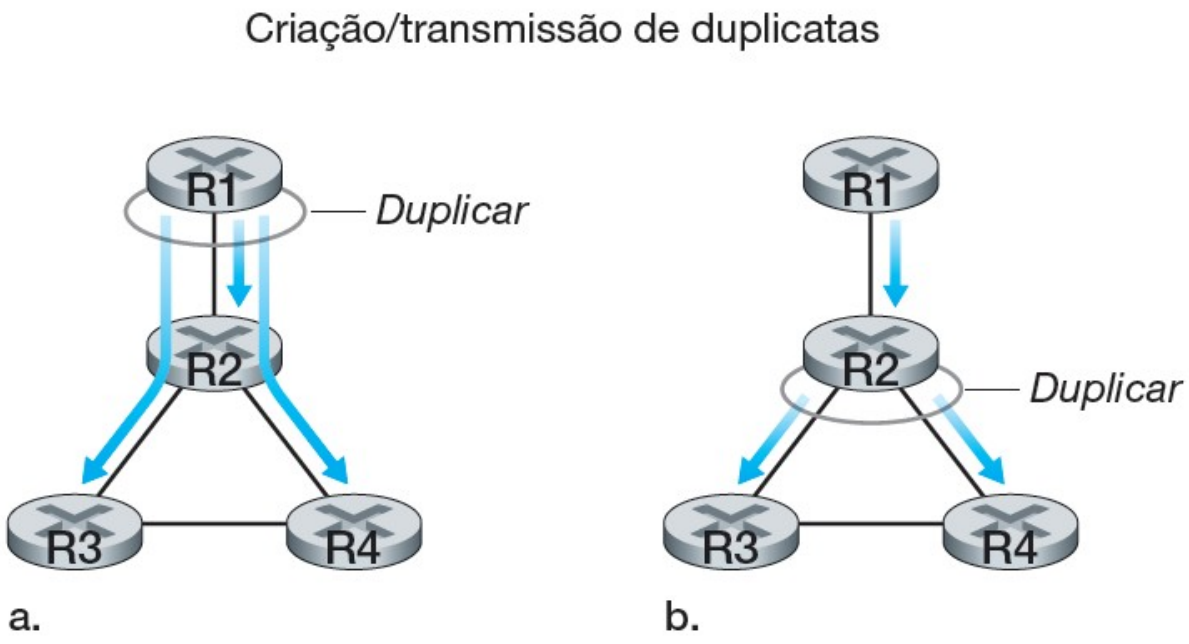


Sumário

4.1 – Introdução
4.2 – Redes de circuitos virtuais e de datagramas
4.3 – O que há dentro de um roteador?
4.4 – O Protocolo da Internet (IP): repasse e endereçamento na Internet
4.5 – Algoritmos de roteamento
4.6 – Roteamento na Internet
4.7 – Roteamento por difusão e para um grupo

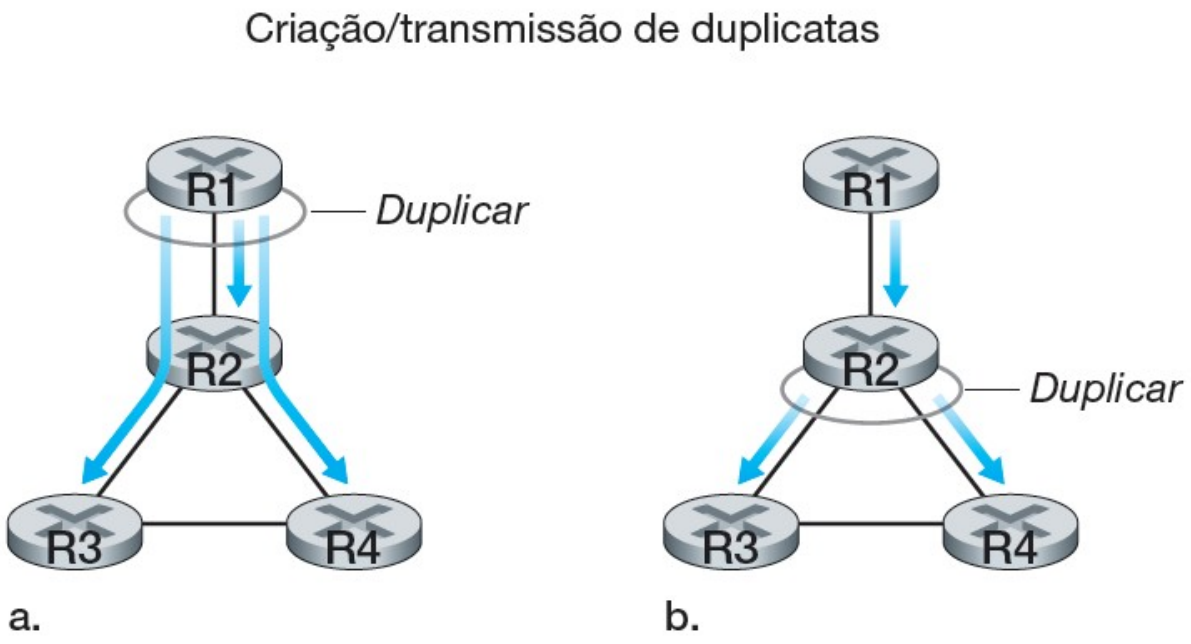
Algoritmos de roteamento por difusão (*broadcast*)

- Talvez o modo mais direto de conseguir comunicação por difusão seja o nó remetente enviar uma cópia separada do pacote para cada destino.



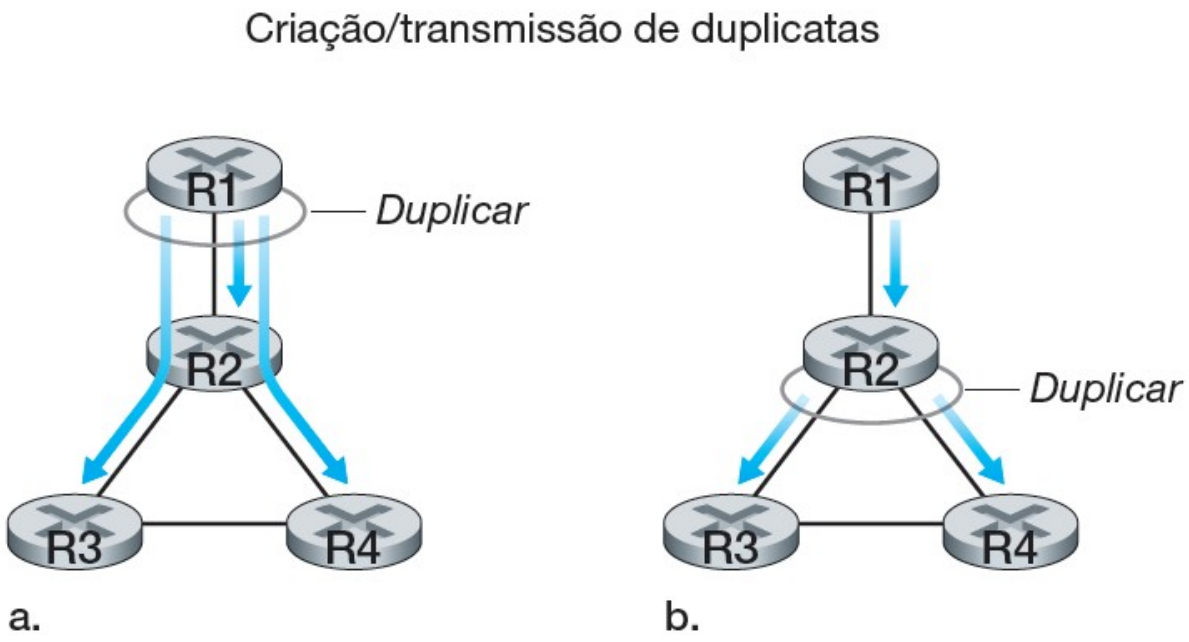
Algoritmos de roteamento por difusão (*broadcast*)

- A técnica mais óbvia para conseguir difusão é uma abordagem de inundação na qual o nó de origem envia uma cópia do pacote a todos os seus vizinhos.



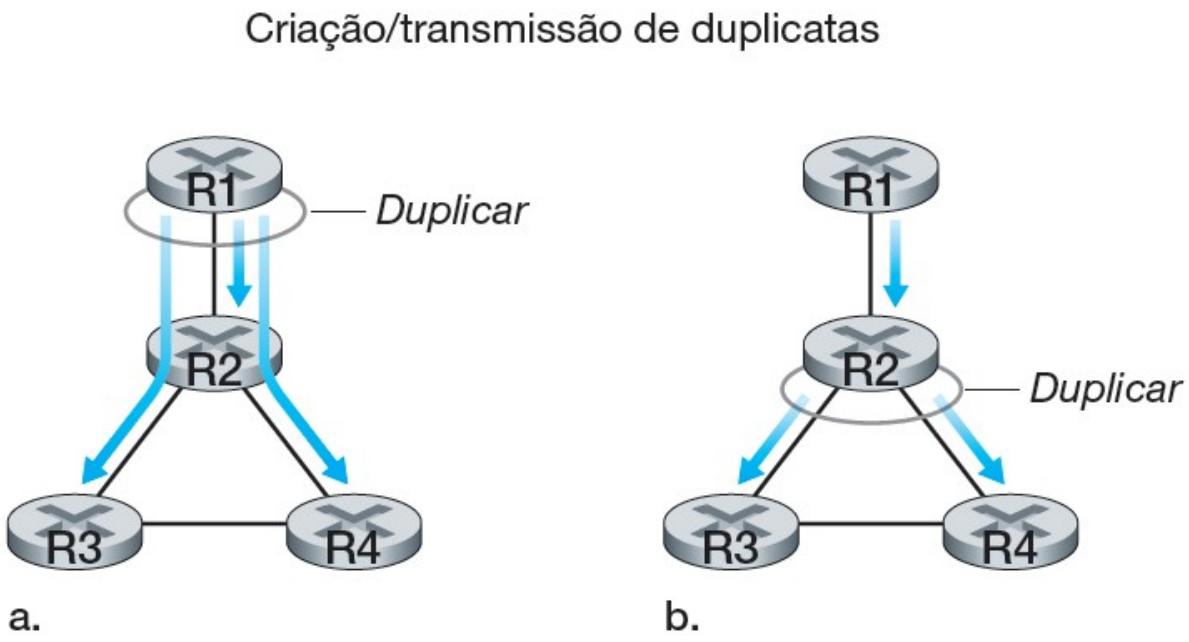
Algoritmos de roteamento por difusão (*broadcast*)

- Na inundação controlada por número de sequência, um nó de origem coloca seu endereço, bem como um número de sequência de difusão em um pacote de difusão e então envia o pacote a todos os seus vizinhos.



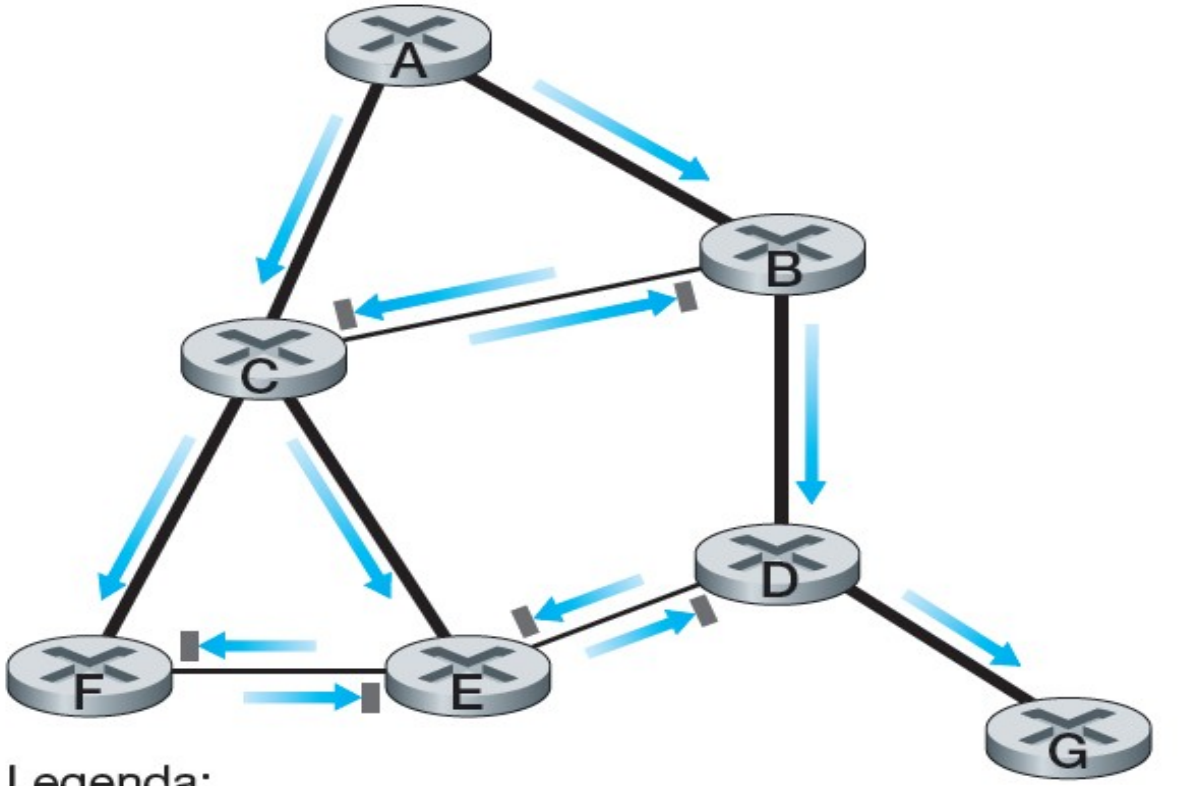
Algoritmos de roteamento por difusão (*broadcast*)

- Duplicação na origem versus duplicação dentro da rede



Algoritmos de roteamento por difusão (*broadcast*)

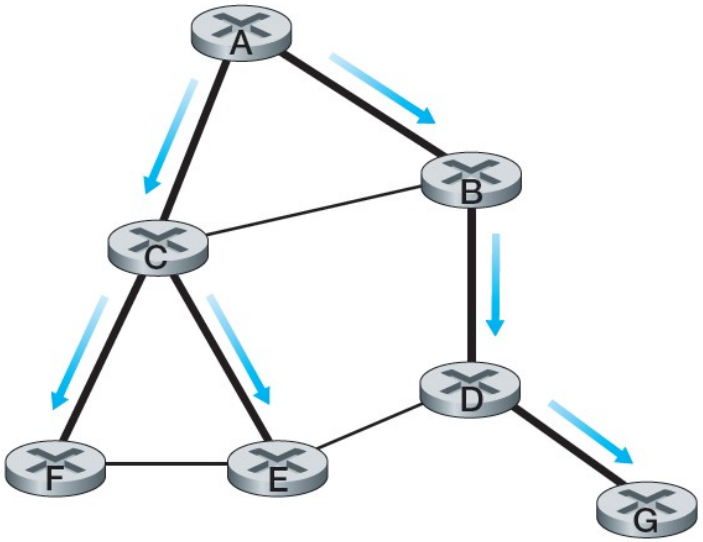
- Repasse pelo caminho inverso (inundação controlada)



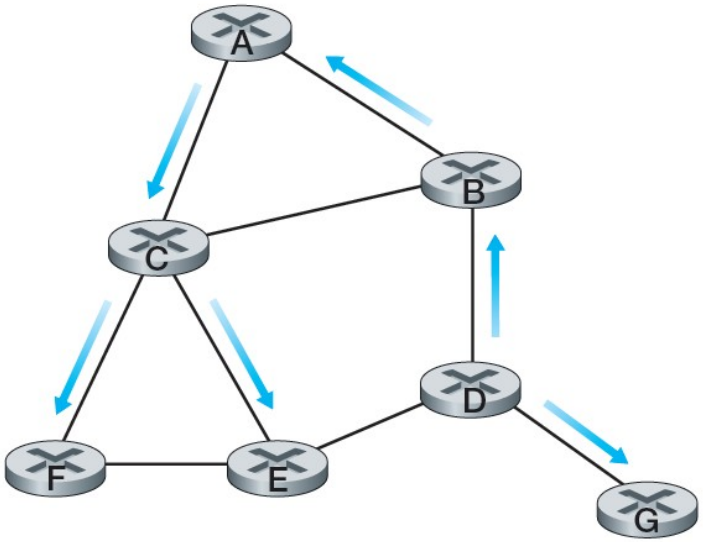
Algoritmos de roteamento por difusão (*broadcast*)

- Assim, outra abordagem para o fornecimento de difusão é os nós da rede construírem uma ***spanning tree***, em primeiro lugar.

- Na **abordagem de nó central** da construção de uma *spanning tree*, é definido um nó central.



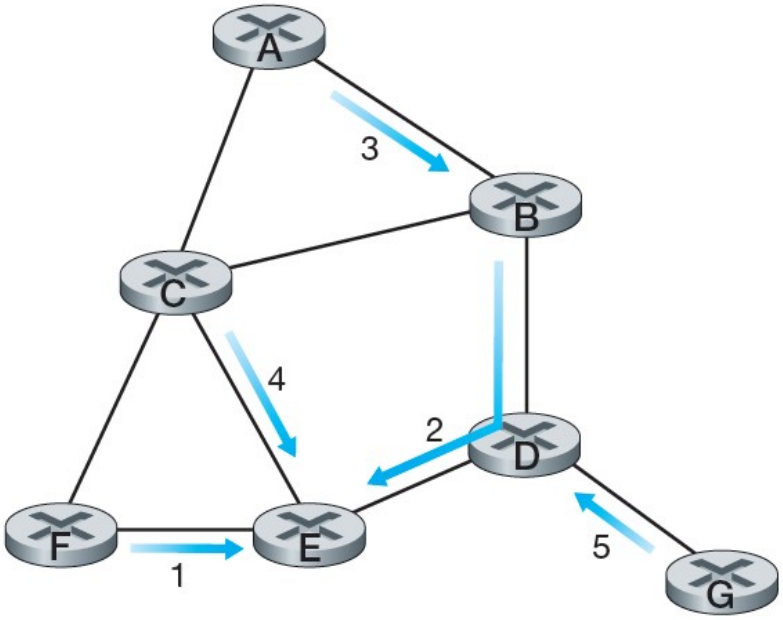
a. Difusão iniciada em *A*



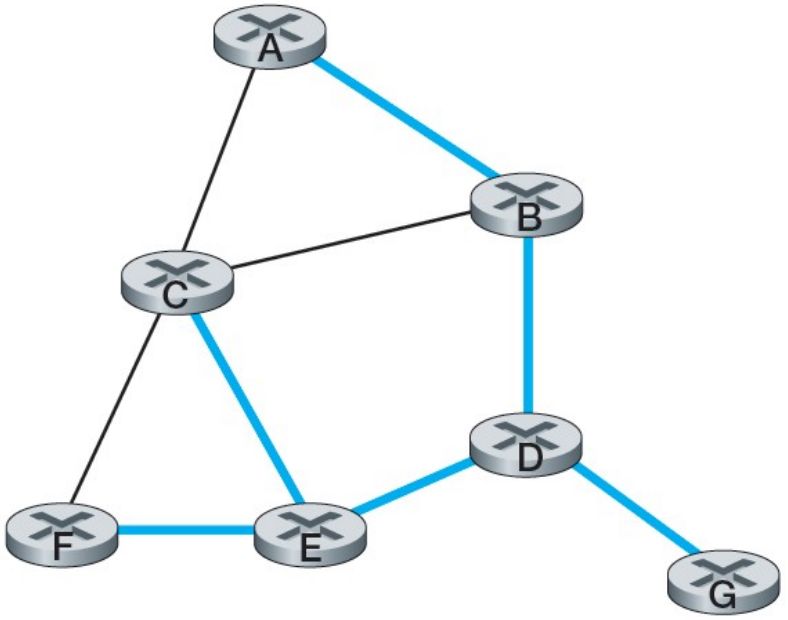
b. Difusão iniciada em *D*

Algoritmos de roteamento por difusão (*broadcast*)

- Construção de uma *spanning tree* com centro
  - Cada nó envia mensagem de junção ponto-a-ponto (unicast) para o nó central
    - Mensagem encaminhada até que chegue em um nó já pertencente à árvore geradora



a. Construção da *spanning tree* passo a passo



b. *Spanning tree* construída

Serviço para um grupo (*multicast*)

- Na comunicação para um grupo, enfrentamos imediatamente dois problemas:

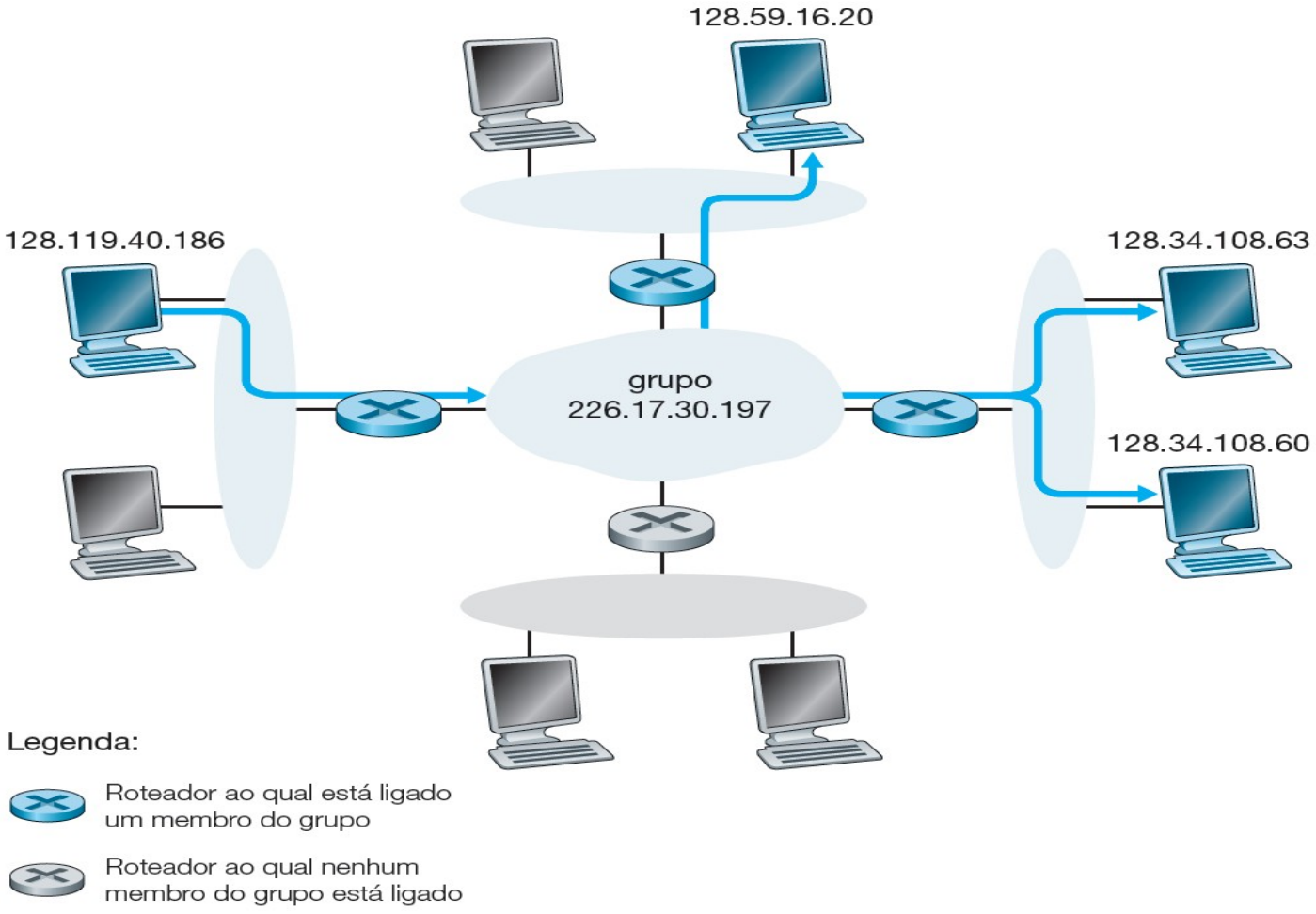
1. como identificar os destinatários de um pacote desse tipo e
2. como endereçar um pacote enviado a um desses destinatários.

•. Um pacote para um grupo é endereçado usando **endereço indireto**.

•. O grupo de destinatários associados a um endereço classe D é denominado **grupo *multicast***.

Serviço para um grupo (*multicast*)

- O serviço para um grupo: um datagrama endereçado ao grupo é entregue a todos os membros do grupo



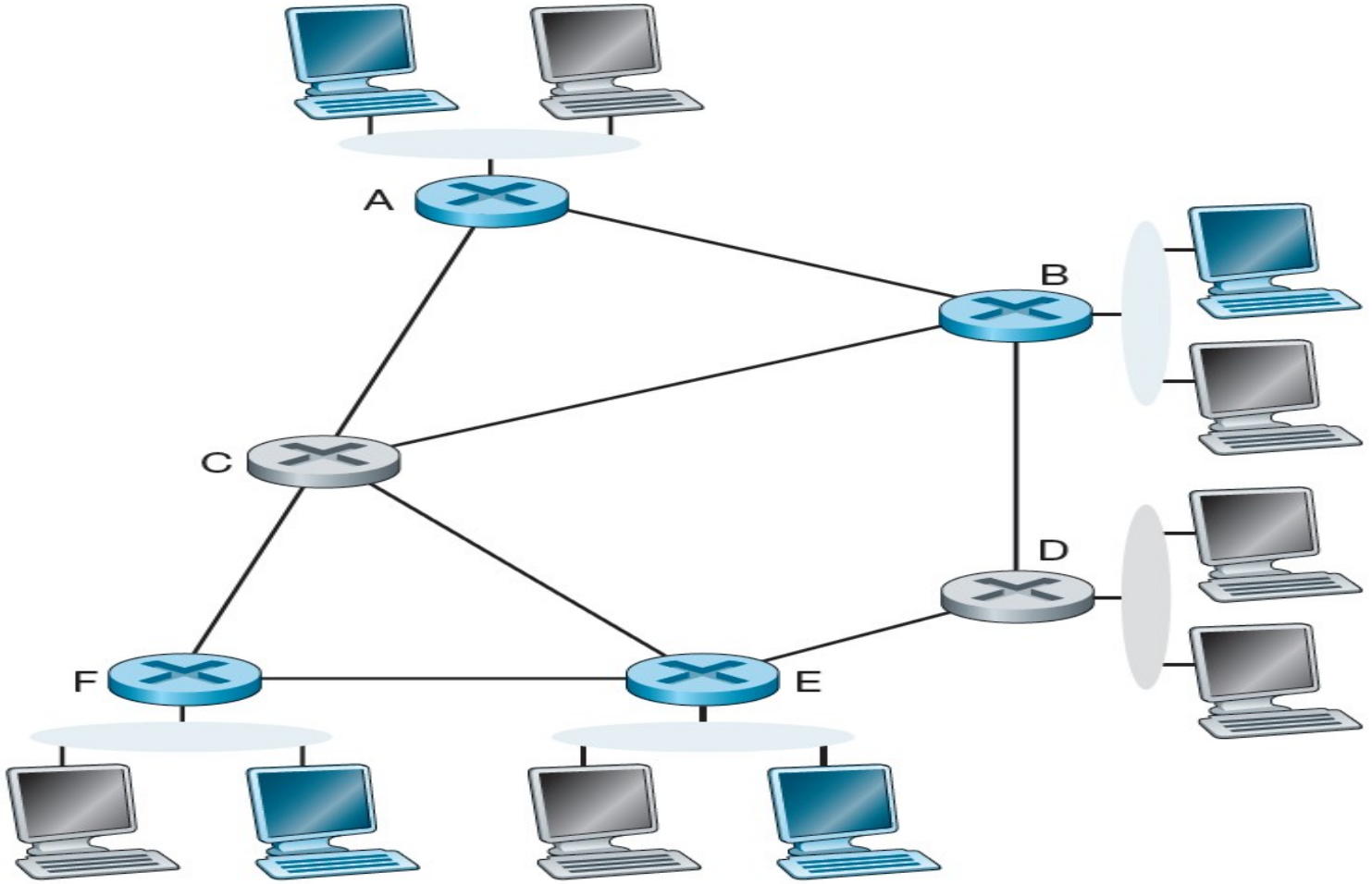
Serviço para um grupo (*multicast*)

- Os dois componentes de grupo da camada de rede: IGMP e protocolos de roteamento para um grupo



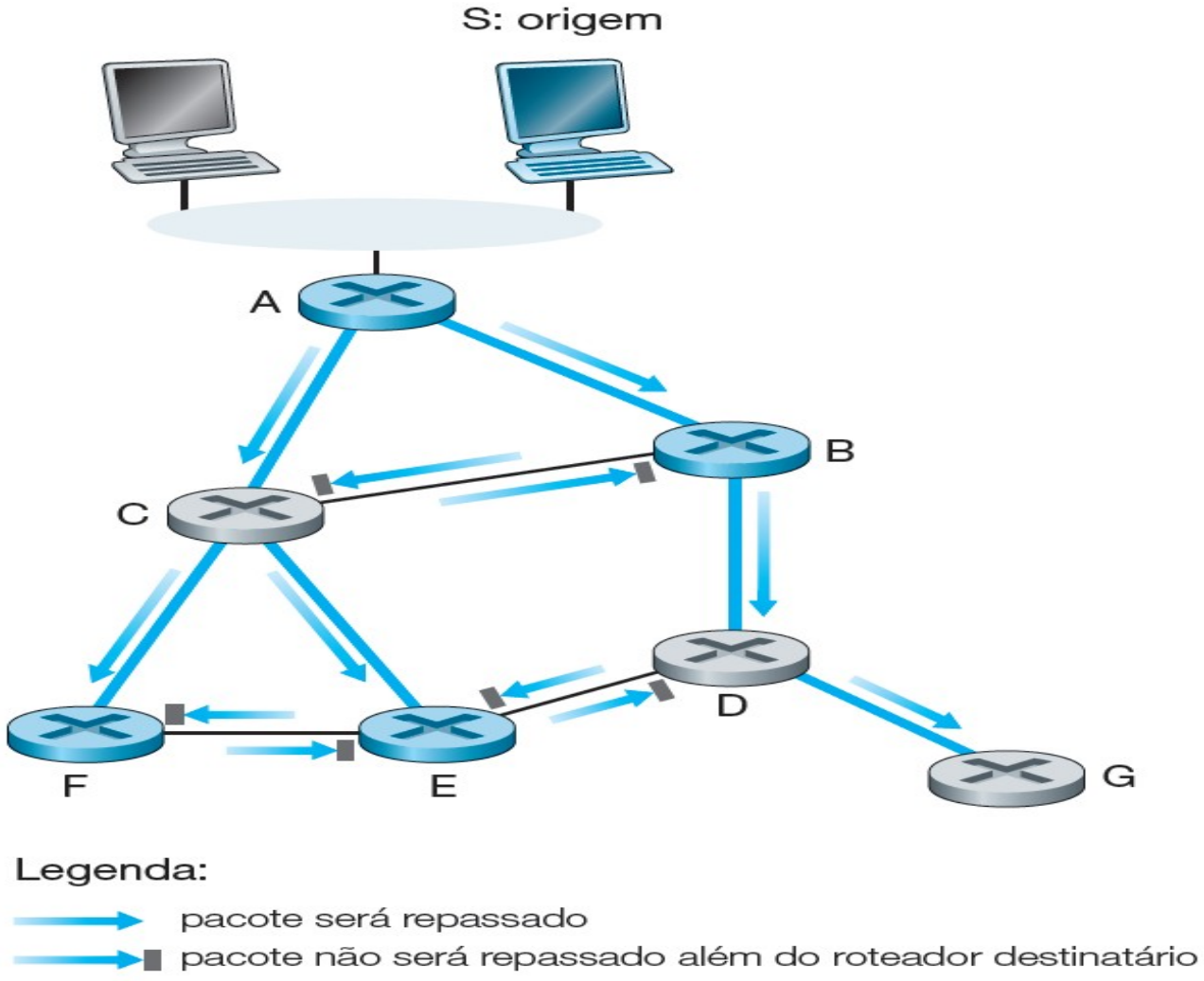
Serviço para um grupo (*multicast*)

- Hospedeiros do grupo, seus roteadores conectados e outros roteadores



Serviço para um grupo (*multicast*)

- Repasse pelo caminho inverso, no caso do serviço para um grupo



Serviço para um grupo (*multicast*) na Internet

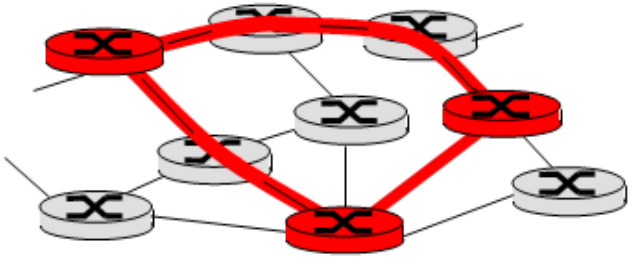
- **DVMRP**: *distance vector multicast routing protocol*, RFC1075
  - **inundação e poda**: envio pelo caminho reverso (RPF), árvore baseada na fonte
  - árvore RPF baseada em tabelas de roteamento próprias do DVMRP, construídas por meio da comunicação entre roteadores DVMRP
  - nada assume sobre o roteamento *unicast* subjacente
  - datagrama inicial para o grupo mcast é inundado por todo lugar via RPF
  - **roteadores sem membros**: mensagens de poda para cima

Serviço para um grupo (*multicast*) na Internet

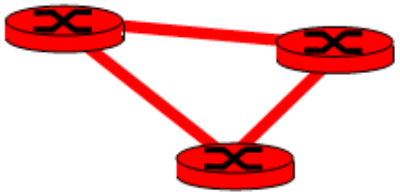
- **DVMRP**: continuando…
  - **estado *soft***: roteador DVMRP “esquece” periodicamente (1 min.) que ramos estão podados:
    - dados **mcast** novamente fluem pelos ramos não podados
    - **roteador de baixo**: refaz a poda ou continua a receber dados
  - **roteadores podem rapidamente se enxertar na árvore**
    - seguindo junção IGMP na folha
  - **considerações finais**
    - comumente implementado em roteadores comerciais
    - roteamento Mbone feito através do DVMRP

Serviço para um grupo (*multicast*) na Internet

- **DVMRP**: continuando…
  - Q: Como conectar “ilhas” de roteadores multicast em um “oceano” de roteadores unicast?



Topologia física



Topologia lógica

  - datagrama mcast encapsulado dentro de um datagrama “normal” (sem endereço multicast)
  - datagrama IP normal enviado através de um “túnel” via IP unicast regular para o roteador mcast receptor
  - roteador mcast receptor desencapsula para obter datagrama mcast

Serviço para um grupo (*multicast*) na Internet

- **PIM: Protocol Independent Multicast**
  - não depende de nenhum algoritmo de roteamento unicast subjacente (trabalha com todos)
  - Dois cenários de distribuição multicast diferentes:
    - **Denso**:
      - membros do grupo densamente empacotados, em "estreita proximidade"
      - maior disponibilidade de banda
    - **Esparso**:
      - Em redes com número de membros do grupo pequeno em relação ao número de redes interconectadas
      - membros do grupo "amplamente dispersos"
      - menor disponibilidade de banda

Serviço para um grupo (*multicast*) na Internet

- **PIM: Protocol Independent Multicast**
  - Consequências da Dicotomia Esparso-Denso:
  - **Denso**:
    - participação dos roteadores nos grupos assumida até que os roteadores se podem explicitamente
    - construção da árvore mcast ditada pelos dados (e.x., RPF)
  - **Esparso**:
    - sem participação até que os roteadores se juntem explicitamente
    - construção da árvore mcast ditada pelos receptores (e.x., baseada em centro)

Capítulo 4 - FIM